

Week 3

Linear Models

Nebius Academy

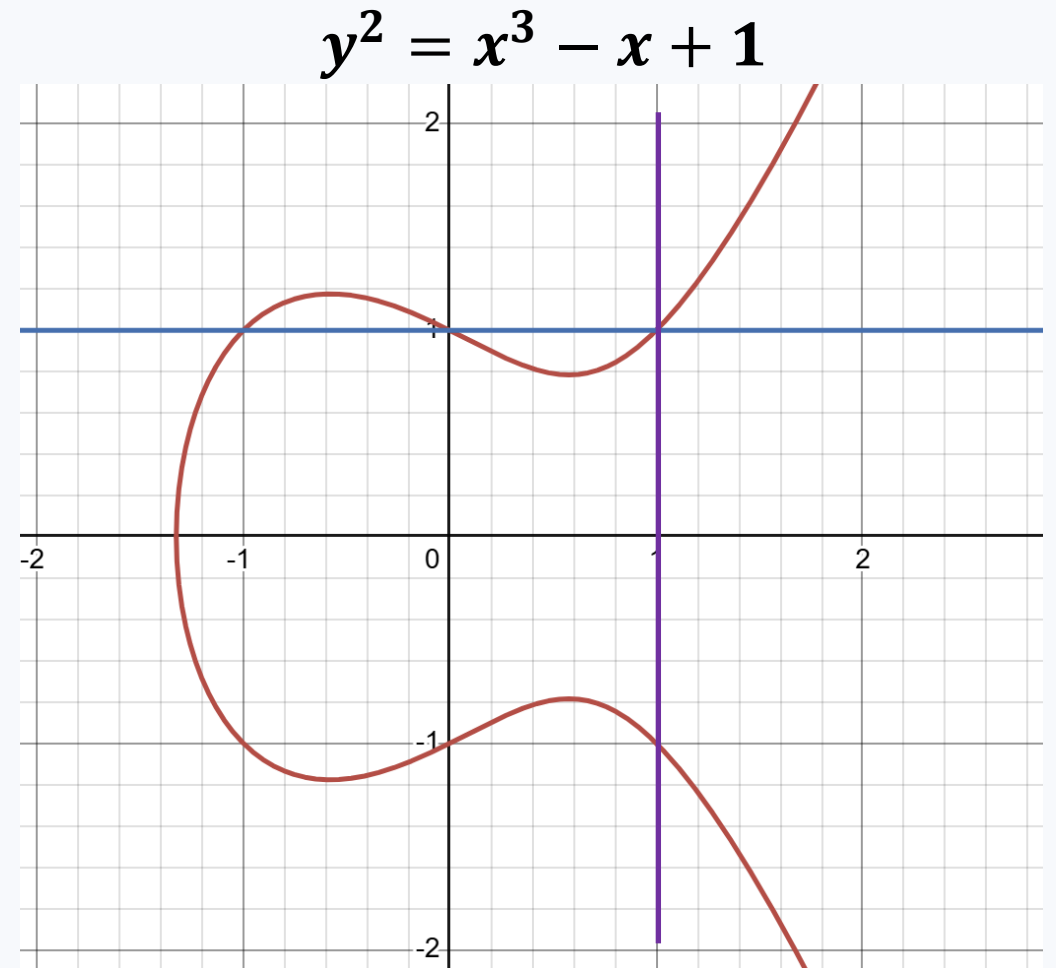


A motivational example:
Murmurations of elliptic
curves

Elliptic curves

An **elliptic curve** over a field $\mathbb{K}$ is a smooth projective cubic

Over $\text{char}(\mathbb{K}) \neq 2, 3$ can be described as $y^2 = x^3 + ax + b$ in $\mathbb{K}^2$.

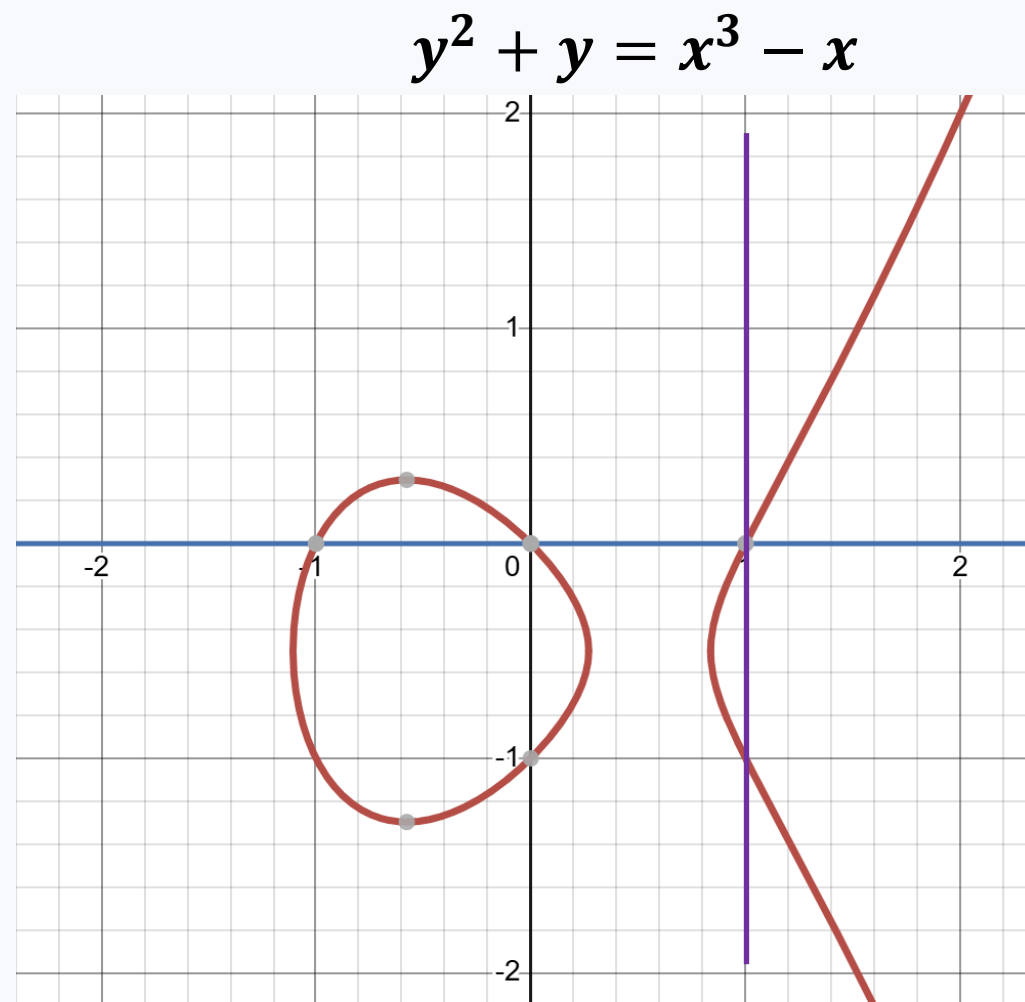
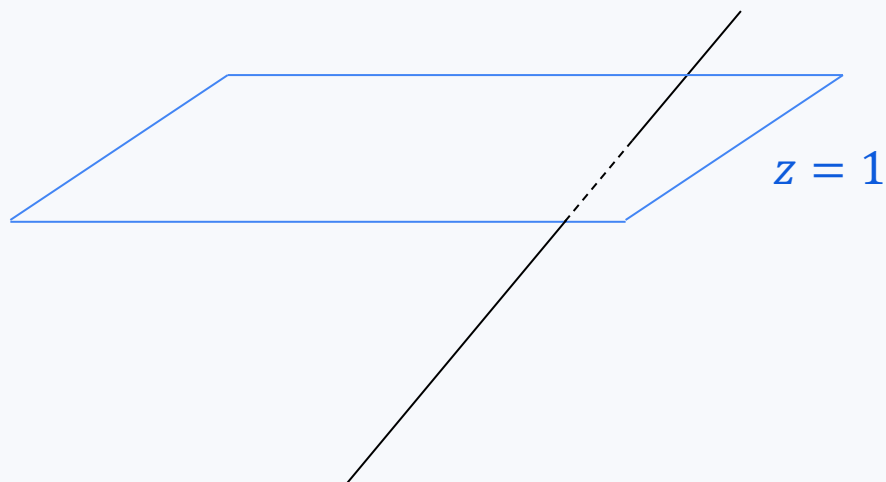


Abelian group law on elliptic curves

An **elliptic curve** over a field $\mathbb{K}$ is a smooth **projective** cubic

In the projective space:

points $(x:y:z) = \left(\frac{x}{z}:\frac{y}{z}:1\right)$ (if $z \neq 0$)

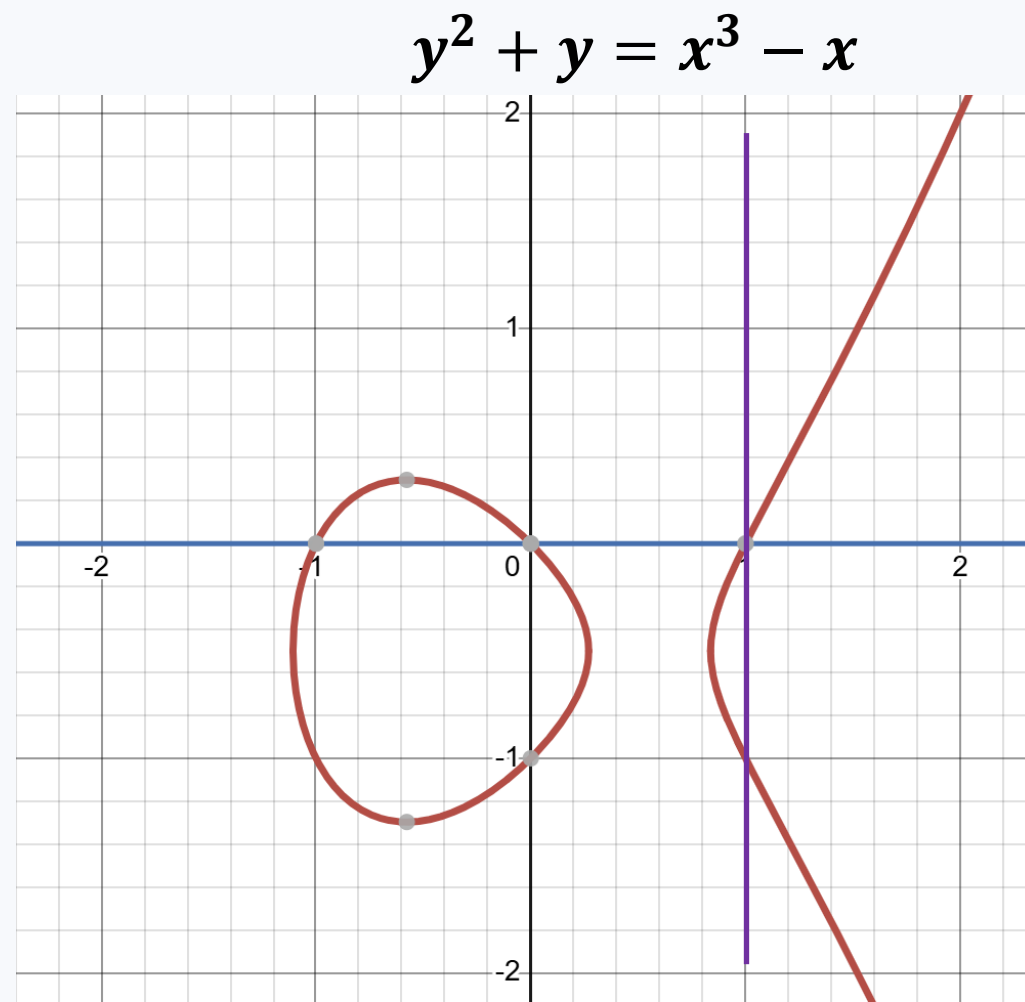
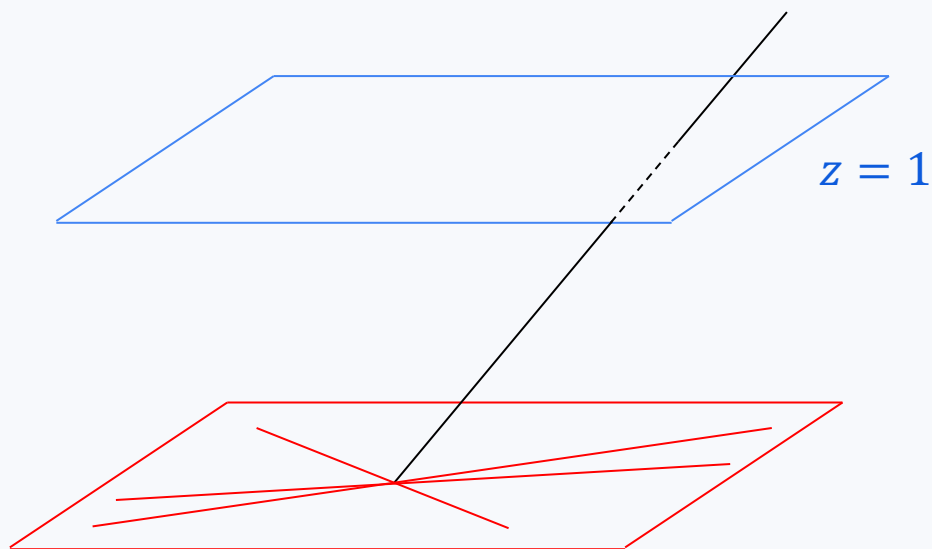


Abelian group law on elliptic curves

An **elliptic curve** over a field $\mathbb{K}$ is a smooth **projective** cubic

In the projective space:

points $(x:y:z) = \left(\frac{x}{z}:\frac{y}{z}:1\right)$ (if $z \neq 0$)

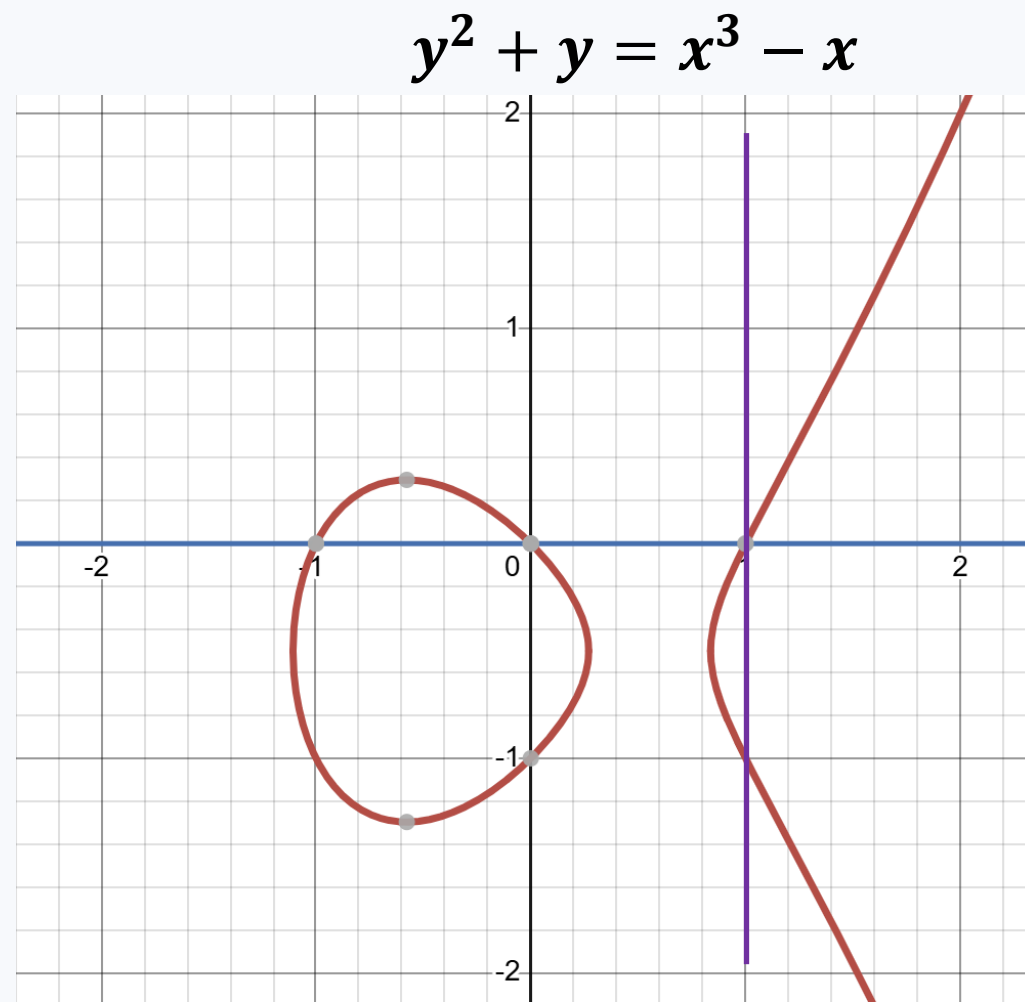


Abelian group law on elliptic curves

An **elliptic curve** over a field $\mathbb{K}$ is a smooth **projective** cubic

In the projective space:
points $(x:y:z) = \left(\frac{x}{z}:\frac{y}{z}:1\right)$,

$$\left(\frac{y}{z}\right)^2 + \frac{y}{z} = \left(\frac{x}{z}\right)^3 - \frac{x}{z}$$



Abelian group law on elliptic curves

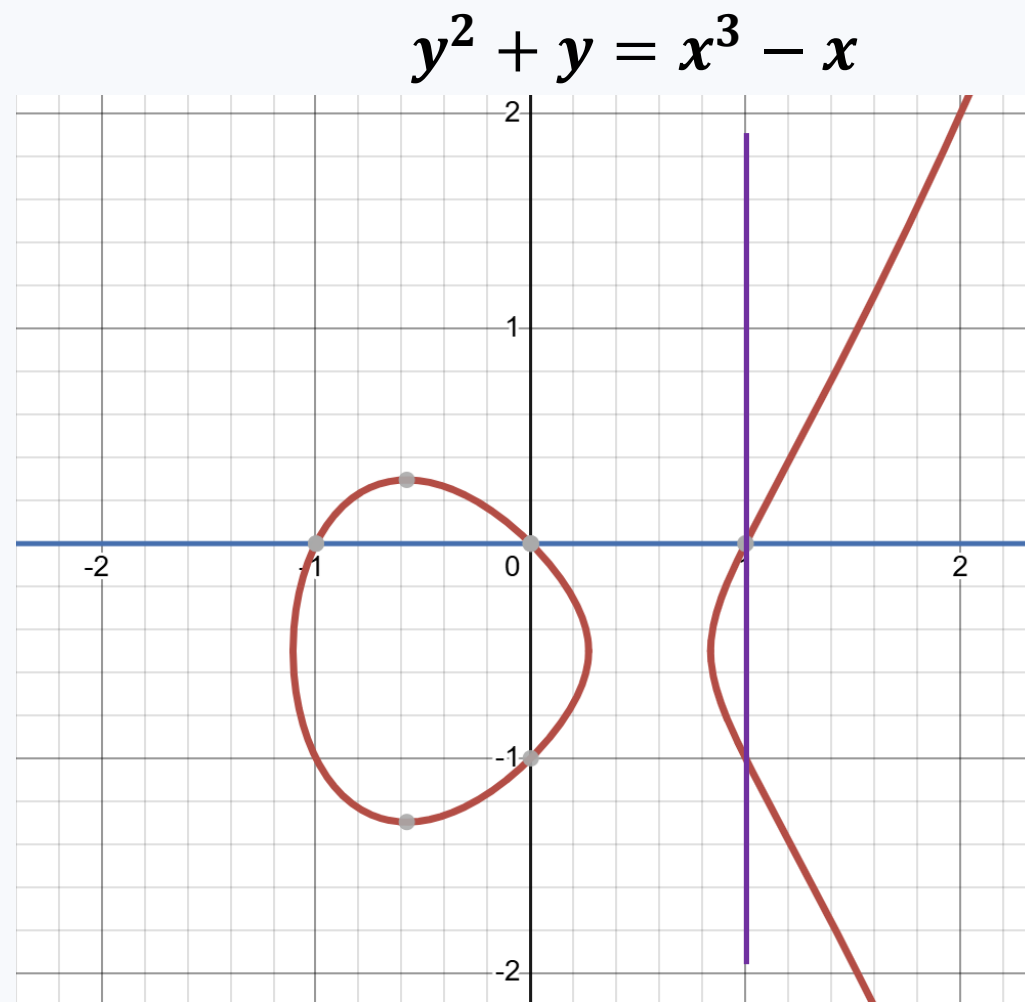
An **elliptic curve** over a field $\mathbb{K}$ is a smooth **projective** cubic

In the projective space:
points $(x:y:z) = \left(\frac{x}{z}:\frac{y}{z}:1\right)$,

$$\left(\frac{y}{z}\right)^2 + \frac{y}{z} = \left(\frac{x}{z}\right)^3 - \frac{x}{z}$$

Rewrite as a homogeneous equation

$$y^2z + yz^2 = x^3 - xz^2$$



Abelian group law on elliptic curves

An **elliptic curve** over a field $\mathbb{K}$ is a smooth cubic

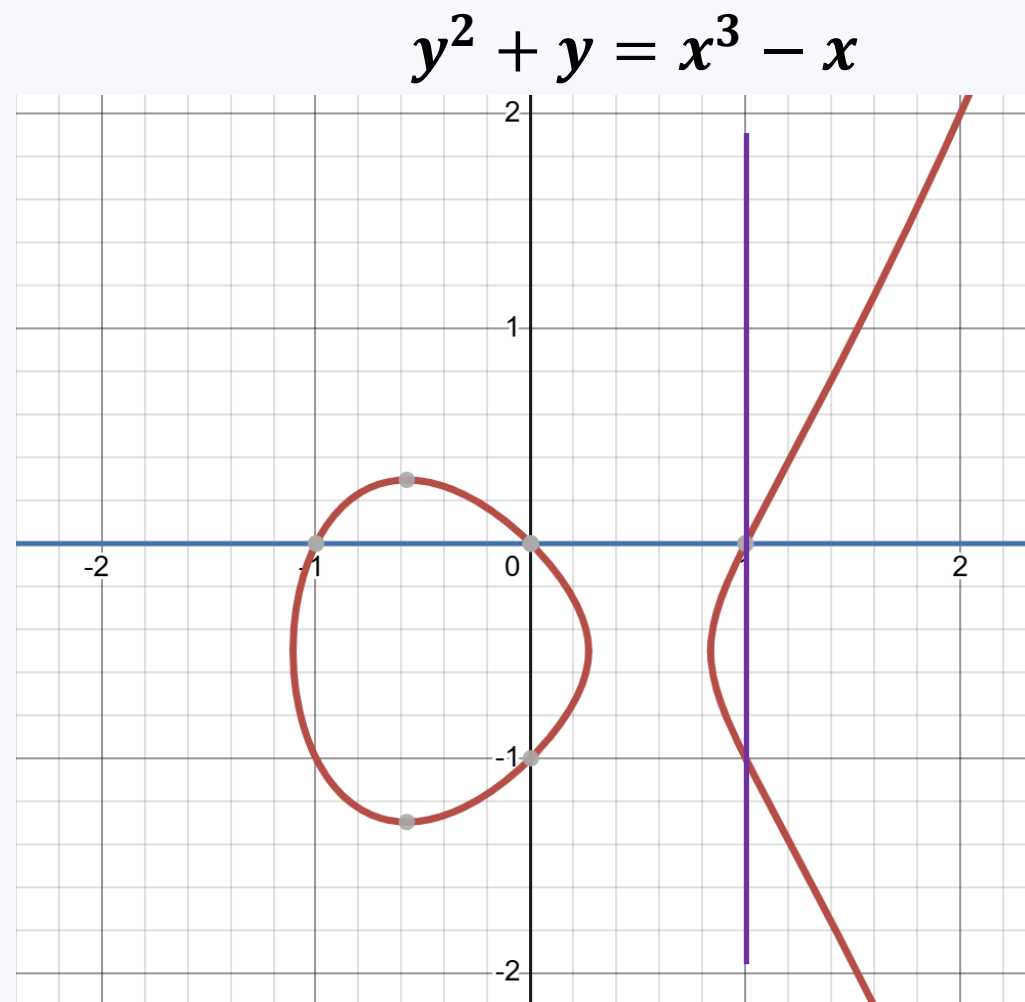
Point at the infinity:

$$y^2 z + y z^2 = x^3 - x z^2$$

If $z = 0$, we have $x = 0$, so $0 = (0:1:0)$.

$(u:v:0)$ is the point corresponding to the direction (u,v) .

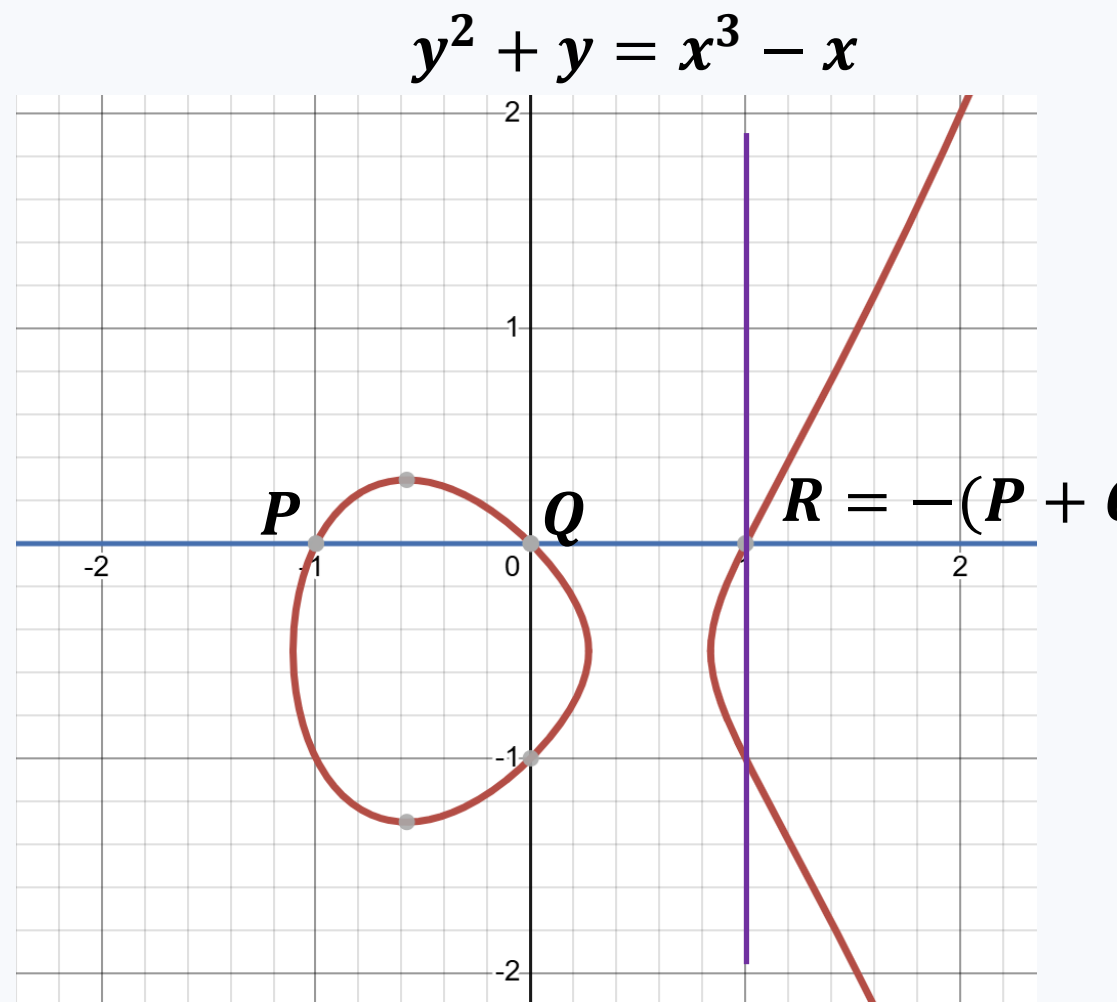
$(0:1:0)$ is the vertical direction.



Abelian group law on elliptic curves

$P + Q + R = 0$ if P, Q, R are on one line

We define 0 as the point at infinity. In our case it's $(0:1:0)$.

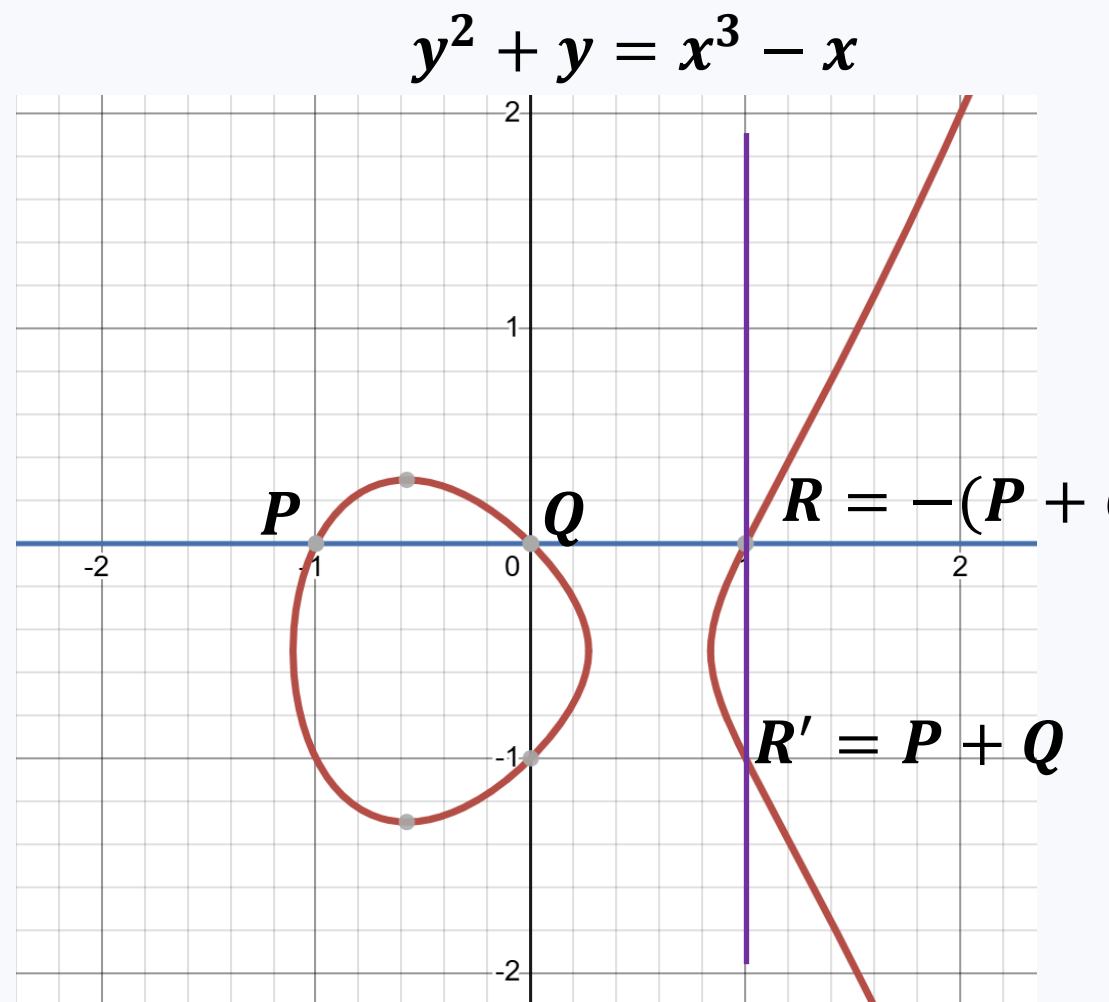


Abelian group law on elliptic curves

$P + Q + R = 0$ if P, Q, R are on one line

We define 0 as the point at infinity. In our case it's $(0:1:0)$.

A line through R and 0 is a vertical one. Here, $R' = -R = P + Q$.



Mordell-Weil rank

An abelian group is decomposed as:

$$E(\mathbb{Q}) \cong E_{\text{tors}} \oplus \mathbb{Z}^r$$

r is the **(Mordell-Weil) rank**.

$$y^2 + y = x^3 - x$$

Mordell-Weil rank

An abelian group is decomposed as:

$$E(\mathbb{Q}) \cong E_{\text{tors}} \oplus \mathbb{Z}^r$$

r is the **(Mordell-Weil) rank**.

The example curve:

- No torsion
- Rank is 1, so $E(\mathbb{Q}) \cong \mathbb{Z}$, generated, e.g., by $(0,0)$.

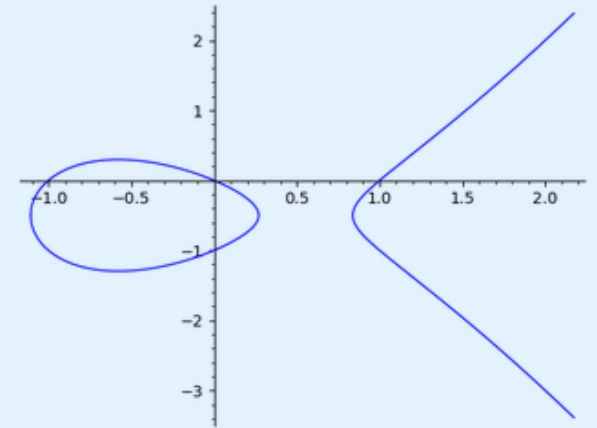
The L-functions and modular forms database (LMFDB)

<https://www.lmfdb.org/EllipticCurve/Q/37/a/1>

$$y^2 + y = x^3 - x$$

Label

37.a1



Conductor

37

Discriminant

37

j-invariant

$\frac{110592}{37}$

CM

no

Rank

1

Torsion structure

trivial

Mordell-Weil rank

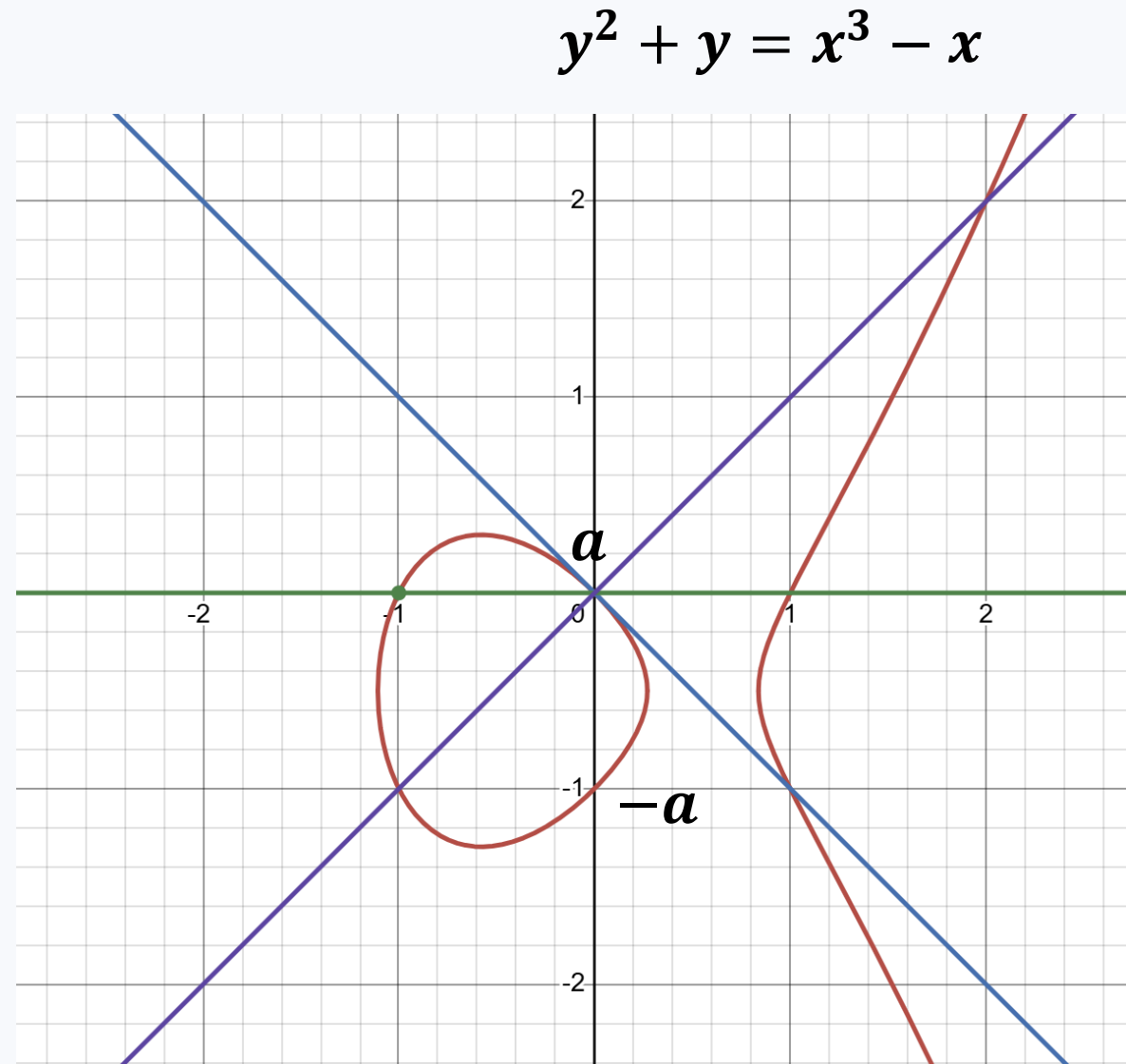
An abelian group is decomposed as:

$$E(\mathbb{Q}) \cong E_{\text{tors}} \oplus \mathbb{Z}^r$$

r is the **(Mordell-Weil) rank**.

The example curve:

- Rank is 1, so $E(\mathbb{Q}) \cong \mathbb{Z}$, generated, e.g., by $(0,0)$.



Mordell-Weil rank

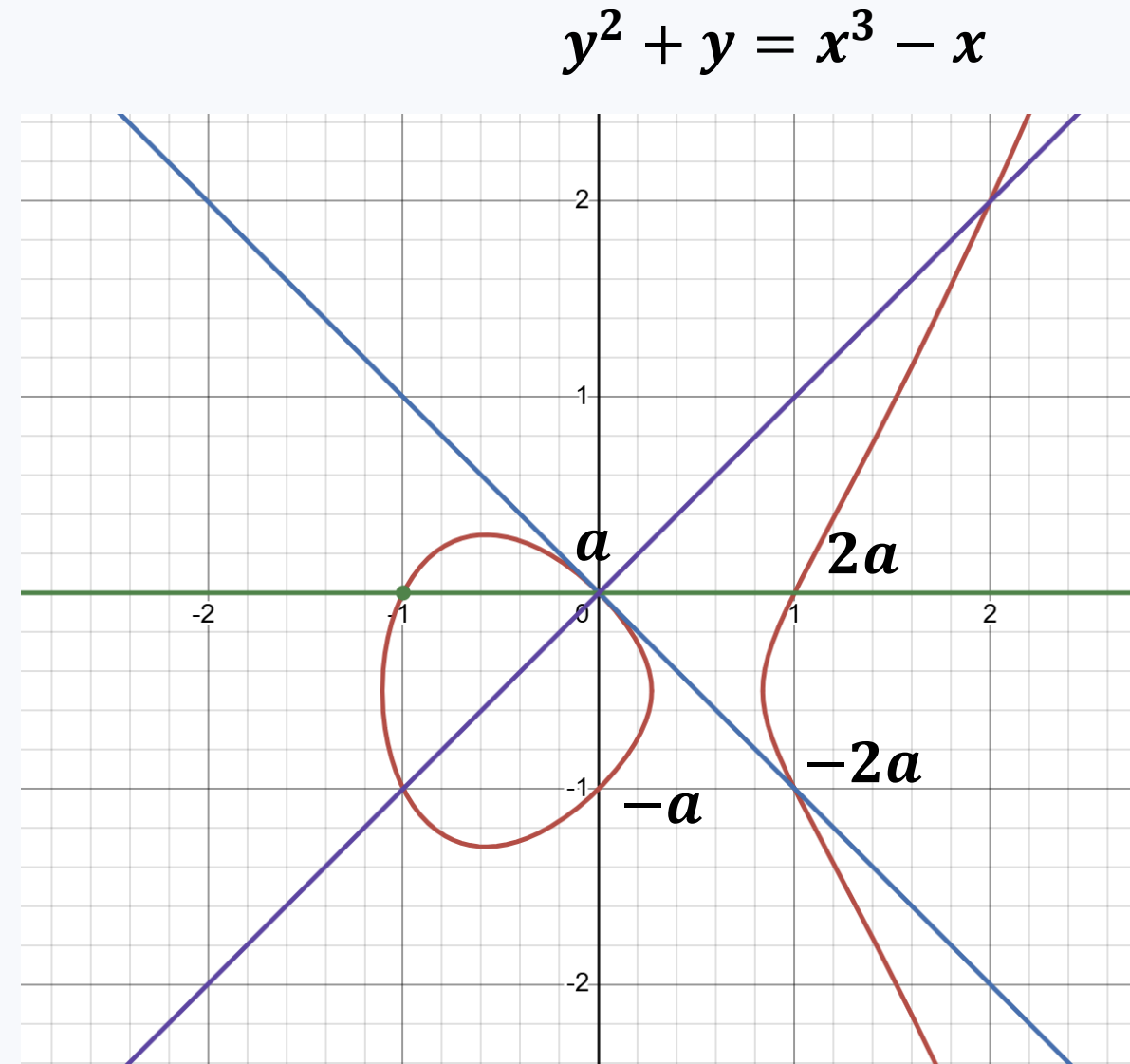
An abelian group is decomposed as:

$$E(\mathbb{Q}) \cong E_{\text{tors}} \oplus \mathbb{Z}^r$$

r is the **(Mordell-Weil) rank**.

The example curve:

- Rank is 1, so $E(\mathbb{Q}) \cong \mathbb{Z}$, generated, e.g., by $(0,0)$.



Mordell-Weil rank

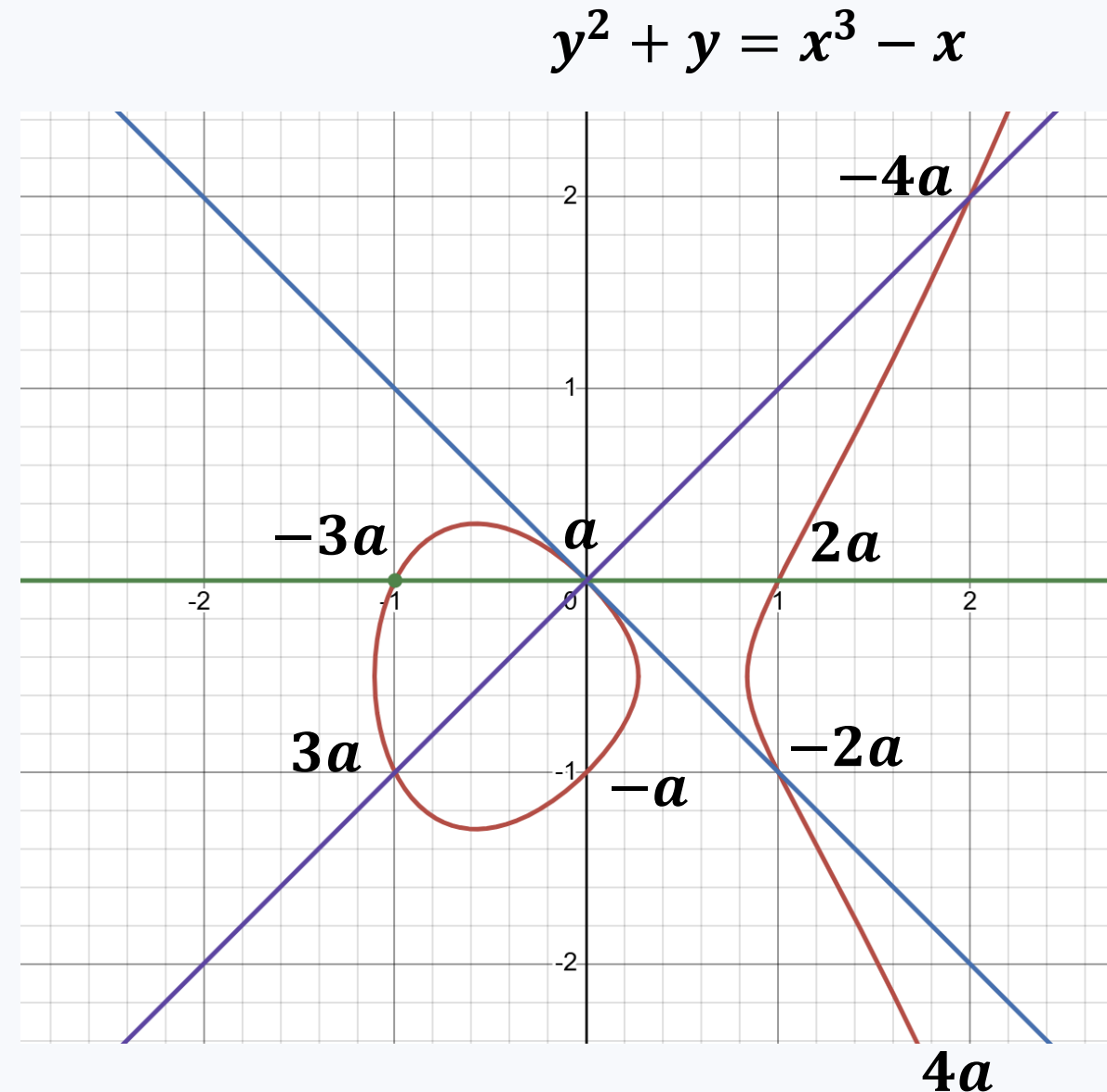
An abelian group is decomposed as:

$$E(\mathbb{Q}) \cong E_{\text{tors}} \oplus \mathbb{Z}^r$$

r is the **(Mordell-Weil) rank**.

The example curve:

- Rank is 1, so $E(\mathbb{Q}) \cong \mathbb{Z}$, generated, e.g., by $(0,0)$.



Mordell-Weil rank

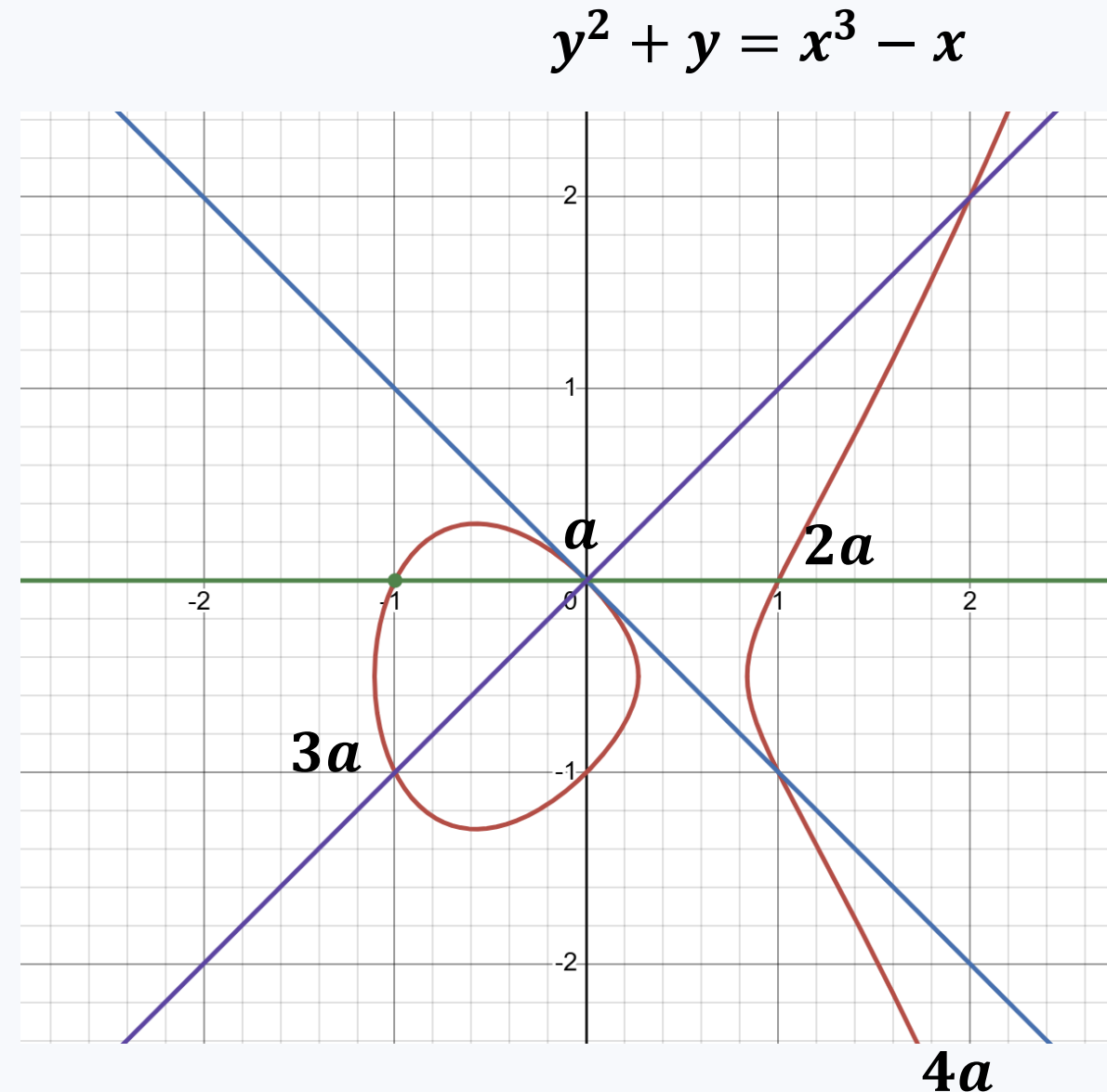
An abelian group is decomposed as:

$$E(\mathbb{Q}) \cong E_{\text{tors}} \oplus \mathbb{Z}^r$$

r is the **(Mordell-Weil) rank**.

The example curve:

- Rank is 1, so $E(\mathbb{Q}) \cong \mathbb{Z}$, generated, e.g., by $(0,0)$.



Mordell-Weil rank

An abelian group is decomposed as:

$$E(\mathbb{Q}) \cong E_{\text{tors}} \oplus \mathbb{Z}^r$$

r is the **(Mordell-Weil) rank**.

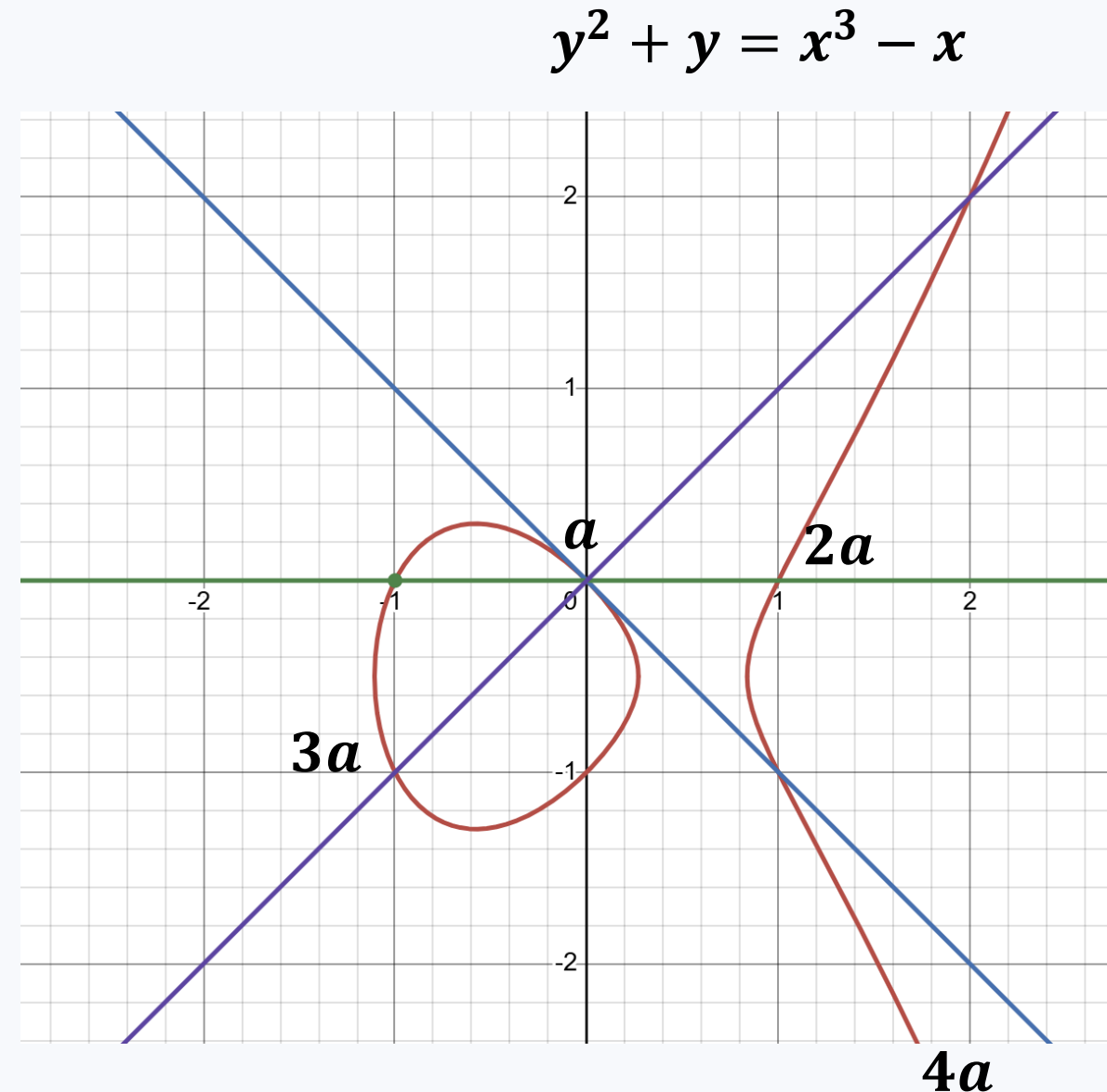
The example curve:

- Rank is 1, so $E(\mathbb{Q}) \cong \mathbb{Z}$, generated, e.g., by $(0,0)$.

Not much is known about ranks... No general computation algorithm or even upper bounds ☹

The L-functions and modular forms database (LMFDB)

<https://www.lmfdb.org/EllipticCurve/Q/37/a/1>

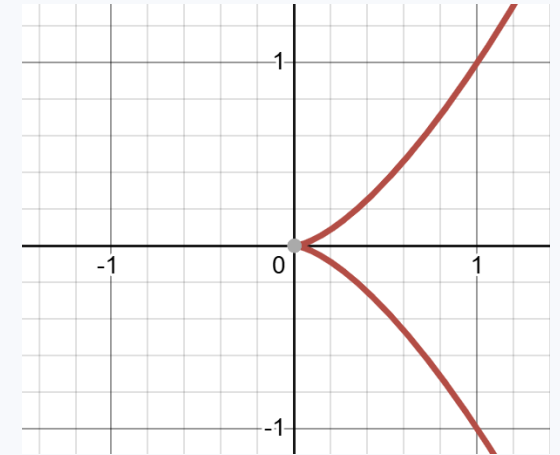


Conductor

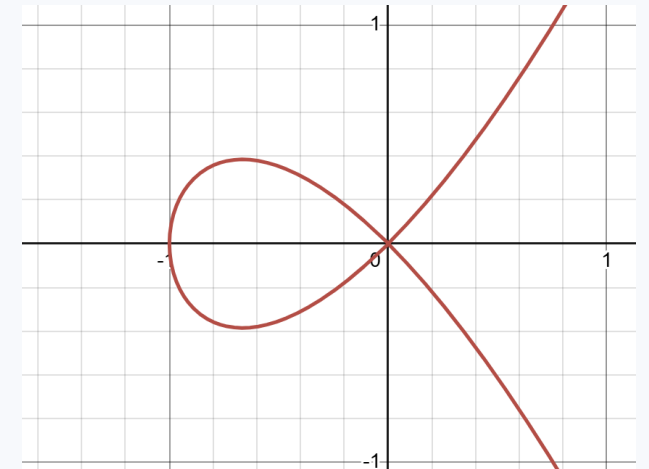
Conductor encodes what happens when we reduce to $E(\mathbb{F}_p)$

- **Good prime:** $E(\mathbb{F}_p)$ is still smooth
- **Additive reduction:** cusp
- **Multiplicative reduction:** node

$$y^2 = x^3 \text{ (cusp)}$$



$$y^2 = x^3 + x^2 \text{ (node)}$$



Conductor

Conductor encodes what happens when we reduce to $E(\mathbb{F}_p)$

- **Good prime:** $E(\mathbb{F}_p)$ is still smooth
- **Additive reduction:** cusp
- **Multiplicative reduction:** node

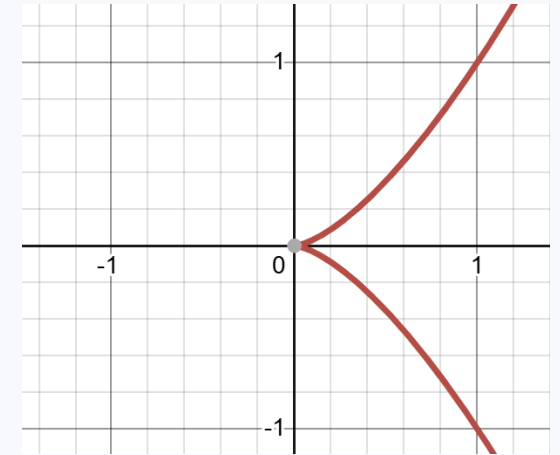
The **conductor** N of an [elliptic curve](#) E defined over $\mathbb{Q}$ is a positive integer divisible by the primes of bad reduction and no others. It has the form $N = \prod p^{e_p}$, where the exponent e_p is

- $e_p = 1$ if E has [multiplicative reduction](#) at p ,
- $e_p = 2$ if E has [additive reduction](#) at p and $p \geq 5$,
- $2 \leq e_p \leq 5$ if E has additive reduction and $p = 3$, and
- $2 \leq e_p \leq 8$ if E has additive reduction and $p = 2$.

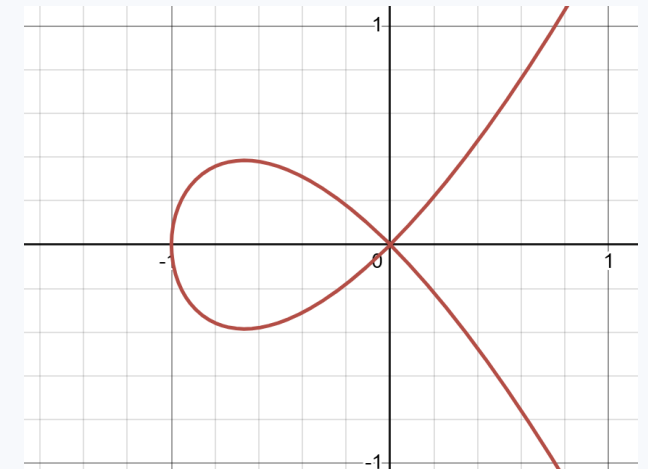
The L-functions and modular forms database (LMFDB)

<https://www.lmfdb.org/EllipticCurve/Q/37/a/1>

$$y^2 = x^3 \text{ (cusp)}$$



$$y^2 = x^3 + x^2 \text{ (node)}$$



ML time! Point count vs ranks

$$a_p(E) = p + 1 - \#E(\mathbb{F}_p)$$

Data: $E \mapsto v_L(E) = (a_{p_1}, \dots, a_{p_{1000}})$, up to $p_{1000} = 7919$.

Classifying: Mordell–Weil rank

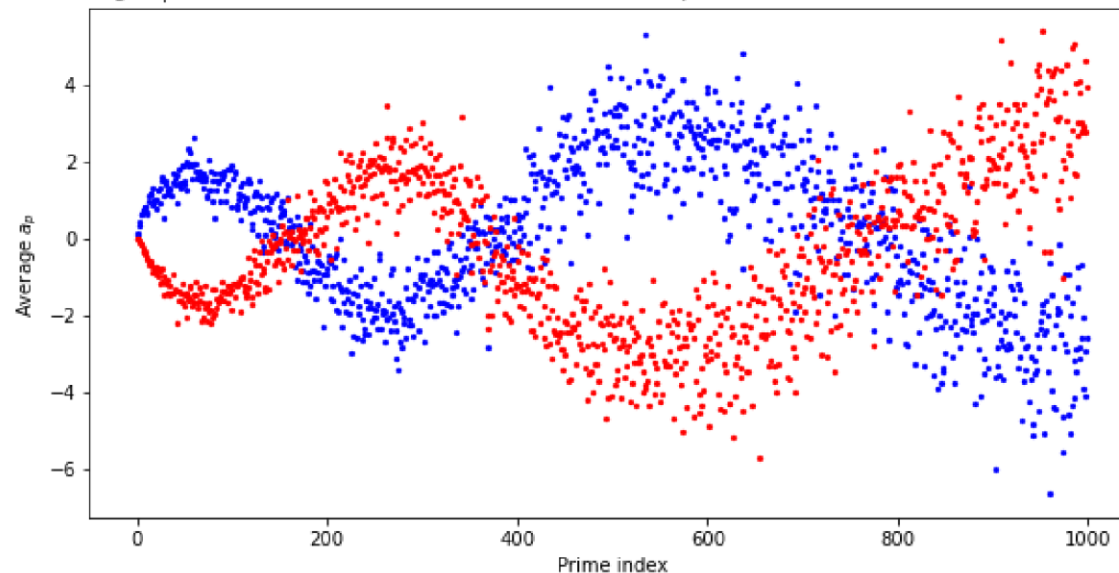
N_E range	r_E	$ \text{Data} = \#\{E\}$	Precision	Confidence
$[1, 1 \times 10^5]$	$\{0, 1\}$	2.0×10^4 ($\times 2$)	0.961	0.92
"	$\{0, 2\}$	"	0.996	0.99
"	$\{1, 2\}$	"	0.999	0.99
"	$\{0, 1, 2\}$	2.0×10^4 ($\times 3$)	0.975	0.96

Murmurations

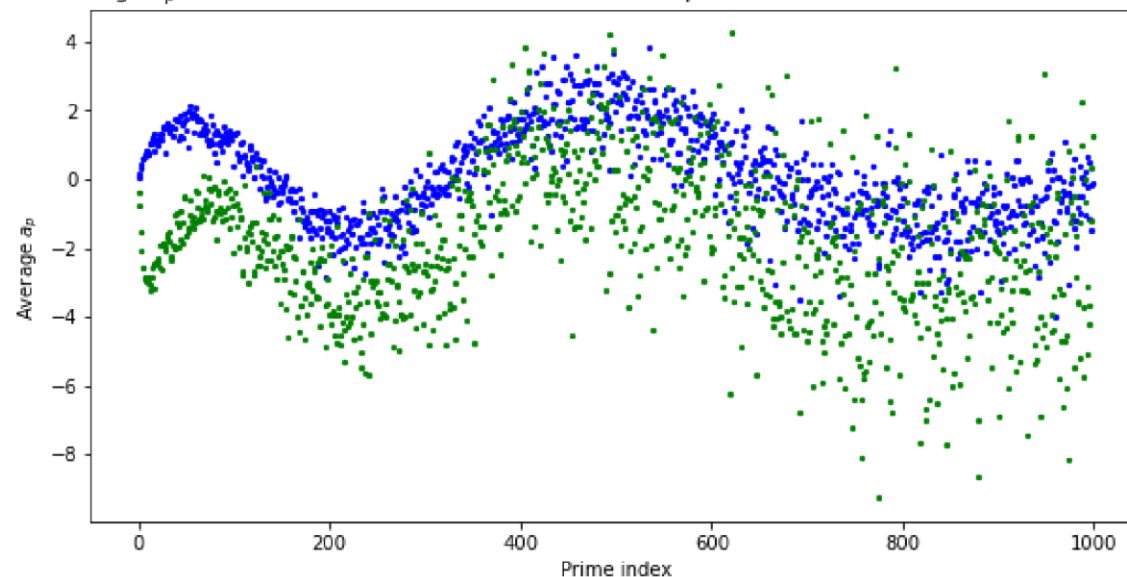
Consider the average over a conductor range $[N_1, N_2]$ for a given rank r and a given prime p_n :

$$f_r(n) = \frac{1}{\#\mathcal{E}_r[N_1, N_2]} \sum_{E \in \mathcal{E}_r[N_1, N_2]} a_{p_n}(E),$$

Average a_p over 4328 rank 0 (blue) and 5194 rank 1 elliptic curves with conductor in $[7500, 10000]$



Average a_p over 8536 rank 0 (blue) and 1380 rank 2 elliptic curves with conductor in $[5000, 10000]$



The Birch and Swinnerton-Dyer (BSD) conjecture

Curve $y^2 = x^3 + ax + b$; $a_p(E) = p + 1 - \#E(\mathbb{F}_p)$. L-series:

$$L(C, s) := \prod_{p \nmid 2\Delta} (1 - a_p p^{-s} + p^{1-2s})^{-1}.$$

Conjecture (Birch and Swinnerton-Dyer). *The Taylor expansion of $L(C, s)$ at $s = 1$ has the form*

$$L(C, s) = c(s - 1)^r + \text{higher order terms}$$

with $c \neq 0$ and $r = \text{rank}(C(\mathbb{Q}))$.

Observed in 1960s.

<https://www.claymath.org/wp-content/uploads/2022/05/birchswin.pdf>

Two classical ML scenarios

Approximating particular formula

- Model provides value
- Aim for a very precise model
- Models are usually powerful and complex

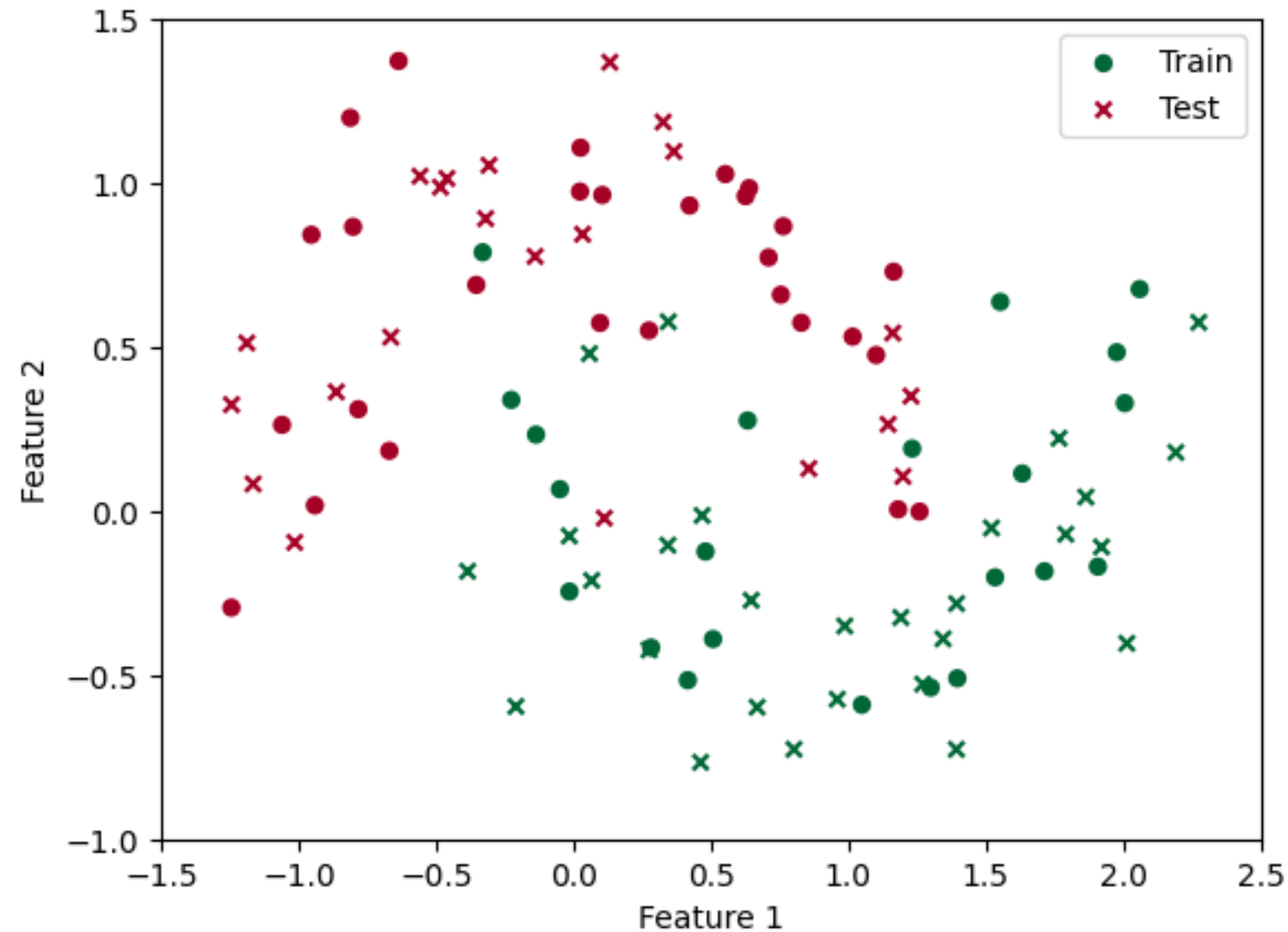
Mining dependencies; fast checks

- Model is a POC
- Models are usually simple, best if interpretable
- Inspiration rather than a theorem

Binary linear classification

Binary linear classification

Let's classify 2d points:
red vs green

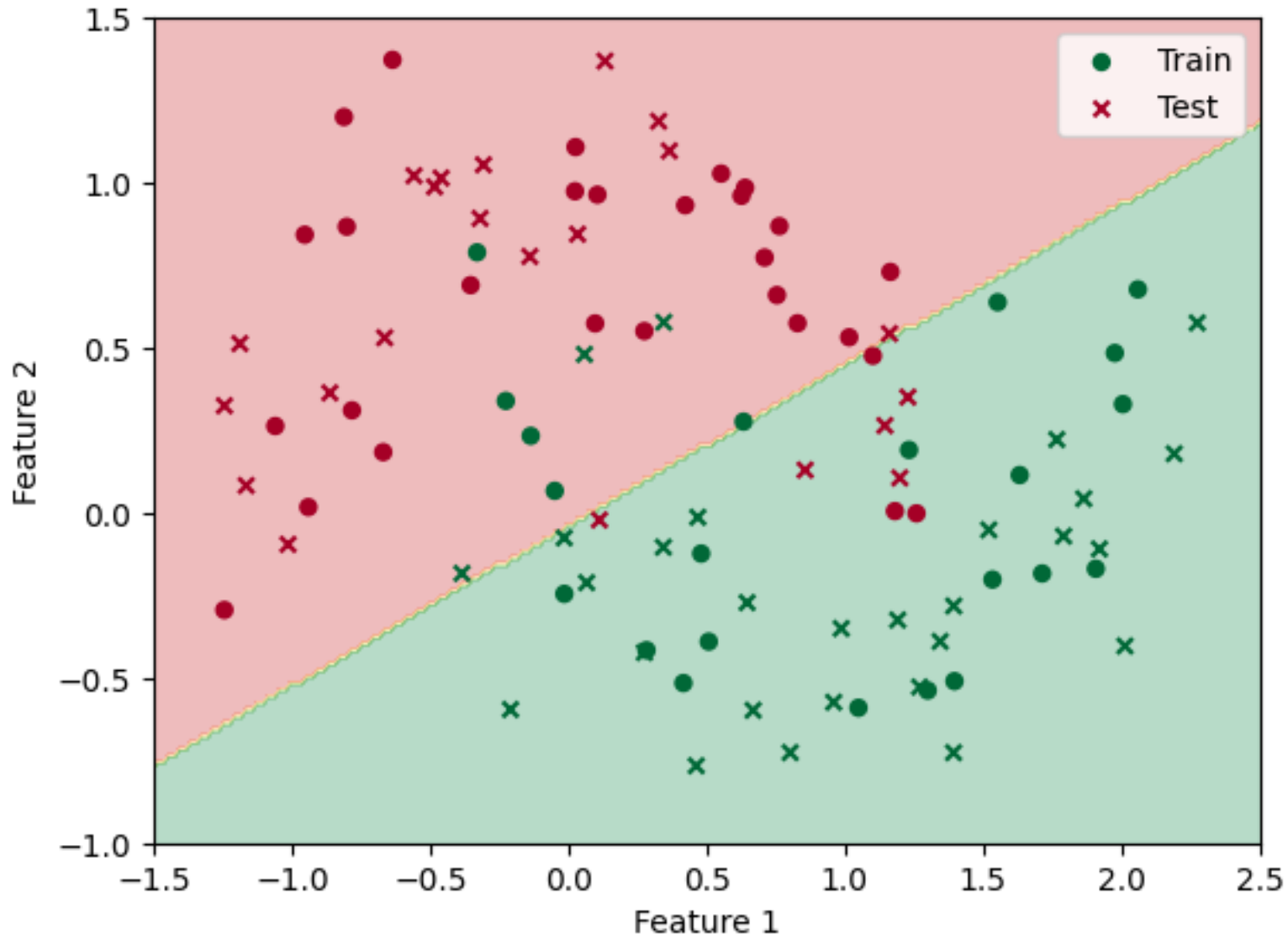


Binary linear classification

Let's classify 2d points:
red vs green

The simplest
classification rule
is a linear rule

So, let's just find a line
optimally
discriminating
between the two
classes!



Let's add math

Feature description of an object:

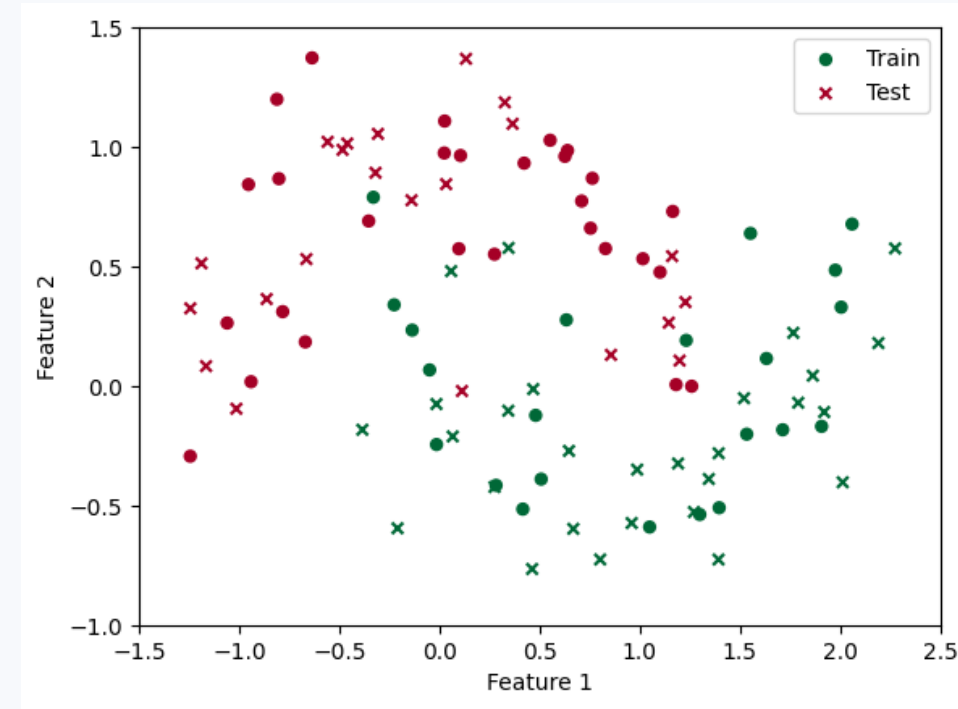
$$x = (x_1, \dots, x_D)$$

x_i – i -th feature value.

In our case it's

$$x = (x_1, x_2)$$

Class 0
Class 1



Let's add math

Feature description of an object:

$$x = (x_1, x_2)$$

A **bias term** w_0 , a **weight vector**:

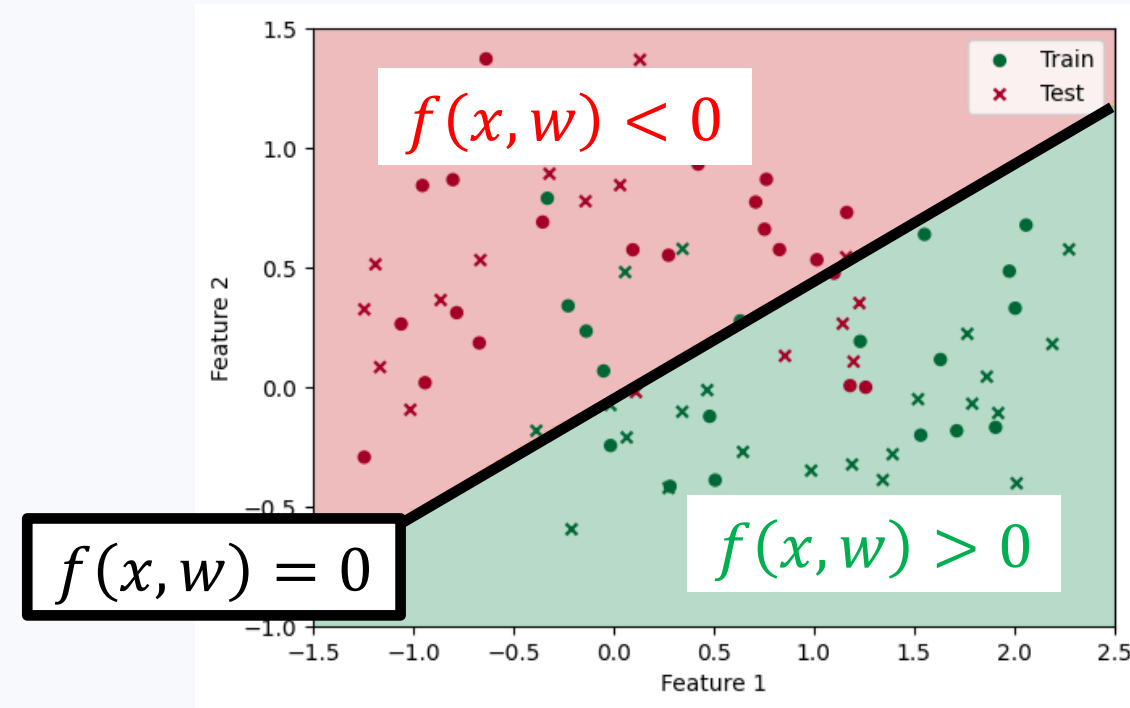
$$w = (w_1, w_2)$$

A **linear model**:

$$f(x, w) = w_0 + x_1 w_1 + x_2 w_2$$

Class 0

Class 1



Let's add math

Feature description of an object:

$$x = (x_1, \dots, x_D)$$

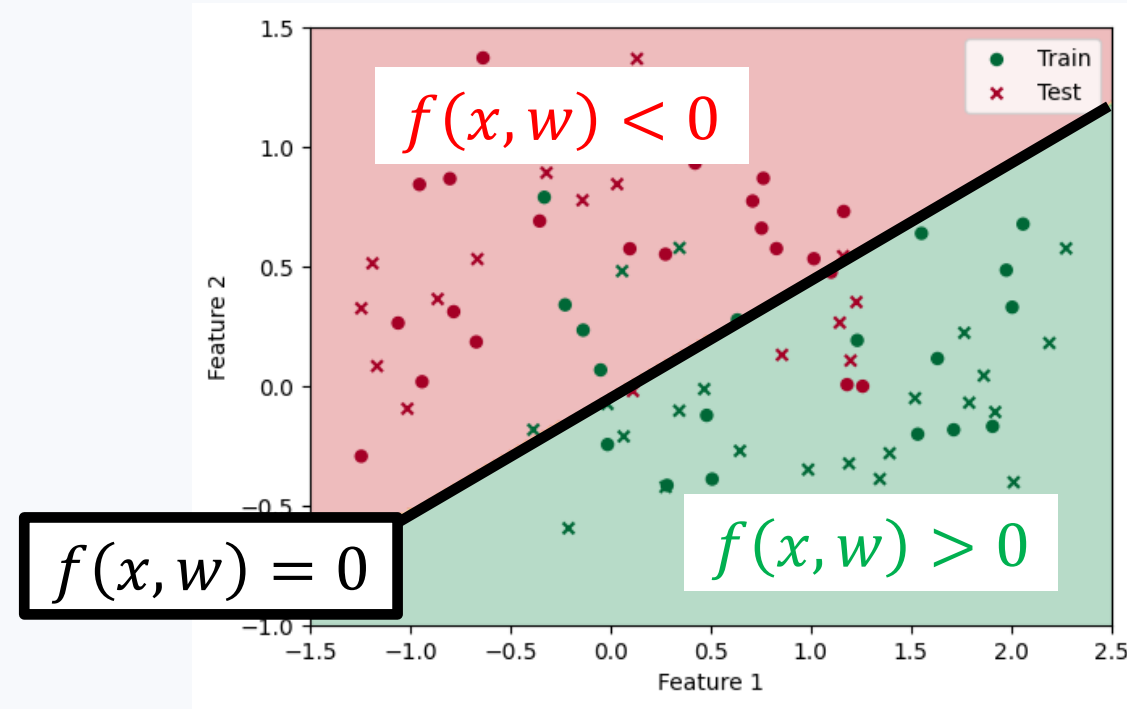
A **bias term** w_0 , a **weight vector**:

$$w = (w_1, \dots, w_D)$$

A **linear model**:

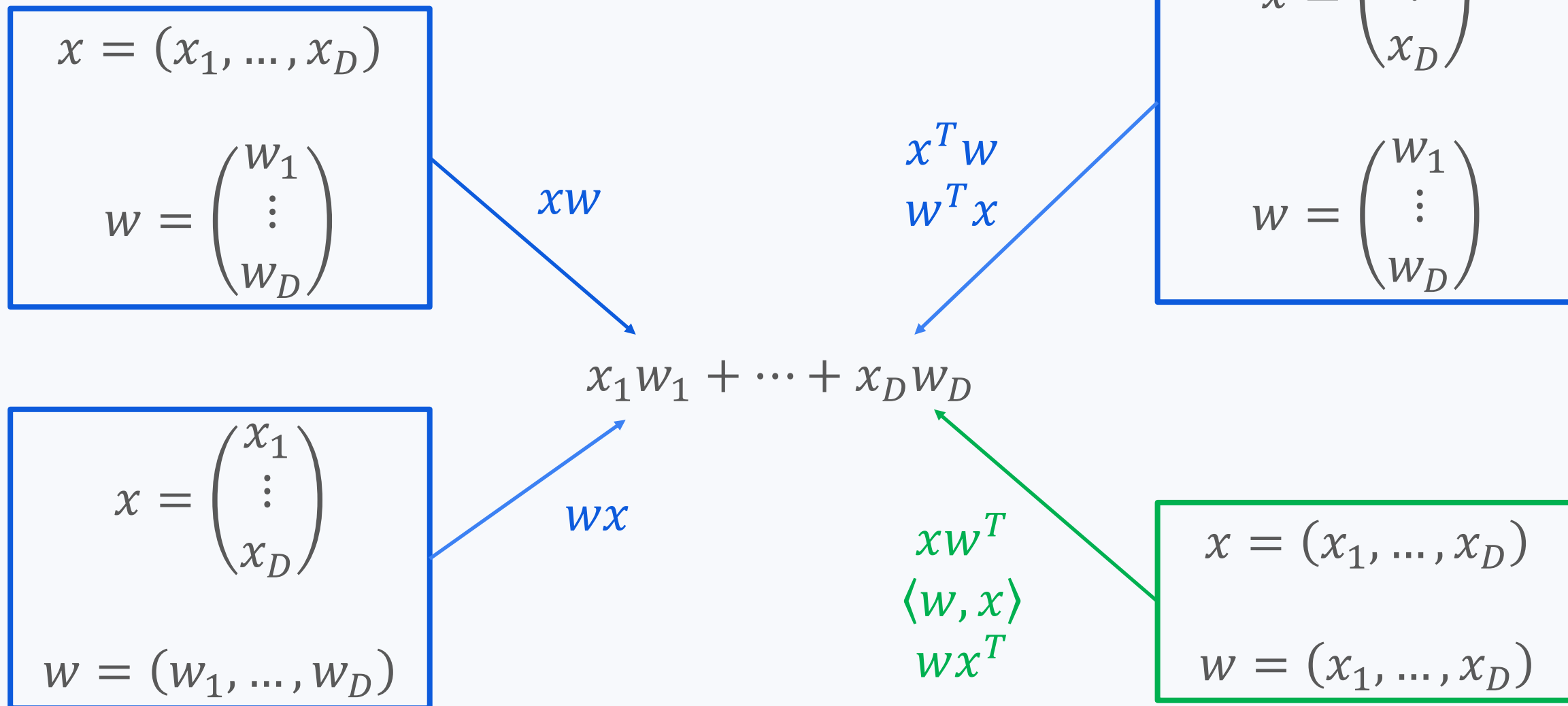
$$f(x, w) = w_0 + x_1 w_1 + \dots + x_D w_D = w_0 + x w^T$$

Class 0
Class 1



$$xw^T = (x_1, \dots, x_D) \begin{pmatrix} w_1 \\ \vdots \\ w_D \end{pmatrix}$$

A word of caution about notation



My explanations

For x :

	Feature 1	...	Feature D
Object 1	.		
...			
Object N			

For w :

The usual weight shape in Pytorch is:

(out_features, in_features)

In our case: (1, D)

Let's add math

Feature description of an object:

$$x = (x_1, \dots, x_D)$$

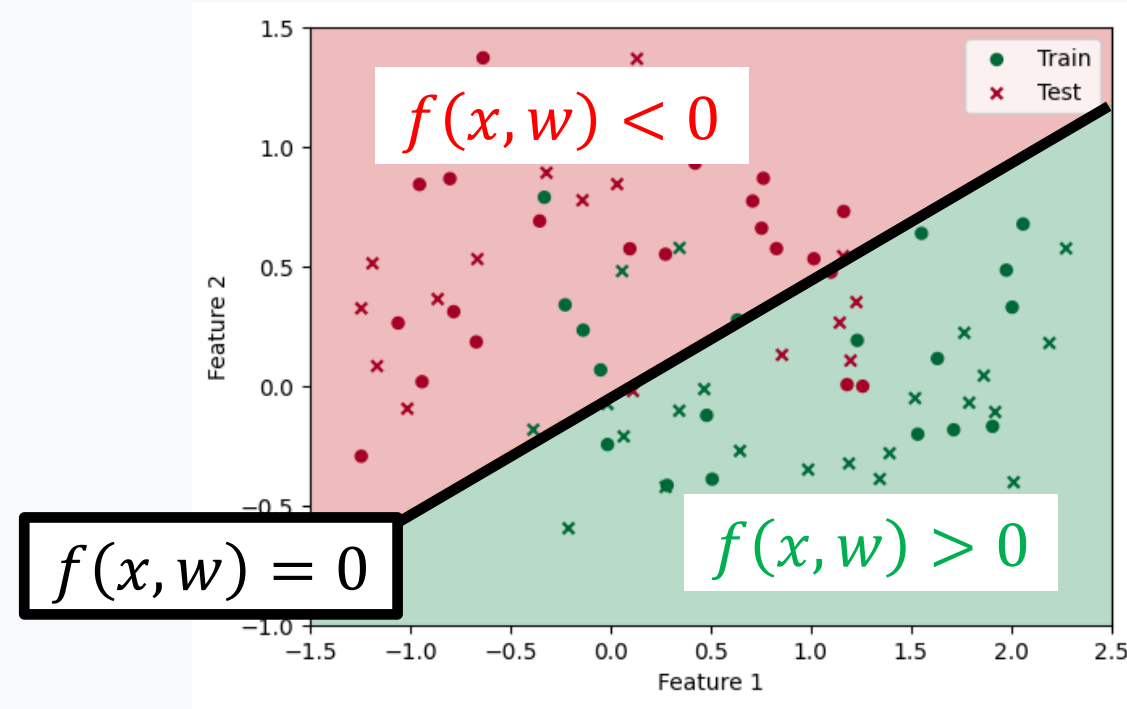
A **bias term** w_0 , a **weight vector**:

$$w = (w_1, \dots, w_D)$$

A **linear model**:

$$f(x, w) = w_0 + x_1 w_1 + \dots + x_D w_D = w_0 + x w^T$$

Class 0
Class 1



$$xw^T = (x_1, \dots, x_D) \begin{pmatrix} w_1 \\ \vdots \\ w_D \end{pmatrix}$$

Let's get rid of the bias term

Feature description of an object:

$$\tilde{x} = (\textcolor{red}{1}, x_1, \dots, x_D)$$

A **bias term** w_0 , a **weight vector**:

$$\tilde{w} = (w_0, w_1, \dots, w_D)$$

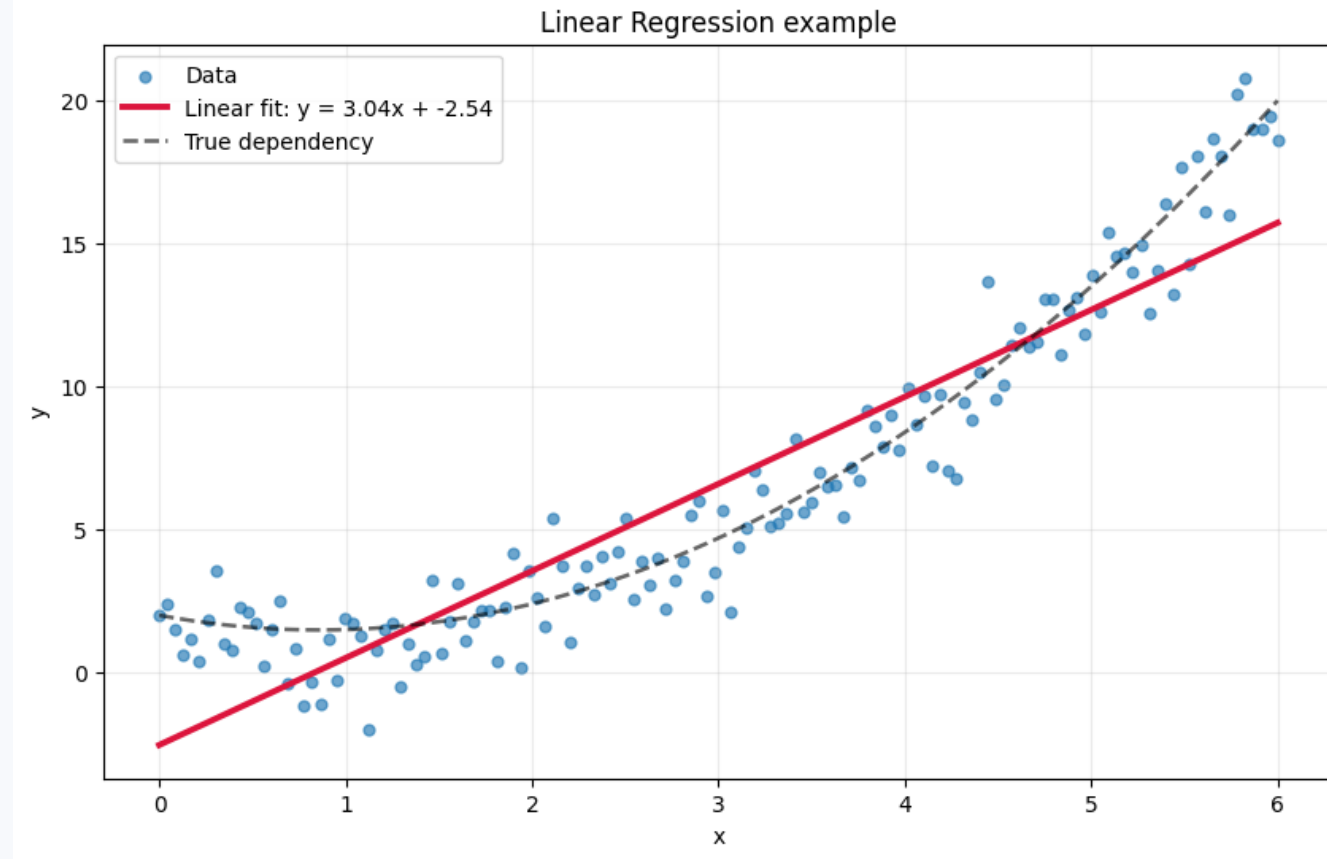
A **linear model**:

$$f(x, w) = 1 \cdot w_0 + x_1 w_1 + \dots + x_D w_D = \tilde{x} \tilde{w}^T$$

Linear regression

A **linear model**:

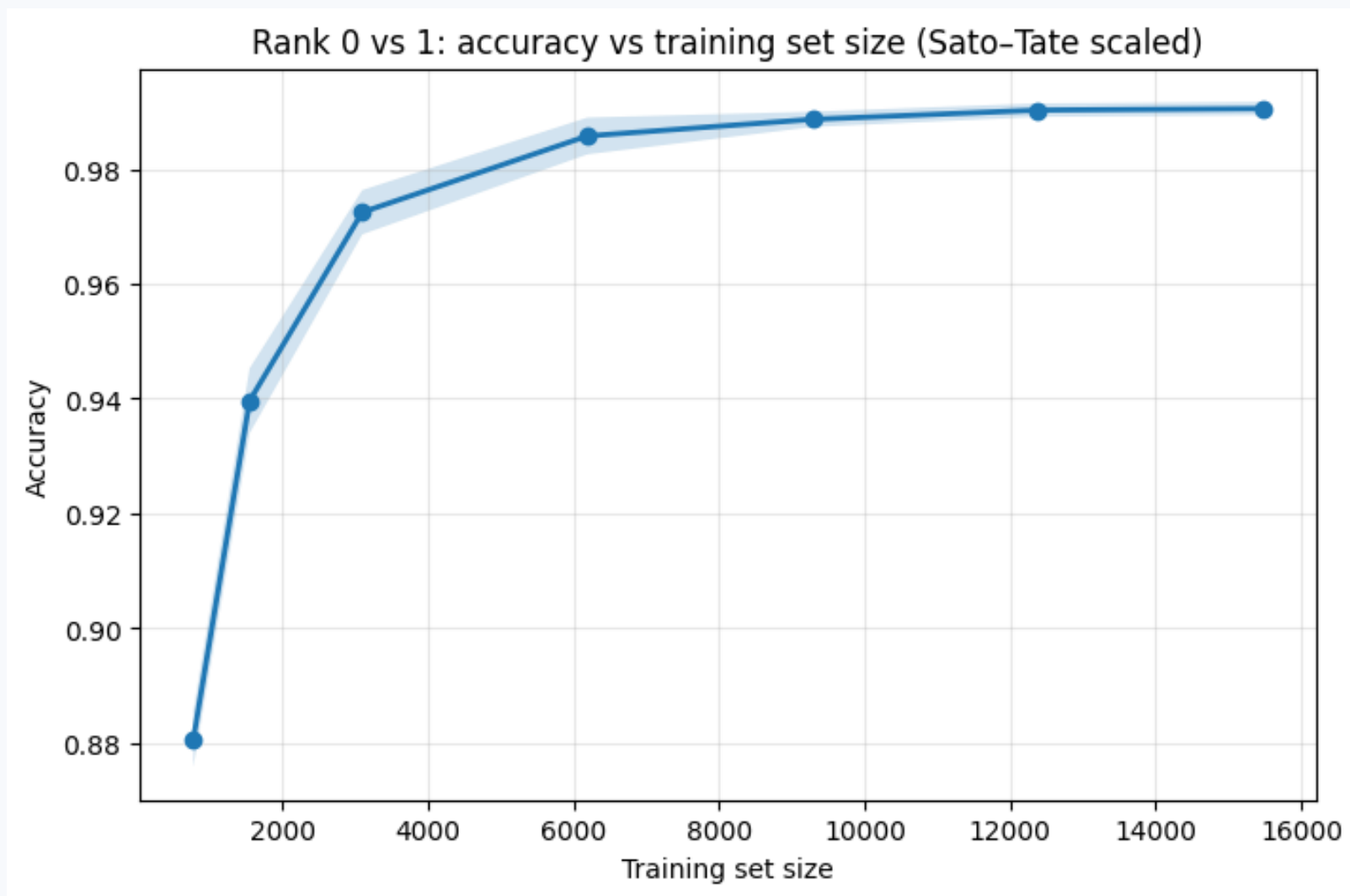
$$f(x, w) = w_0 + x_1 w_1$$



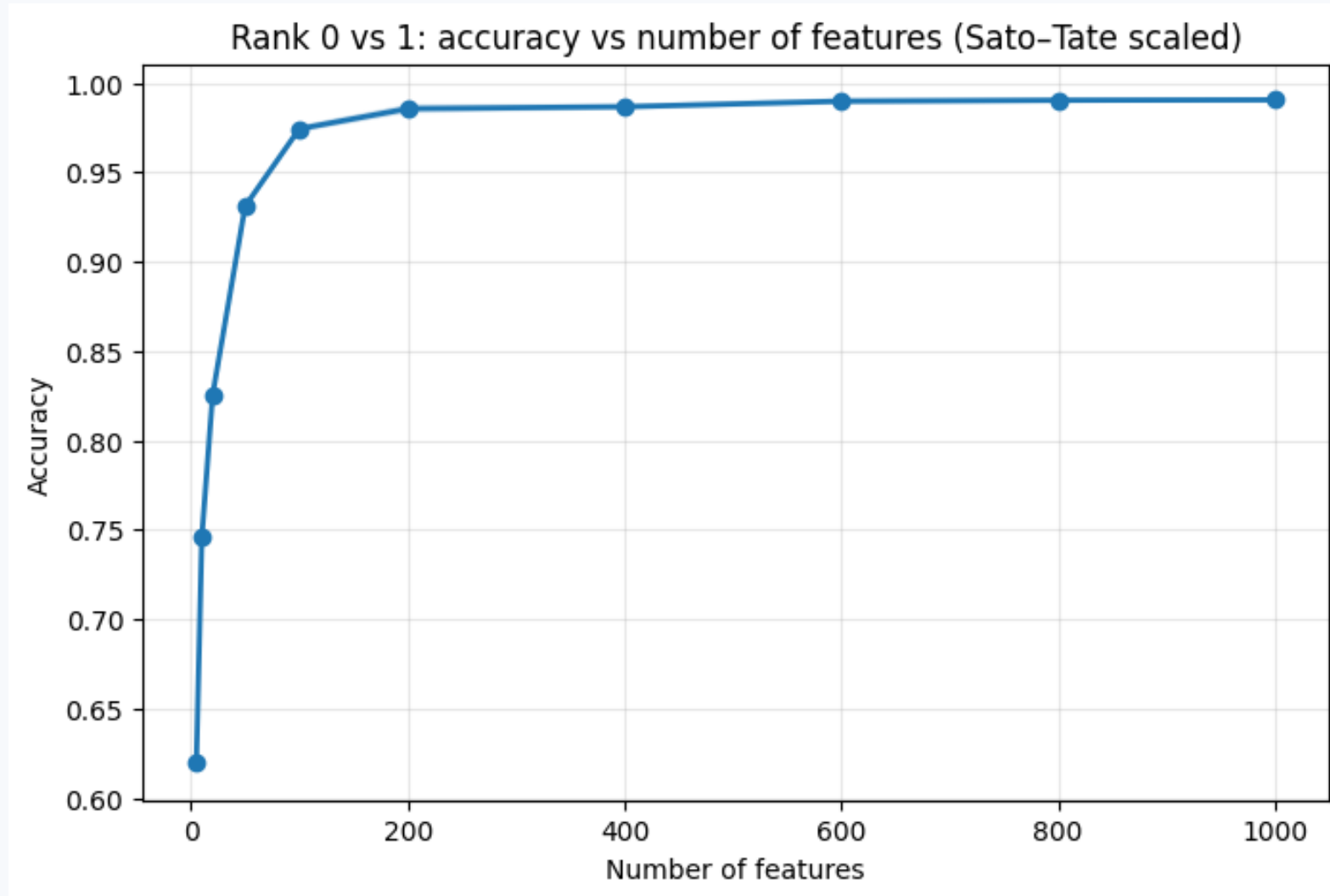
Classification of quadratic equations

Pathology of
~~linear models~~
ML pipelines

Not enough data

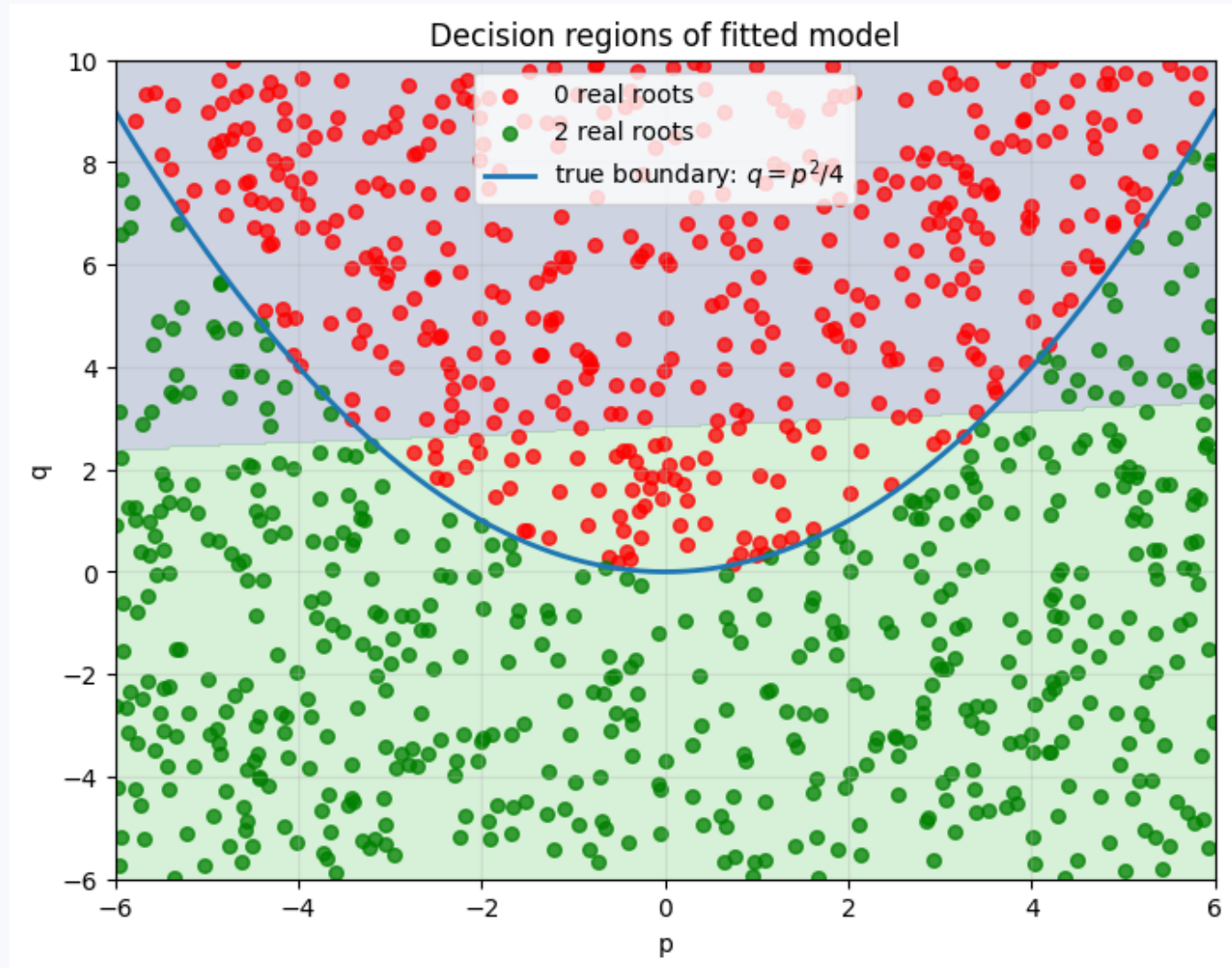


Not enough features



Functional family not expressive enough

Only linear features



Wrong functional family

Features:

1,

$\sin p, \cos p,$

$\sin q, \cos q,$

$\sin 2p, \cos 2p,$

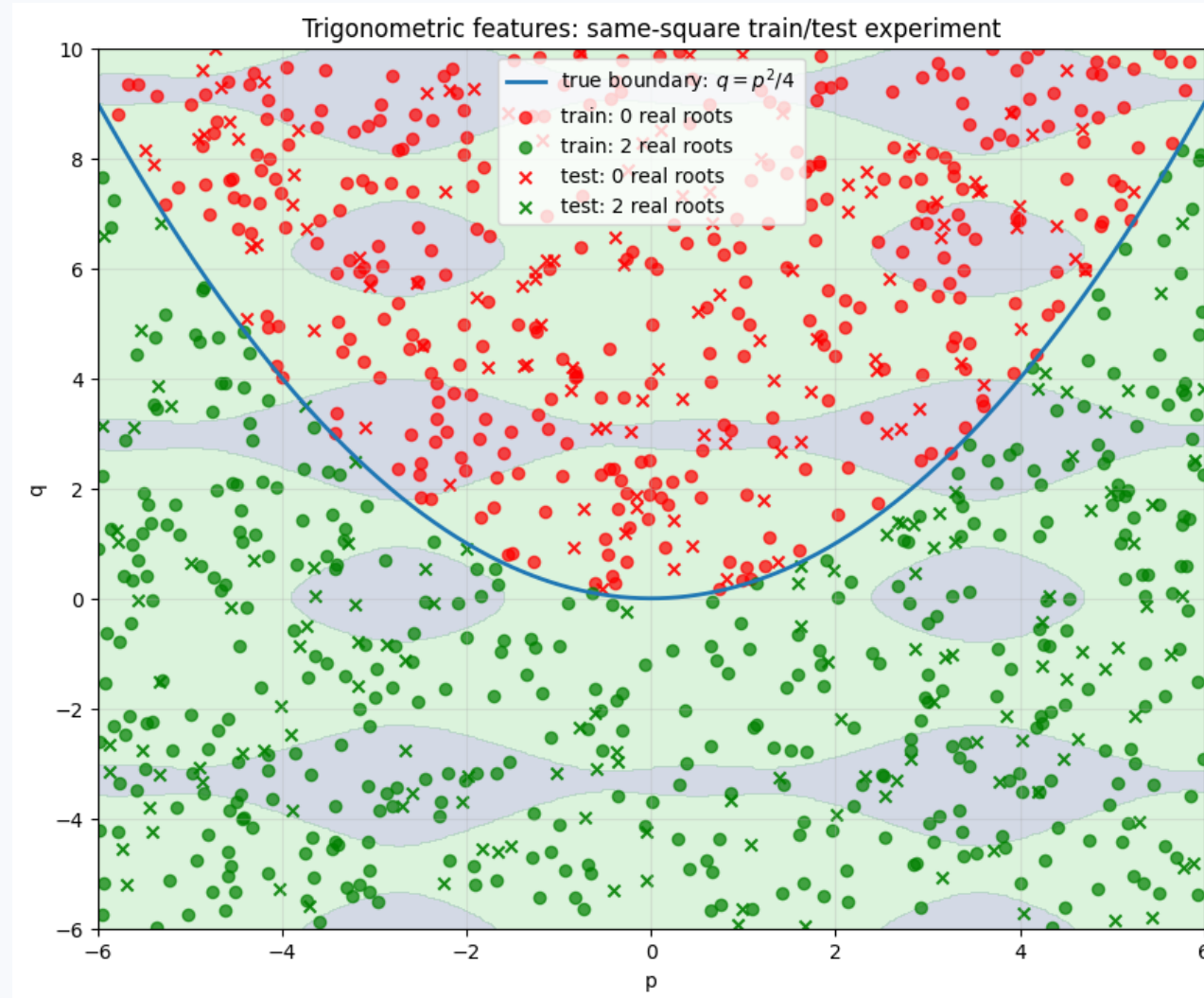
$\sin 2q, \cos 2q$

Train accuracy:

0.5867

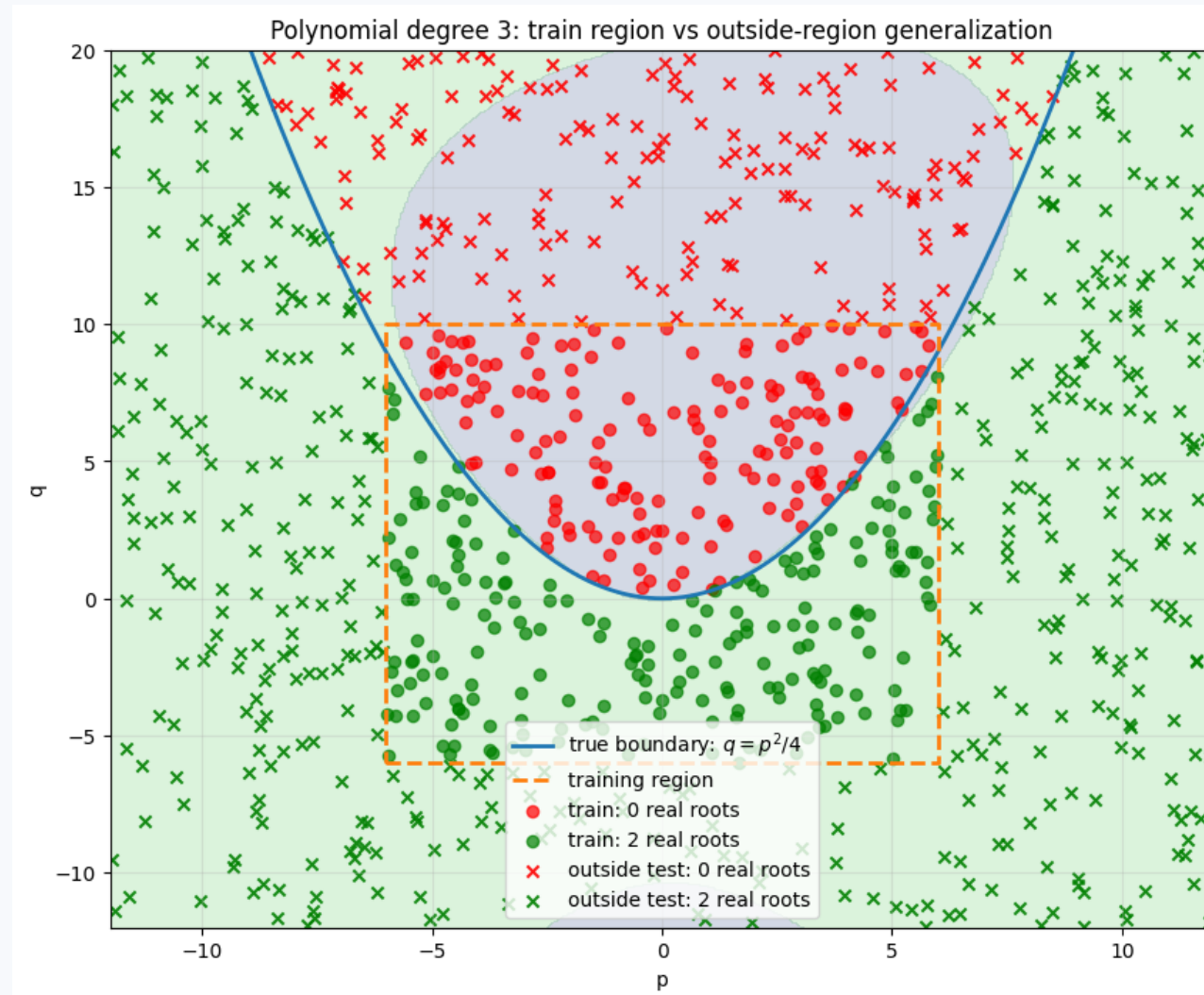
Test accuracy:

0.5160



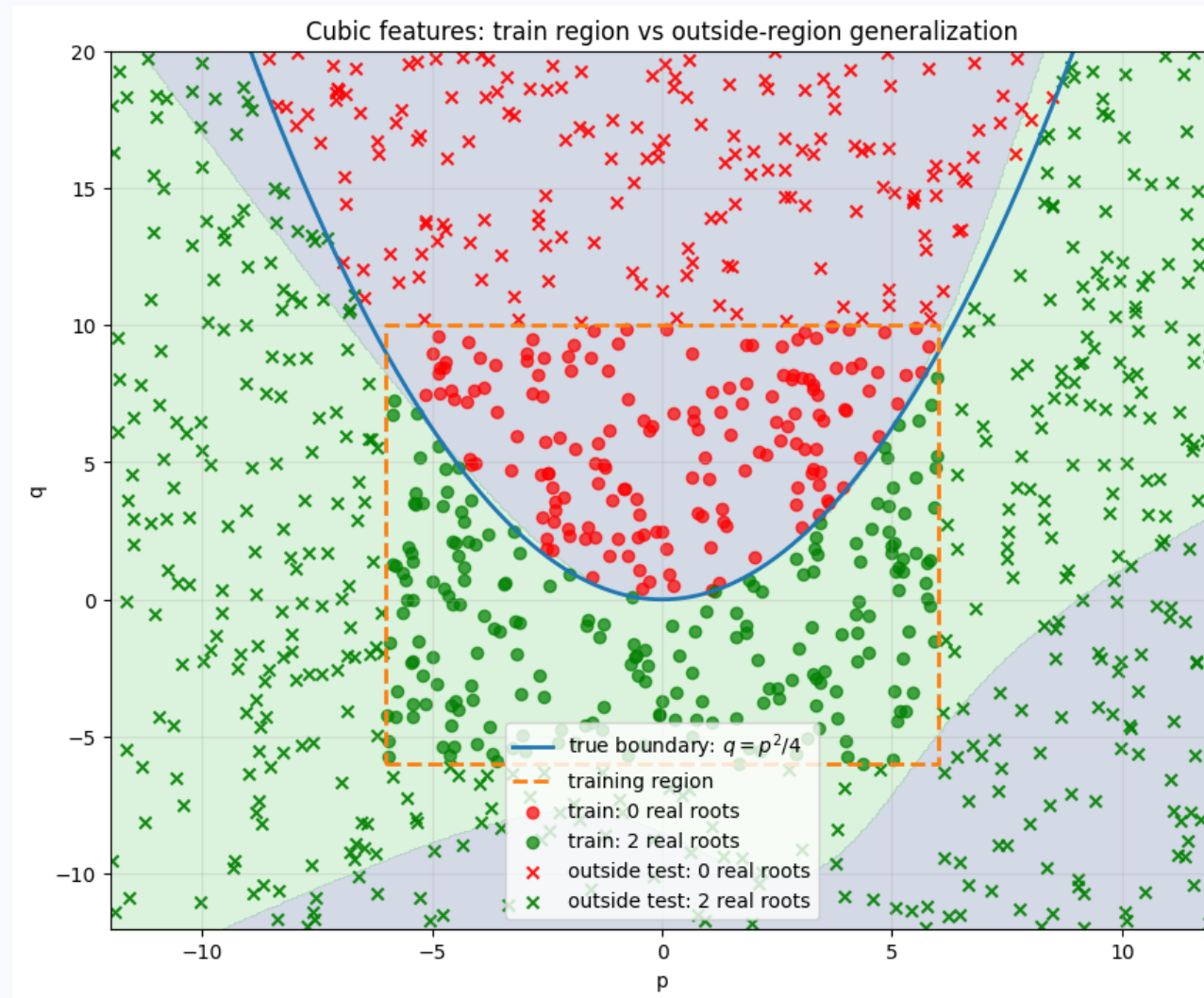
Overfitting / lack of generalization ability

Cubic
features



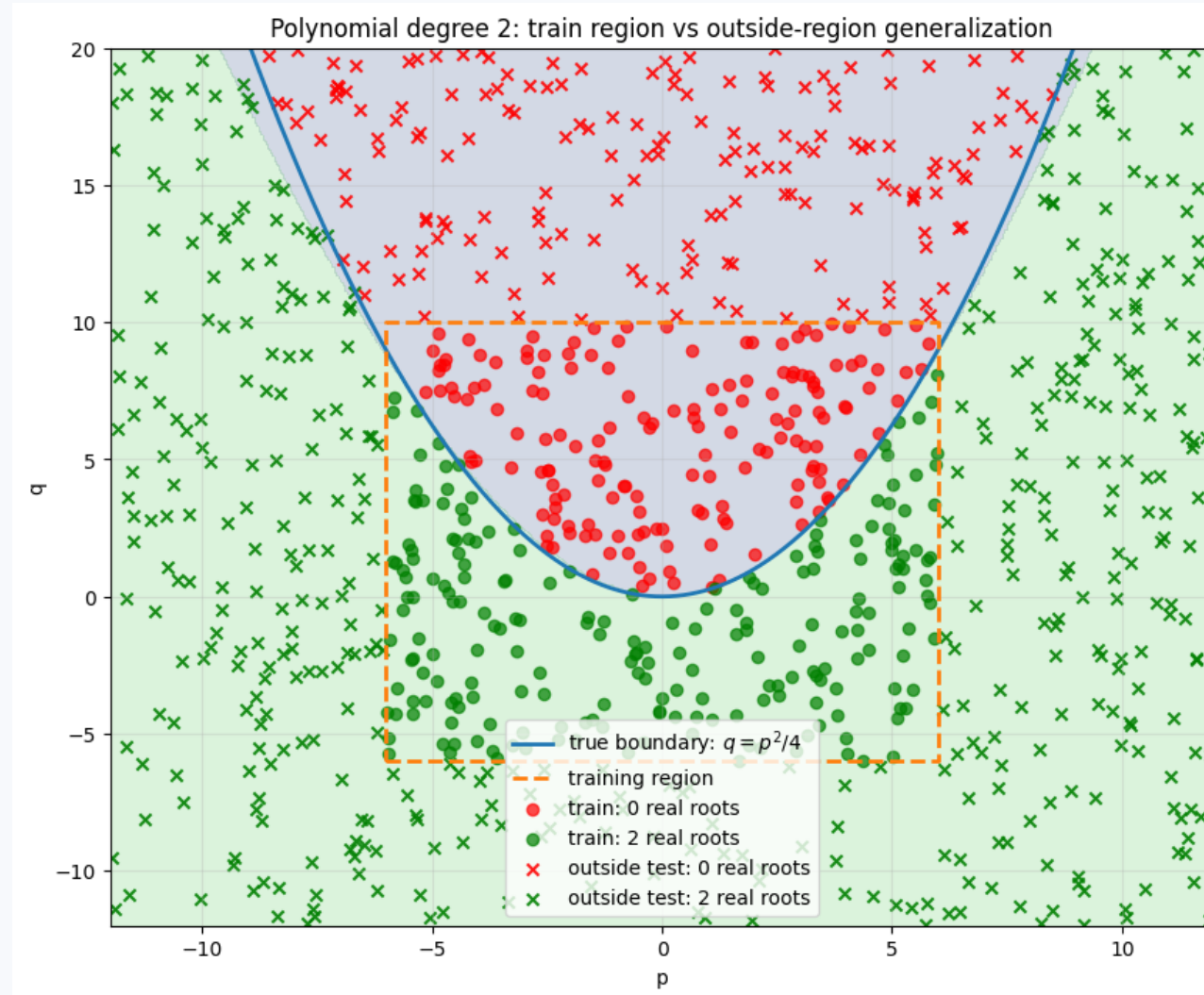
Overfitting / lack of generalization ability

Cubic
features



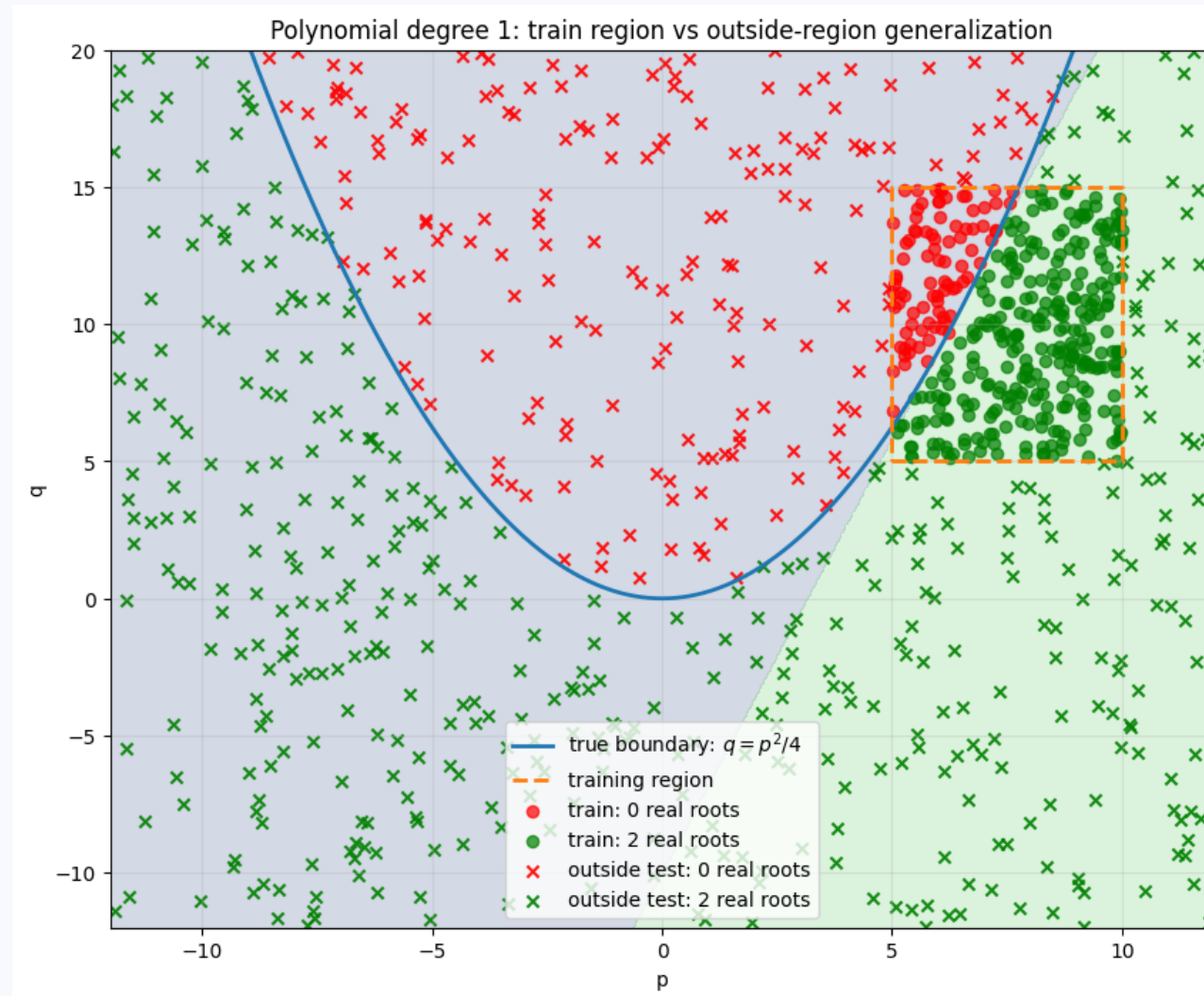
Overfitting / lack of generalization ability

Quadratic
features



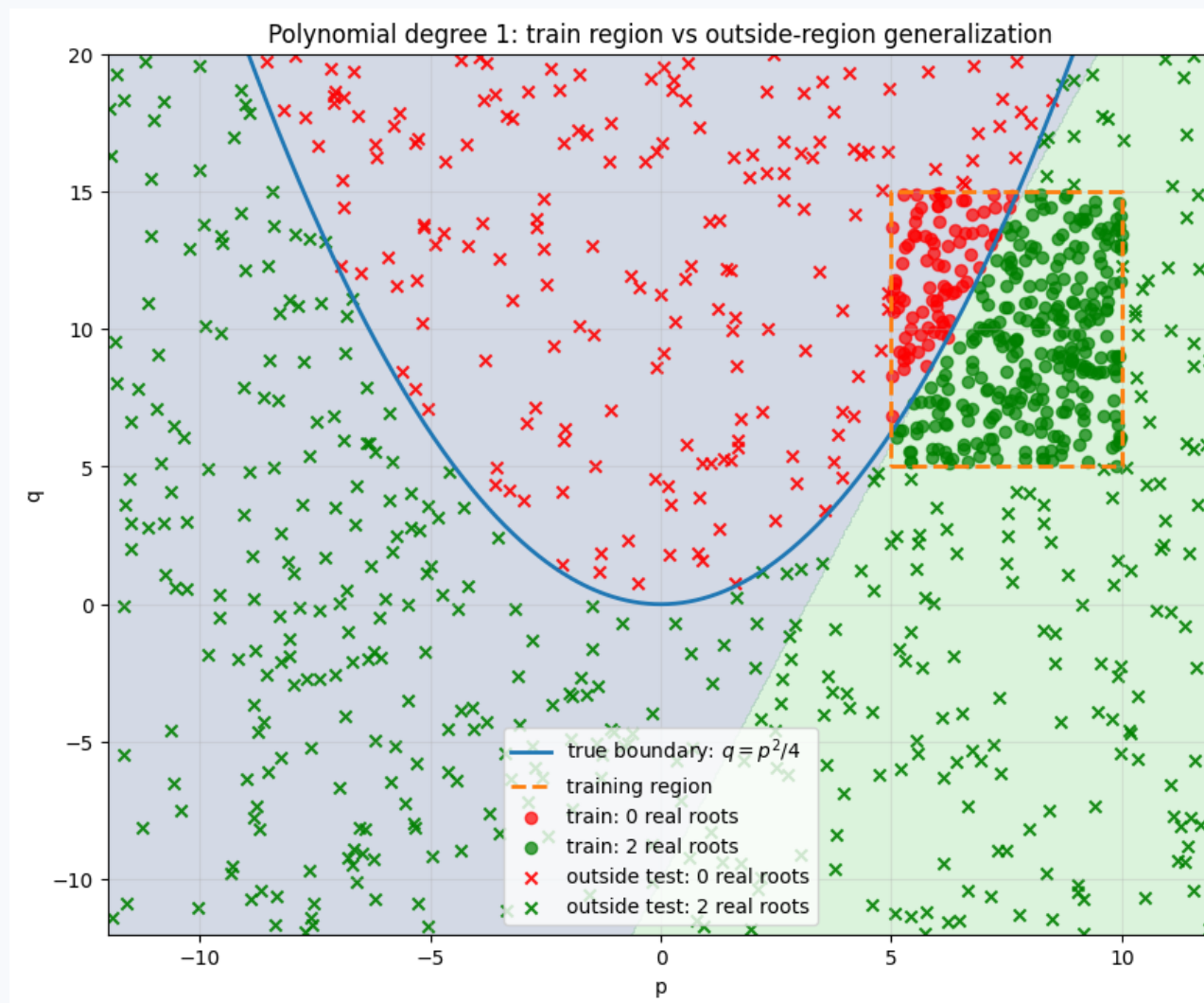
Training data isn't representative

Linear
features

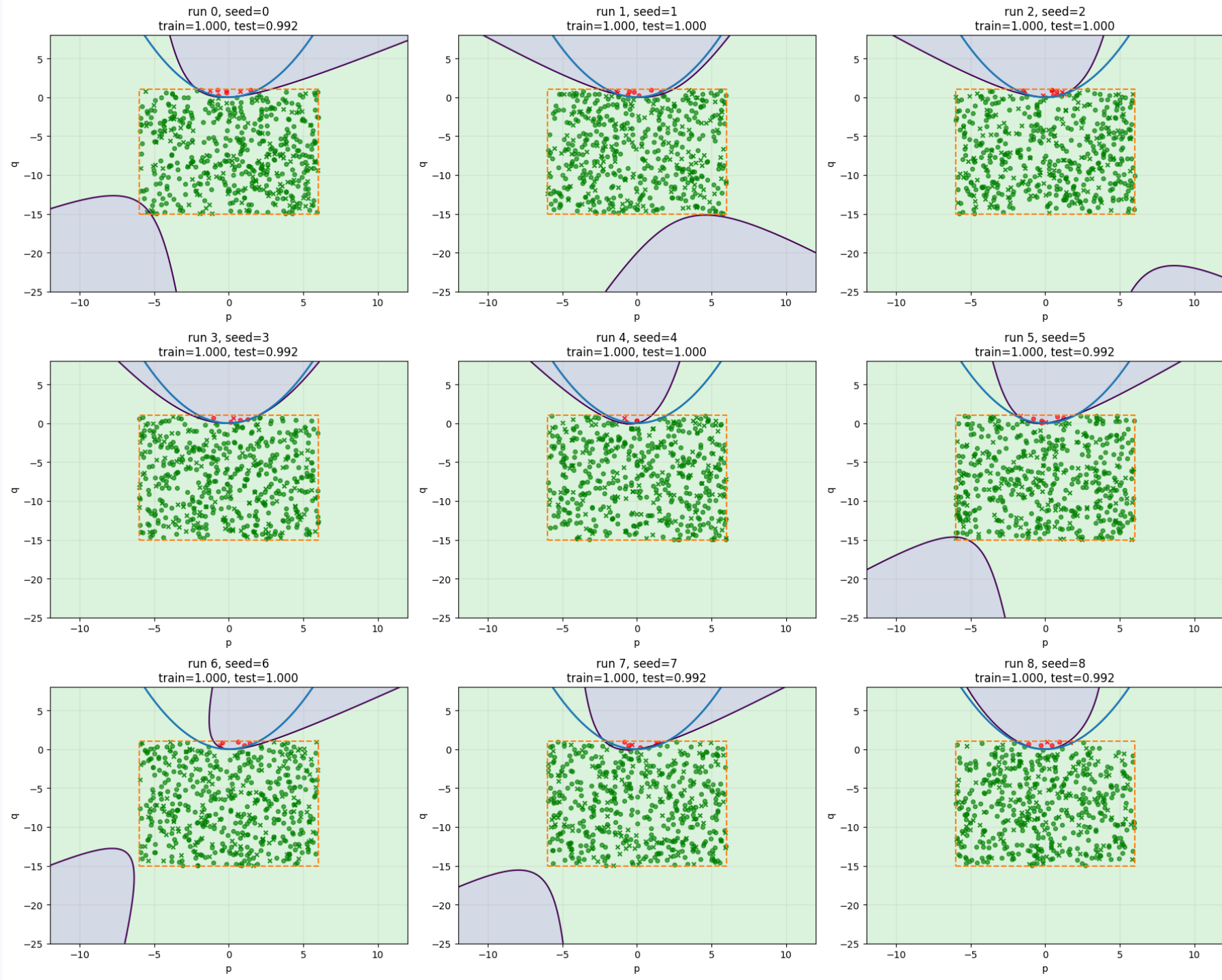


Or this may be your local victory 😊

Linear
features



Class imbalance



The training process goes awry

Next time!

Overcoming
pathology

Strategies of countering pathology

1. Get more data/features
2. Understand the relevant function family and the right distribution to sample data from
3. Evaluate the right things
4. Resort to questionable tricks

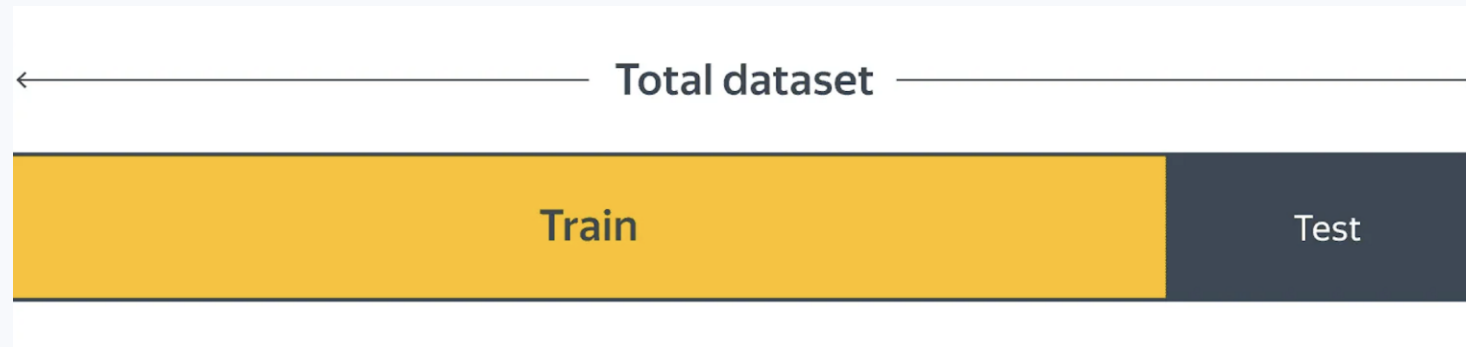
Train/test split

Train set – used to train the model

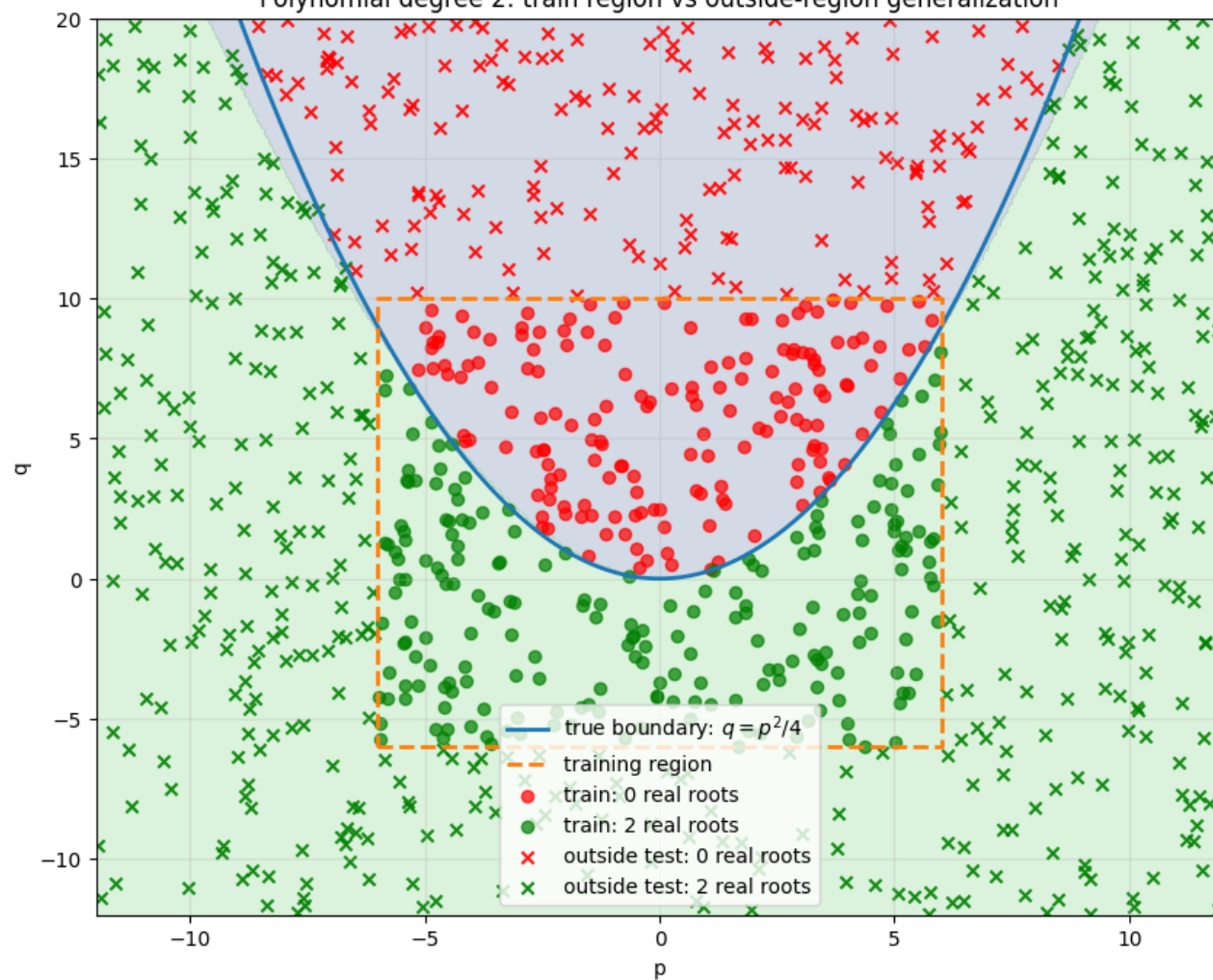
Test set – used to evaluate the model

Split them randomly; fix random seed; stratify by **target**

Train and test set should be from the same data distribution



Polynomial degree 2: train region vs outside-region generalization



Never allow test leak into train!

A typical example of how it happens:

You have a pipeline:

1. Data normalization: $x_i \mapsto \frac{x_i - \mu(x)}{\sigma(x)}$
2. Linear classifier

You compute $\mu(x)$ and $\sigma(x)$ on the **whole** data.

This is wrong! You should compute them only on the training part

Notebook time

Metrics for binary classification

Is accuracy a good metric?

How to understand if we're doing well?

$$\text{accuracy} = \frac{\text{Right guesses}}{\text{Total data}}$$

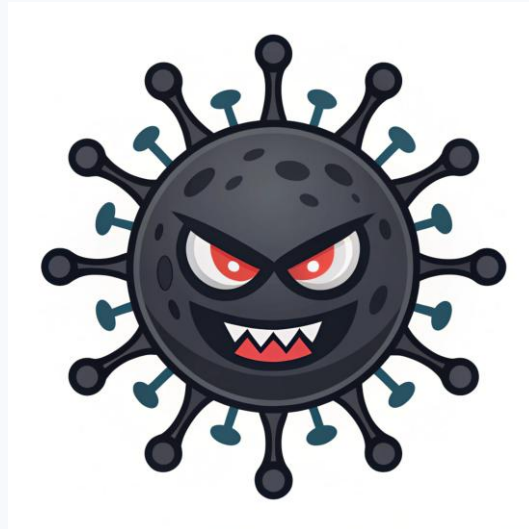
Is accuracy a good metric?

$$\text{accuracy} = \frac{\text{Right guesses}}{\text{Total patients}}$$

Example (Imbalanced classes):



Common cold, 1000



A nasty virus, 20

Is accuracy a good metric?

$$\text{accuracy} = \frac{\text{Right guesses}}{\text{Total patients}}$$

Example (Imbalanced classes):



Common cold, 1000



A nasty virus, 20

Classifying them
all with common
cold gives
98% accuracy

How to choose a metric?

The main question: what is our priority?

Examples:

1. We don't want to miss a single person infected with the nasty virus
2. We don't want to waste our time on treating people with common cold! If we diagnose someone with the nasty virus, we'd better be correct!

The magic square

Notation:

Class 1 = Nasty virus
(**positive**)

Class 0 = Common cold
(**negative**)

	Classified as: Nasty virus	Classified as: Common cold
Nasty virus	True Positive (TP)	False Negative (FN)
Common cold	False Positive (FP)	True Negative (TN)

The magic square

Notation:

Class 1 = Nasty virus
(**positive**)

Class 0 = Common cold
(**negative**)

	Classified as: Nasty virus	Classified as: Common cold
Nasty virus	True Positive (TP)	False Negative (FN)
Common cold	False Positive (FP)	True Negative (TN)

Classified by the model as...

And it is...

The magic square: ideal situation

Notation:

Class 1 = Nasty virus
(**positive**)

Class 0 = Common cold
(**negative**)

	Classified as: Nasty virus	Classified as: Common cold
Nasty virus	True Positive (TP) 20	False Negative (FN) 0
Common cold	False Positive (FP) 0	True Negative (TN) 1000

No serious case missed!

More **positives** (nasty virus) should be **true positives** (classified with the nasty virus). So, we need:

$$\frac{TP}{TP + FN}$$

This metric is called **recall**

	Classified as: Nasty virus	Classified as: Common cold
Nasty virus	True Positive (TP)	False Negative (FN)
Common cold	False Positive (FP)	True Negative (TN)

No penny wasted!

More patients classified with the nasty should be **true positives** (really have the nasty virus). So, we need:

$$\frac{TP}{TP + FP}$$

This metric is called **precision**

	Classified as: Nasty virus	Classified as: Common cold
Nasty virus	True Positive (TP)	False Negative (FN)
Common cold	False Positive (FP)	True Negative (TN)

Hacking recall (both recall = 1)

$$\text{recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

	Classified as: Nasty virus	Classified as: Common cold
Nasty virus	20	0
Common cold	1000	0

	Classified as: Nasty virus	Classified as: Common cold
Nasty virus	20	0
Common cold	0	1000

Hacking precision (precision 1)

$$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

	Classified as: Nasty virus	Classified as: Common cold
Nasty virus	1	19
Common cold	0	1000

	Classified as: Nasty virus	Classified as: Common cold
Nasty virus	20	0
Common cold	0	1000

Combining precision and recall

Maximize precision while keeping recall > 0.9

Or vice versa

An attempt at fusing precision and recall

F1 score is their harmonic mean:

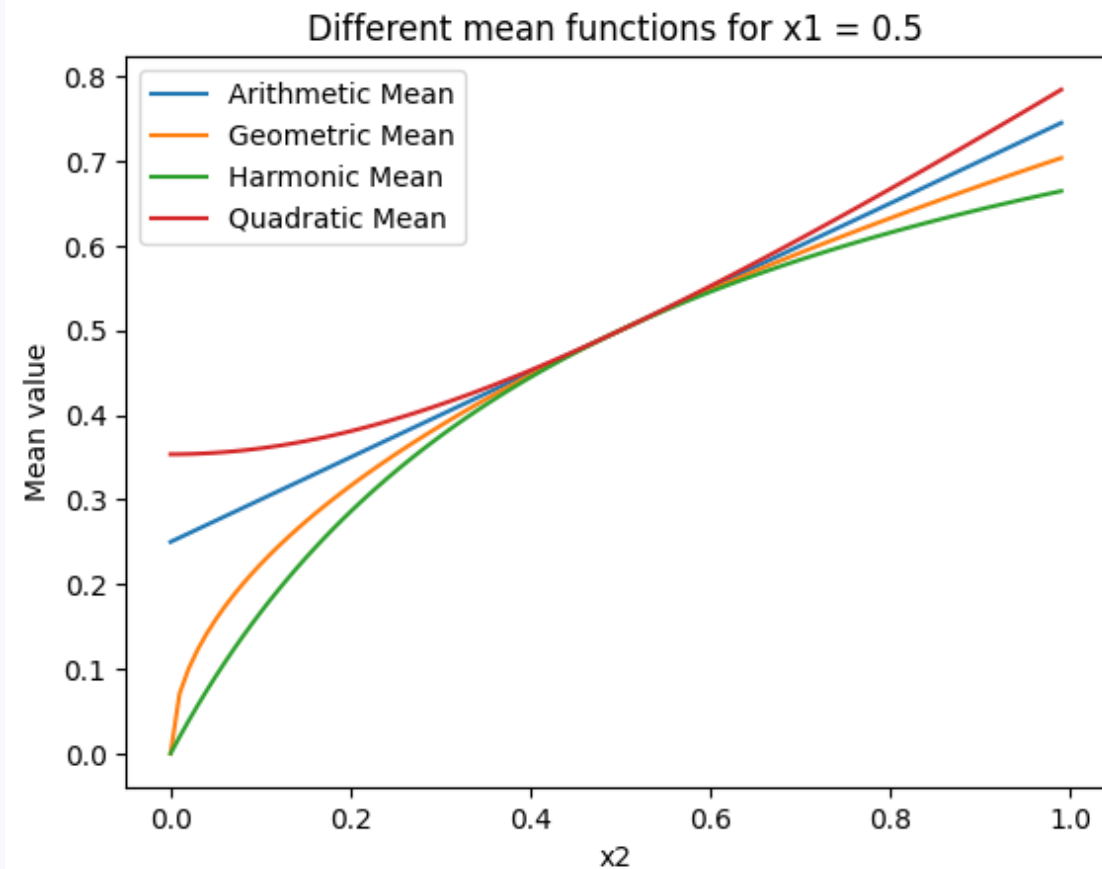
$$F1 = \frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

Math:

$$\frac{1}{F1} = \frac{1}{2} \left(\frac{1}{\text{precision}} + \frac{1}{\text{recall}} \right)$$

Among other means, it is the pessimistic one:

$$\frac{2ab}{a+b} \leq \sqrt{ab} \leq \frac{a+b}{2} \leq \sqrt{\frac{a^2+b^2}{2}}$$



Class probabilities in
binary classification and
where to get them

Logits in binary case

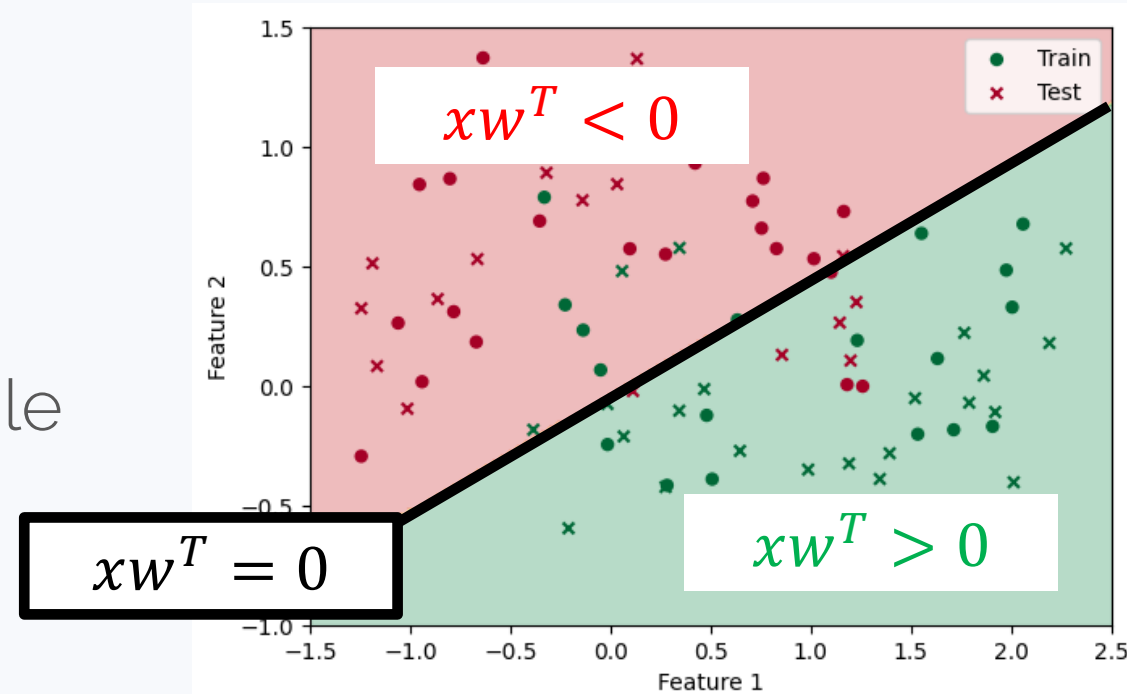
A **logit** is

$$f(x, w) = xw^T$$

This is like a “class 1 affiliation score”, while “class 0 score is” $-xw^T$.

Class 0

Class 1



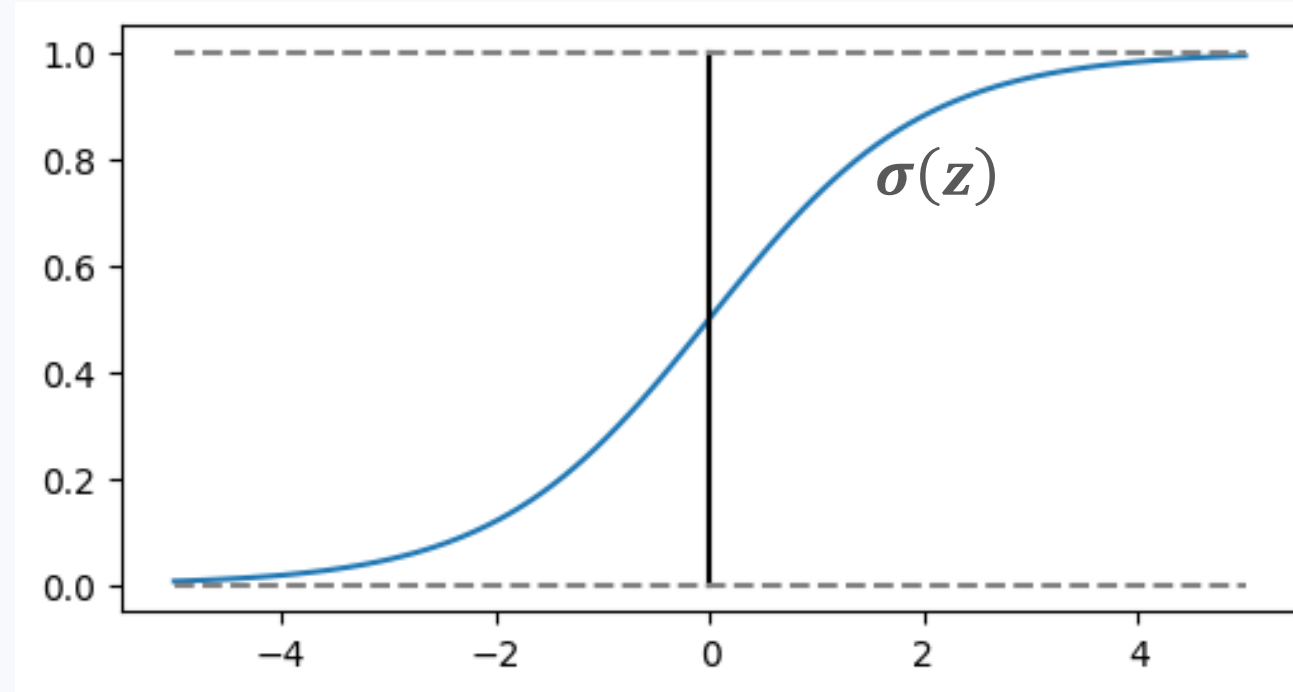
$$xw^T = (x_1, \dots, x_D) \begin{pmatrix} w_1 \\ \vdots \\ w_D \end{pmatrix}$$

Logistic regression

Logistic regression is a particular type of linear classifier:

Class 1 probability = $\sigma(w_0 + xw^T)$

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



Logistic regression

Logistic regression is a particular type of linear classifier:

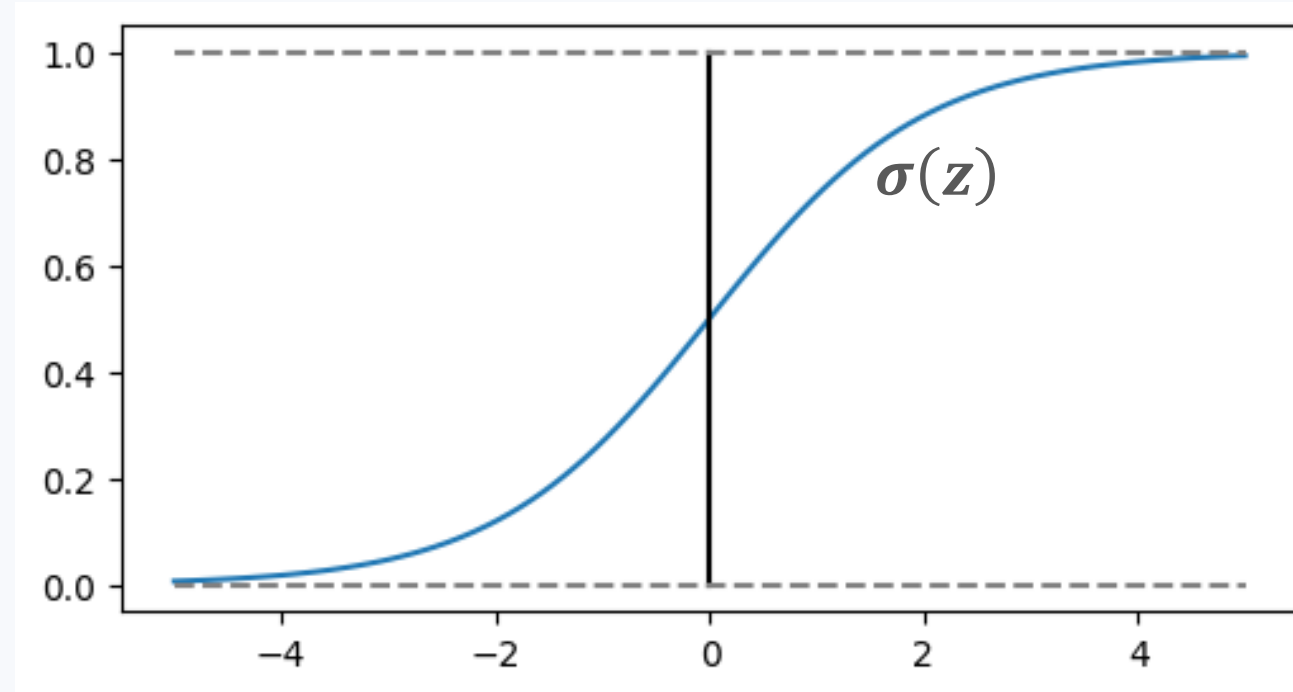
Class 1 probability $= \sigma(w_0 + xw^T)$

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

An important property:

$$1 - \sigma(z) = \sigma(-z)$$

It's class 0 probability



But what does class probability mean?

A data point is either class 0 or class 1, how can we say that it has probability 0.3?

But what does class probability mean?

A data point is either class 0 or class 1, how can we say that it has probability 0.3?

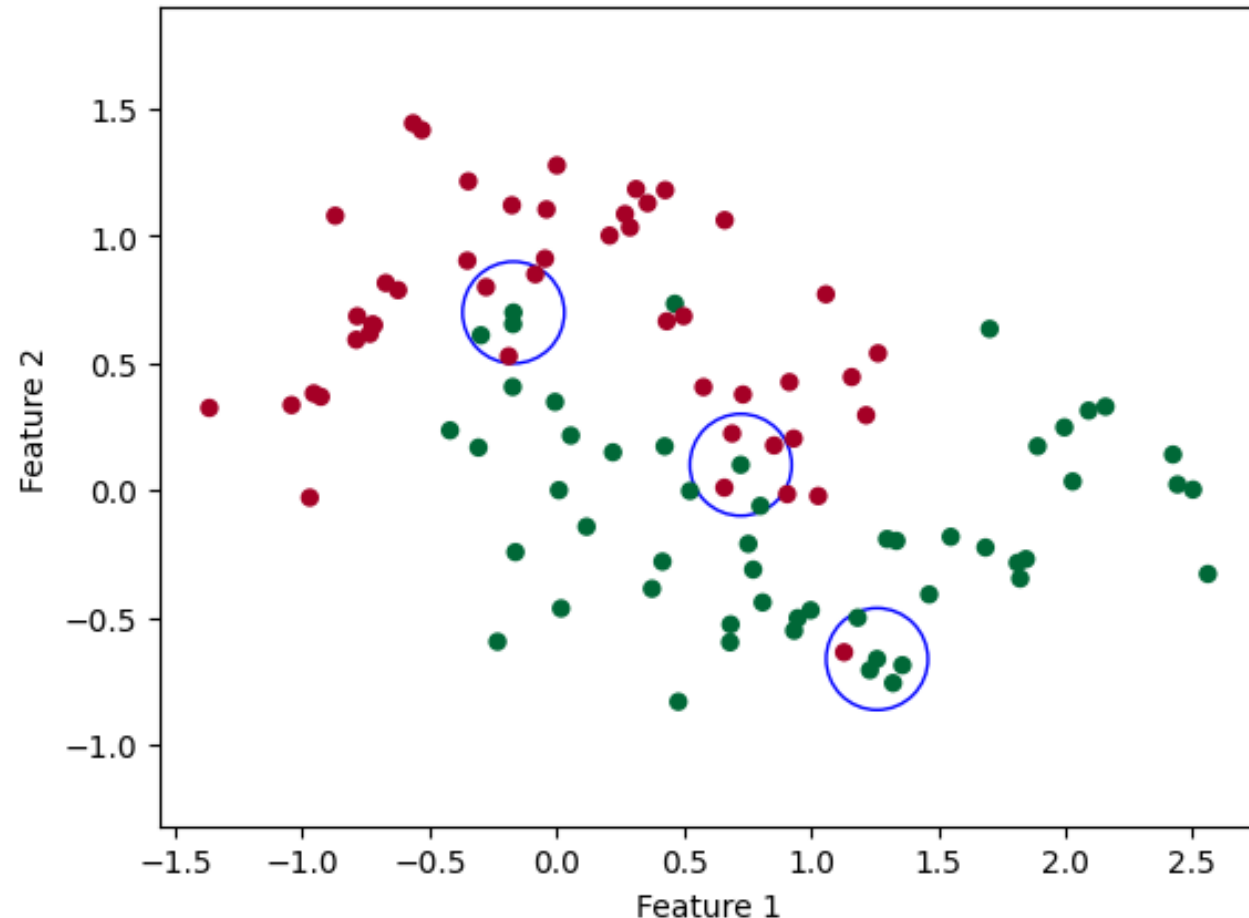
These three points are both **class 1**.

But in their neighborhoods, we have different probabilities of **class 1**:

Top: 0.5

Middle: 0.4

Bottom: 5/6

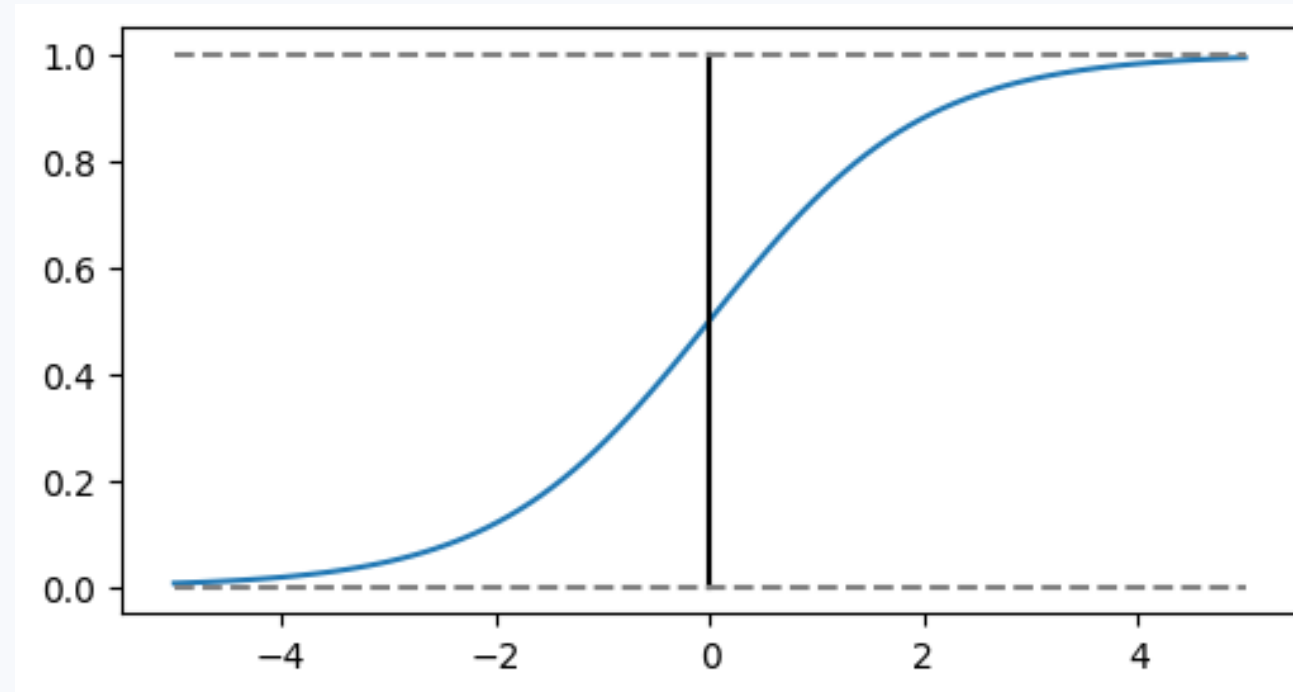


Does logistic regression predict correct probabilities?

Logistic regression is a particular type of linear classifier:

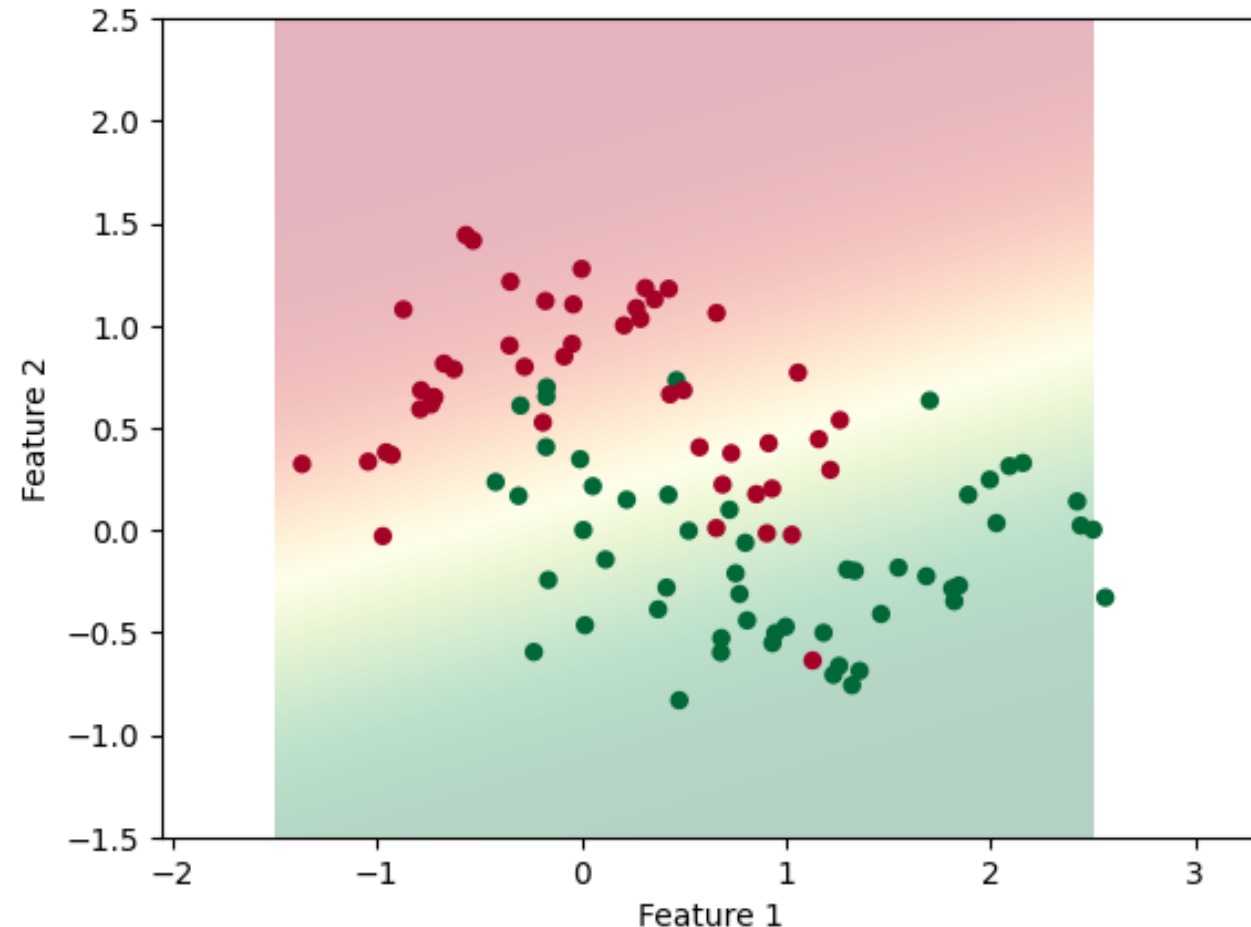
Class 1 probability = $\sigma(w_0 + xw^T)$

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

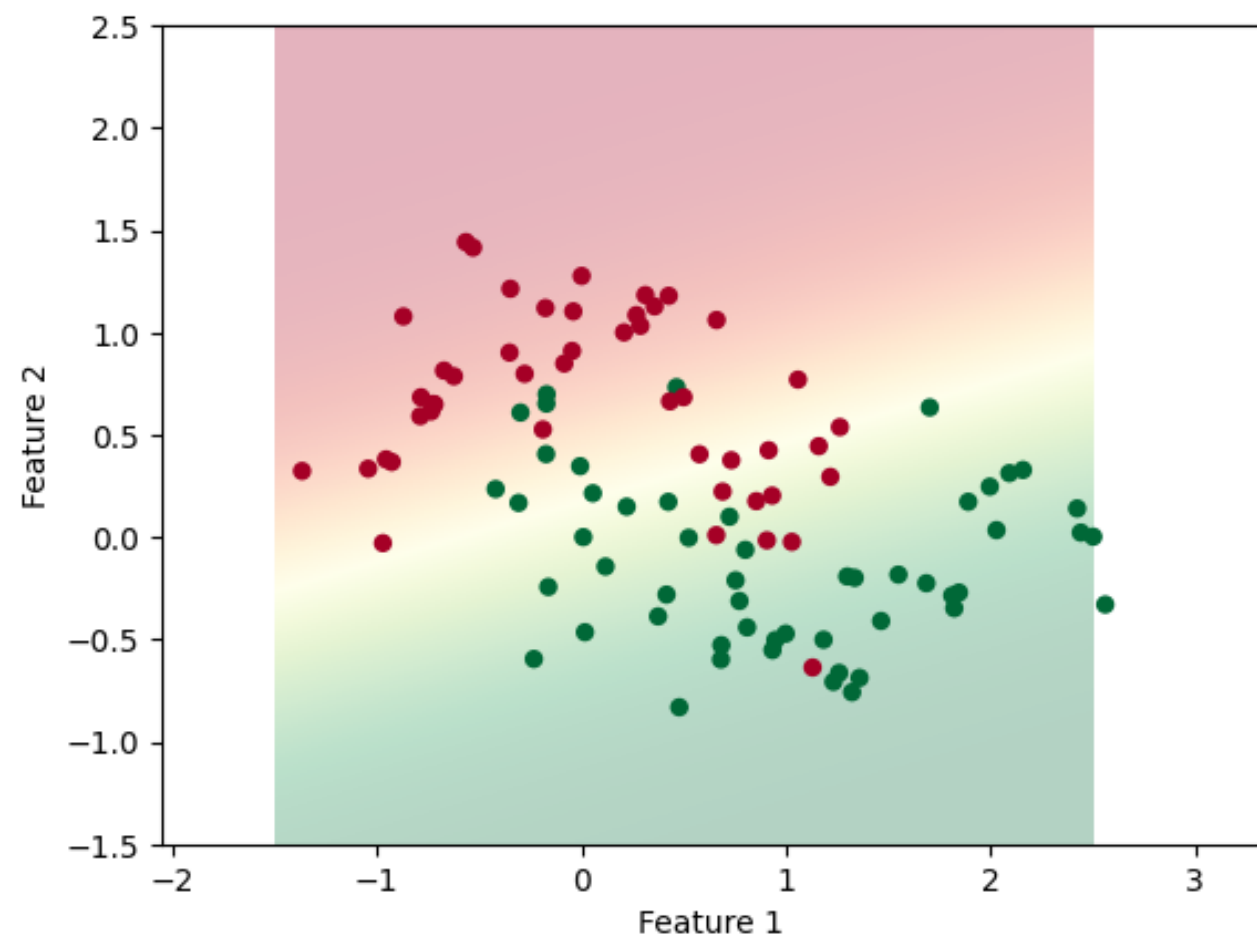
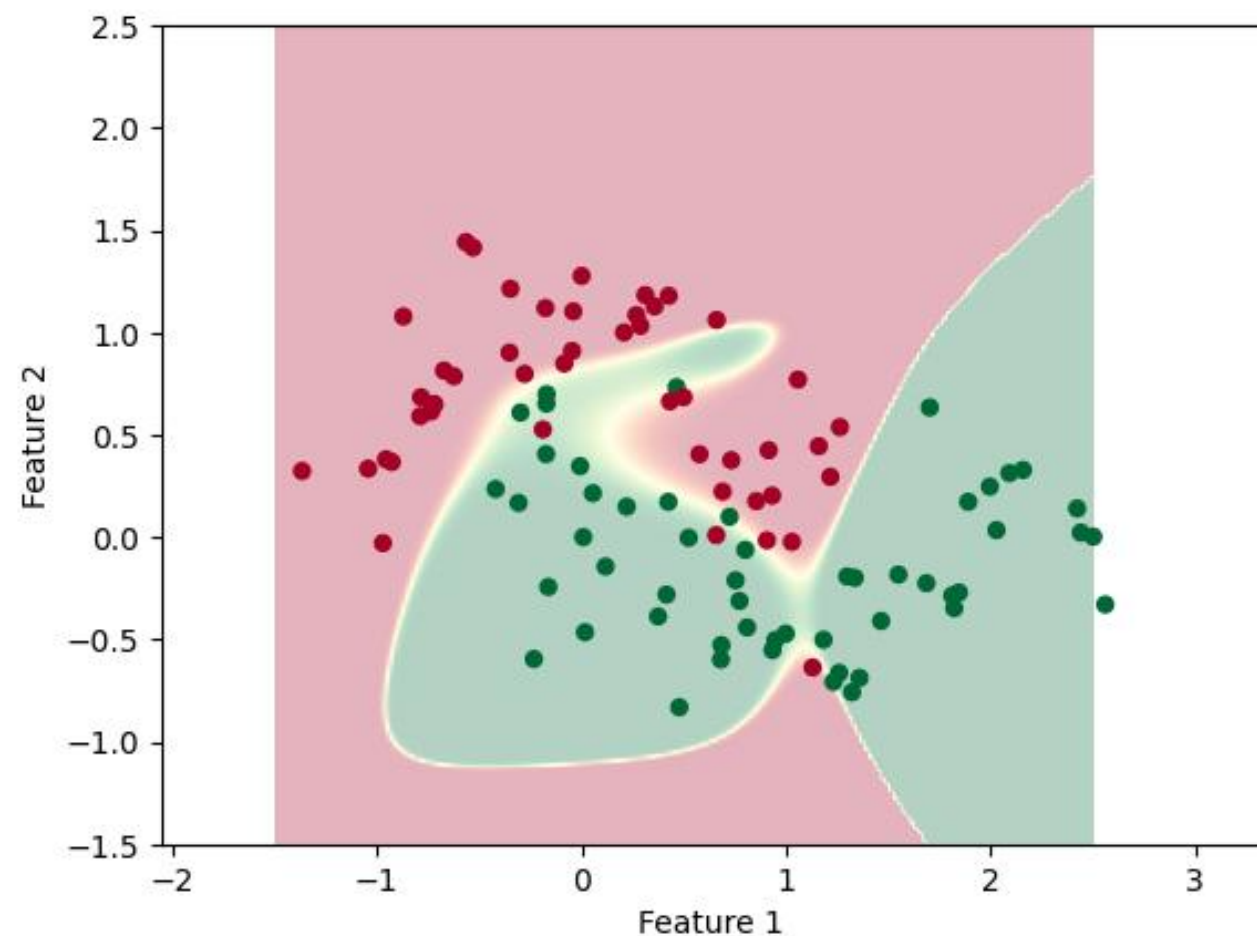


Does logistic regression predict correct probabilities?

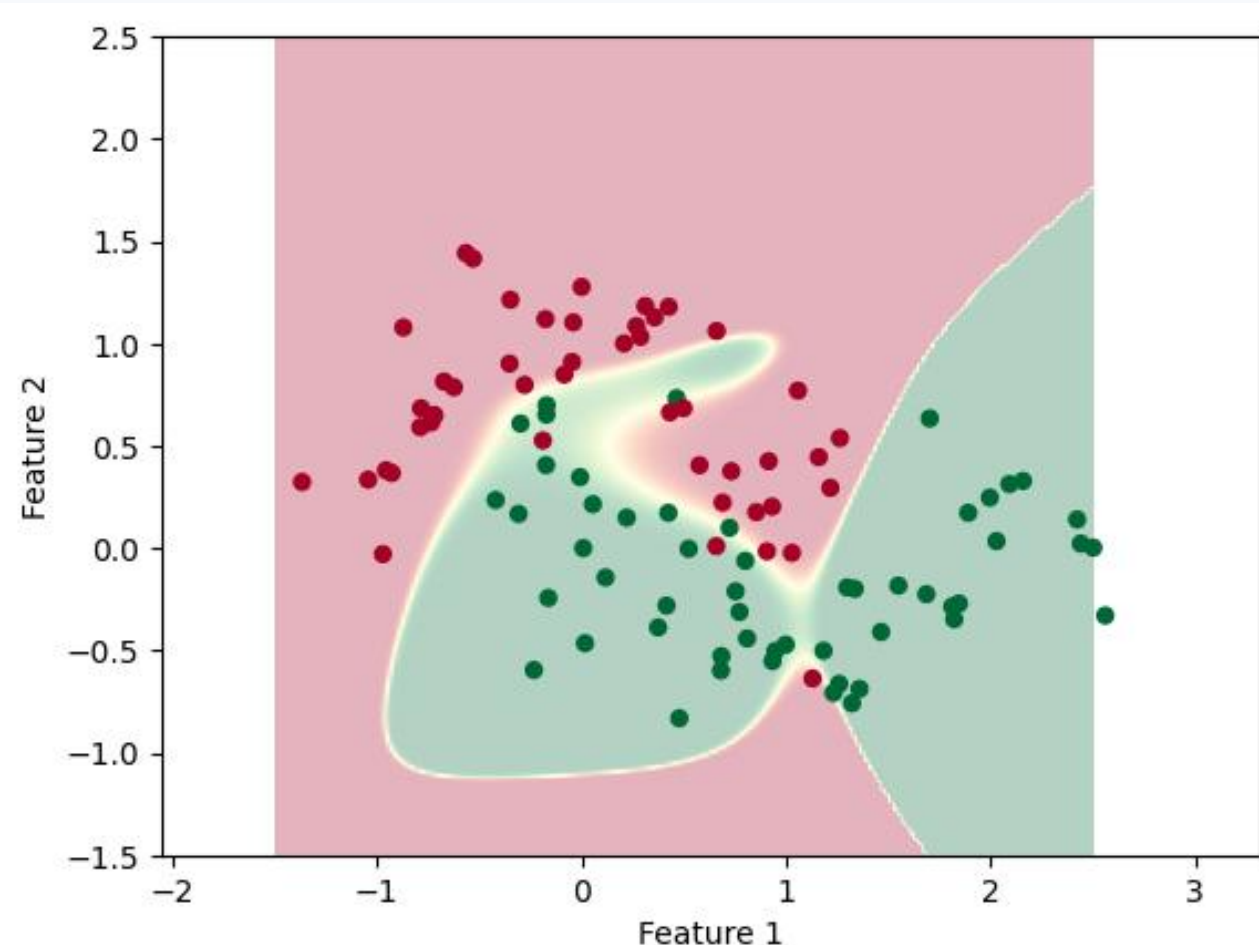
Well, it tries, but it is not guaranteed to succeed.



Does logistic regression predict correct probabilities?



Does logistic regression predict correct probabilities?



This illustrates **overfitting**:

the model does a great job on training data, but the rule is unnatural and wouldn't generalize to test data.

Moreover, the model is **overconfident**

How to train a model

Two parts of the answer

We need to somehow get w :

$$f(x, w) = xw^T$$

Part 1: Loss function. We're searching for an optimal w , and the loss function tells us **what** we want to optimize.

$$\mathcal{L}(X_{train}, y_{train}, w_{better}) > \mathcal{L}(X_{train}, y_{train}, w_{worse})$$

Part 2: Optimization method. Tells us **how** do we find the optimal w . [next week]

Loss functions for linear models

What is a loss function

y_i – true answers, $\hat{y}_i$ – predicted answers

A loss function

$$\mathcal{L}(y, \hat{y}) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}(y_i, \hat{y}_i)$$

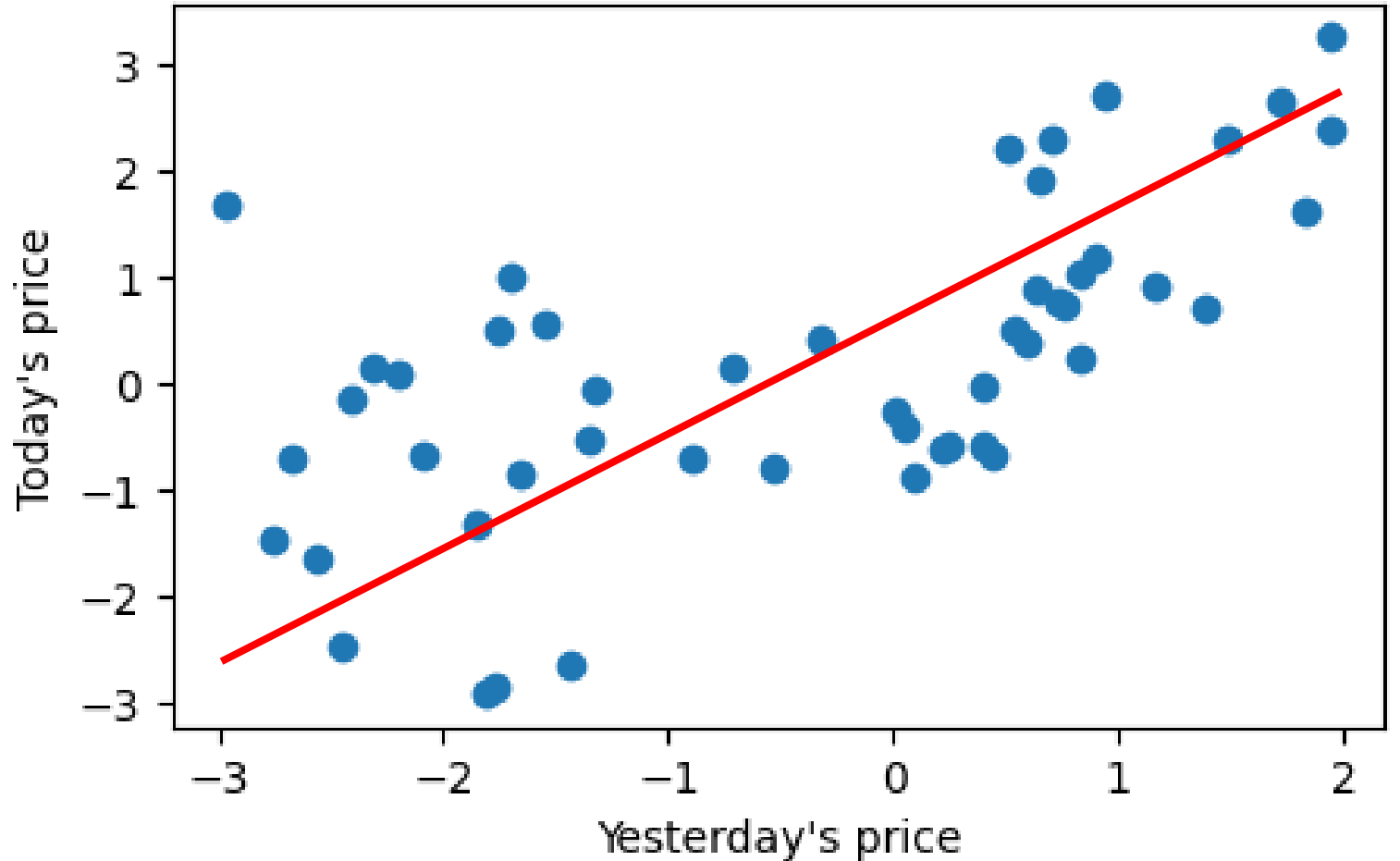
shows how much we deviate from the truth. And it should be **optimizable**.

Choosing the loss = choosing which types of mistakes are worse
Usually, lower loss $\sim$ better model

Example: linear regression

Predicting the value
with a linear function:

$$\hat{y}_i = w_0 + xw^T$$



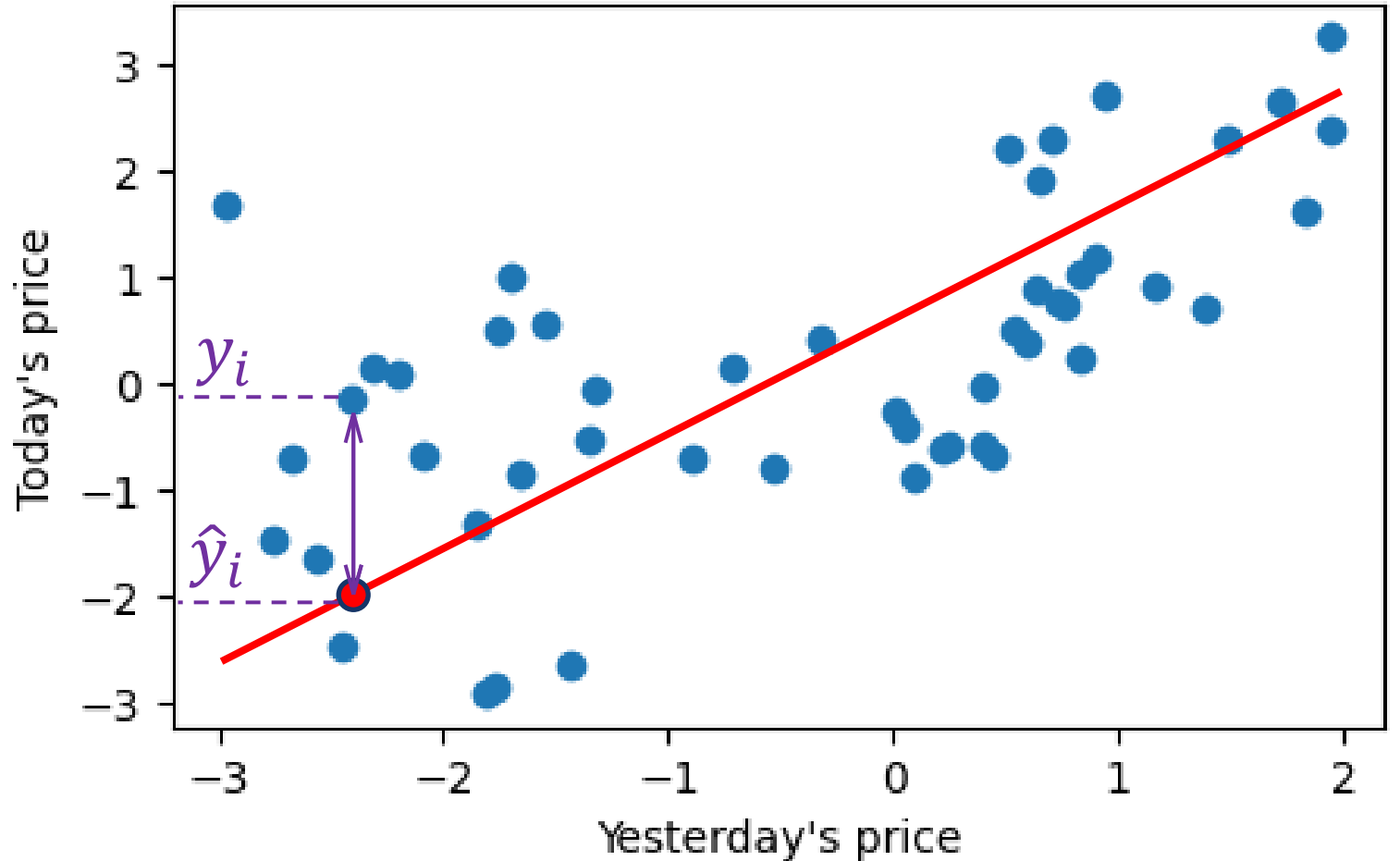
Example: linear regression

Loss

$$\mathcal{L}(y, \hat{y}) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}(y_i, \hat{y}_i)$$

y_i — true value

$\hat{y}_i$ — predicted value



Example: linear regression

Loss

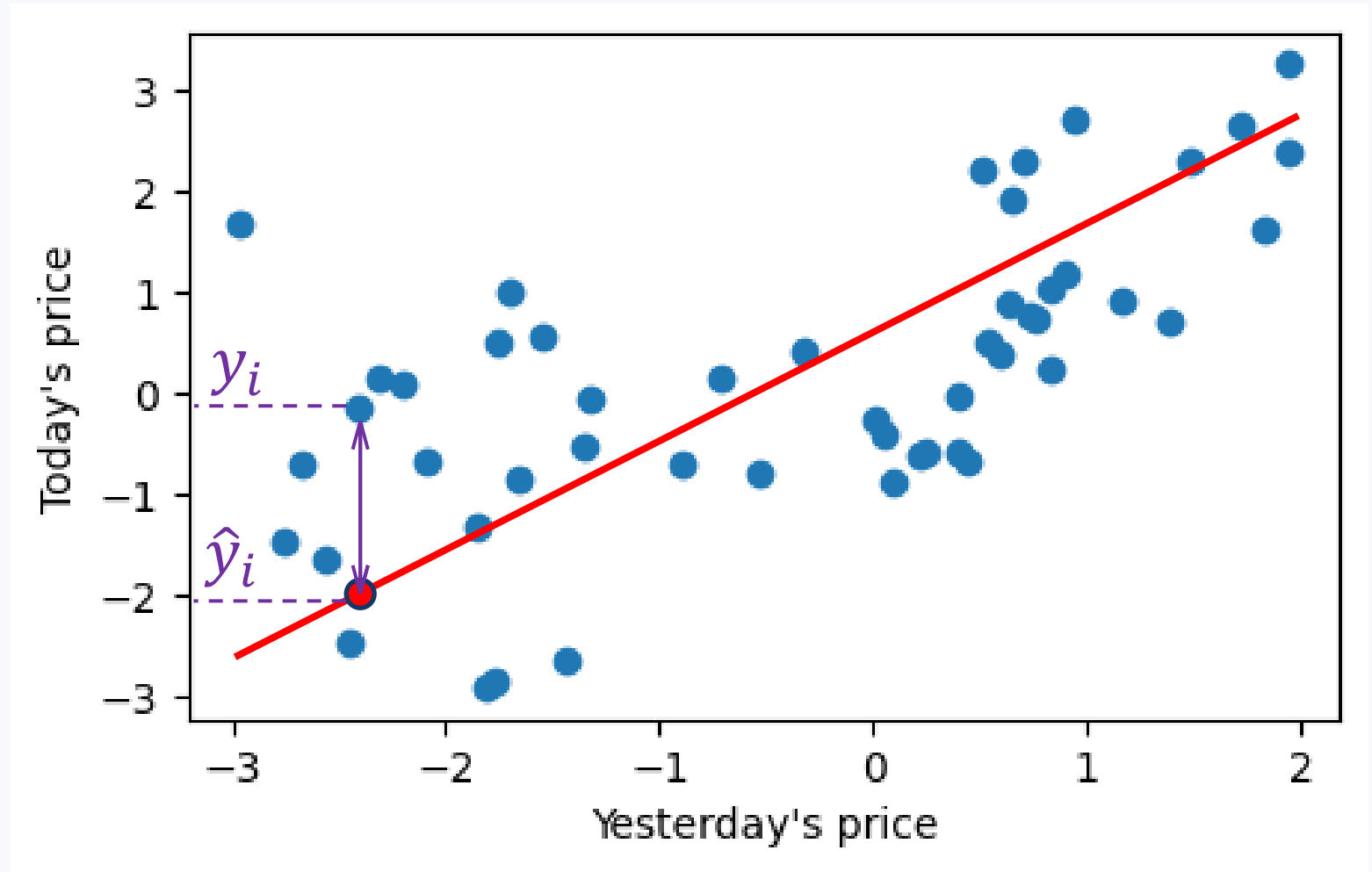
$$\mathcal{L}(y, \hat{y}) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}(y_i, \hat{y}_i)$$

y_i — true value

$\hat{y}_i$ — predicted value

Sanity check: why not

$$\mathcal{L}(y_i, \hat{y}_i) = (y_i - \hat{y}_i)?$$

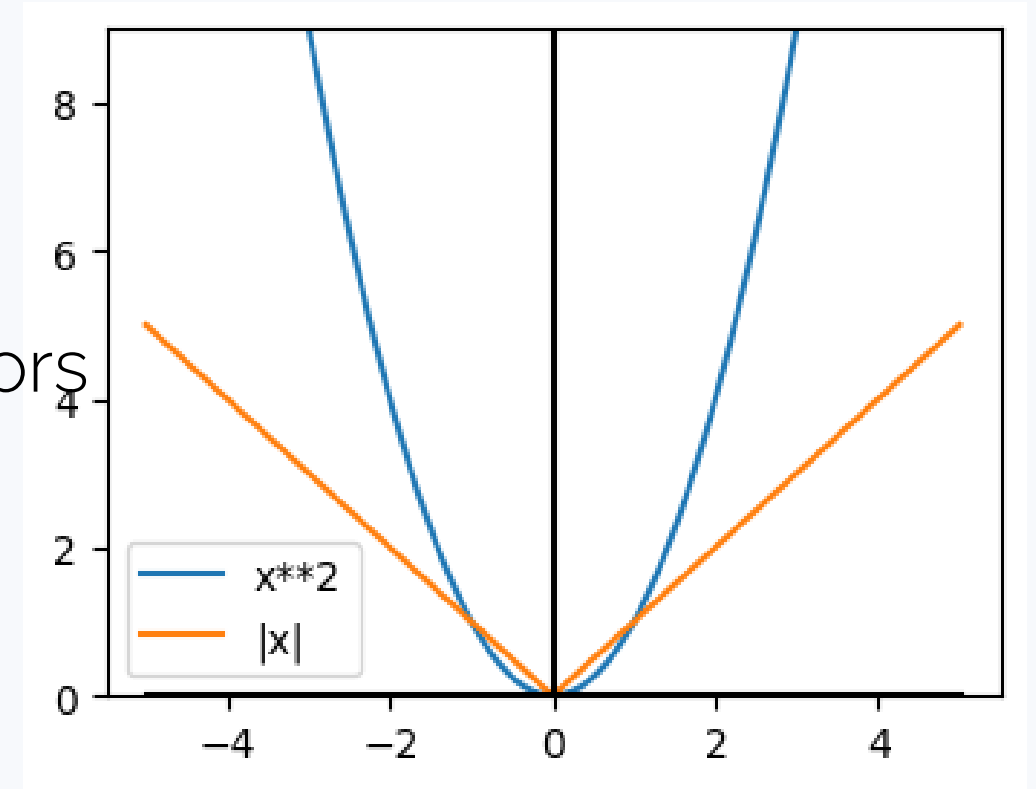


Example: linear regression

$$\mathcal{L}(y_i, \hat{y}_i) = (y_i - \hat{y}_i)^2$$

is penalizing more harshly for large errors
than

$$\mathcal{L}(y_i, \hat{y}_i) = |y_i - \hat{y}_i|$$



Why is it important?

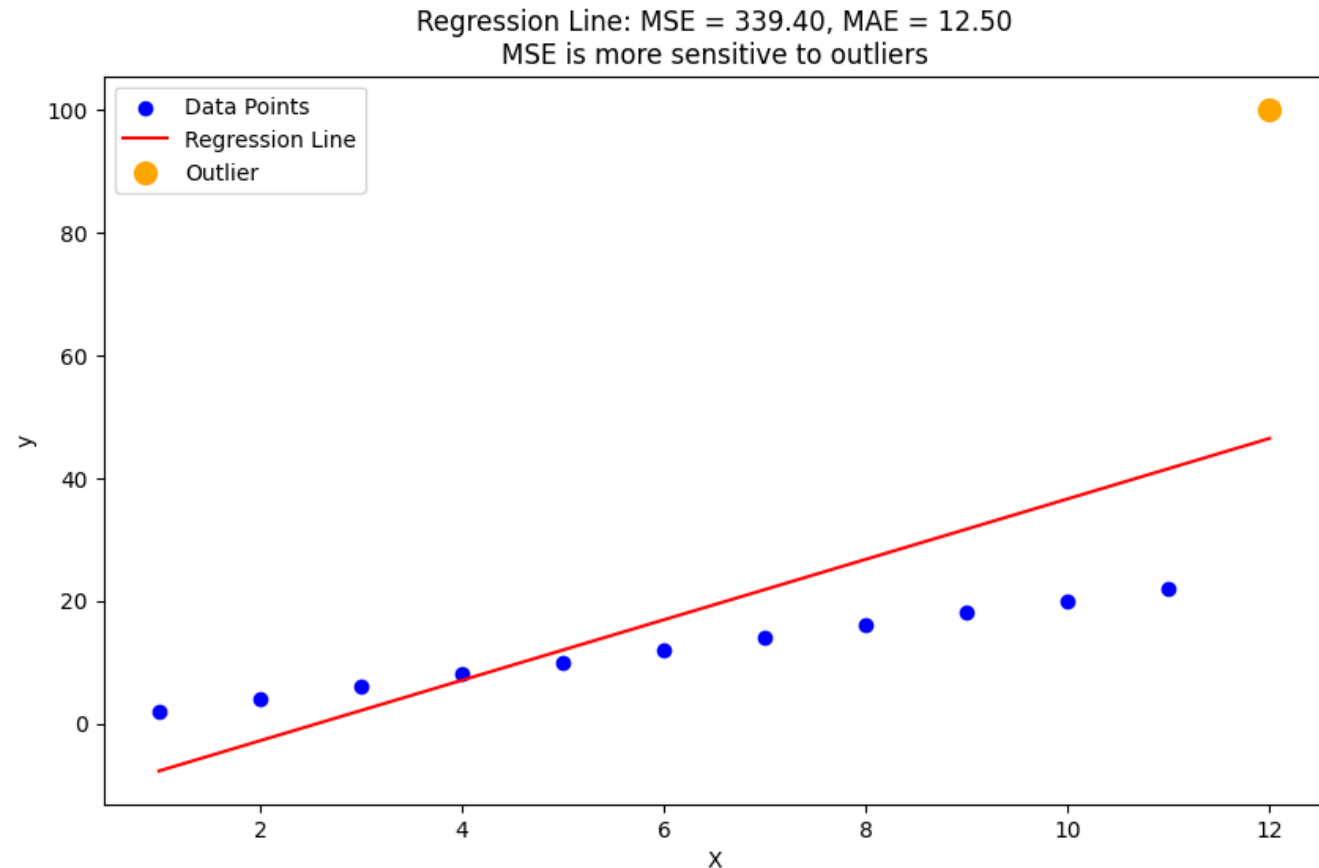
Example: linear regression

$\mathcal{L}(y_i, \hat{y}_i) = (y_i - \hat{y}_i)^2$ is penalizing more harshly for large errors than

$$\mathcal{L}(y_i, \hat{y}_i) = |y_i - \hat{y}_i|$$

Why is it important?

Less problems from outliers.



Regression losses

y_i – true answers, $\hat{y}_i$ – predicted answers

MSE (Mean Squared Error)

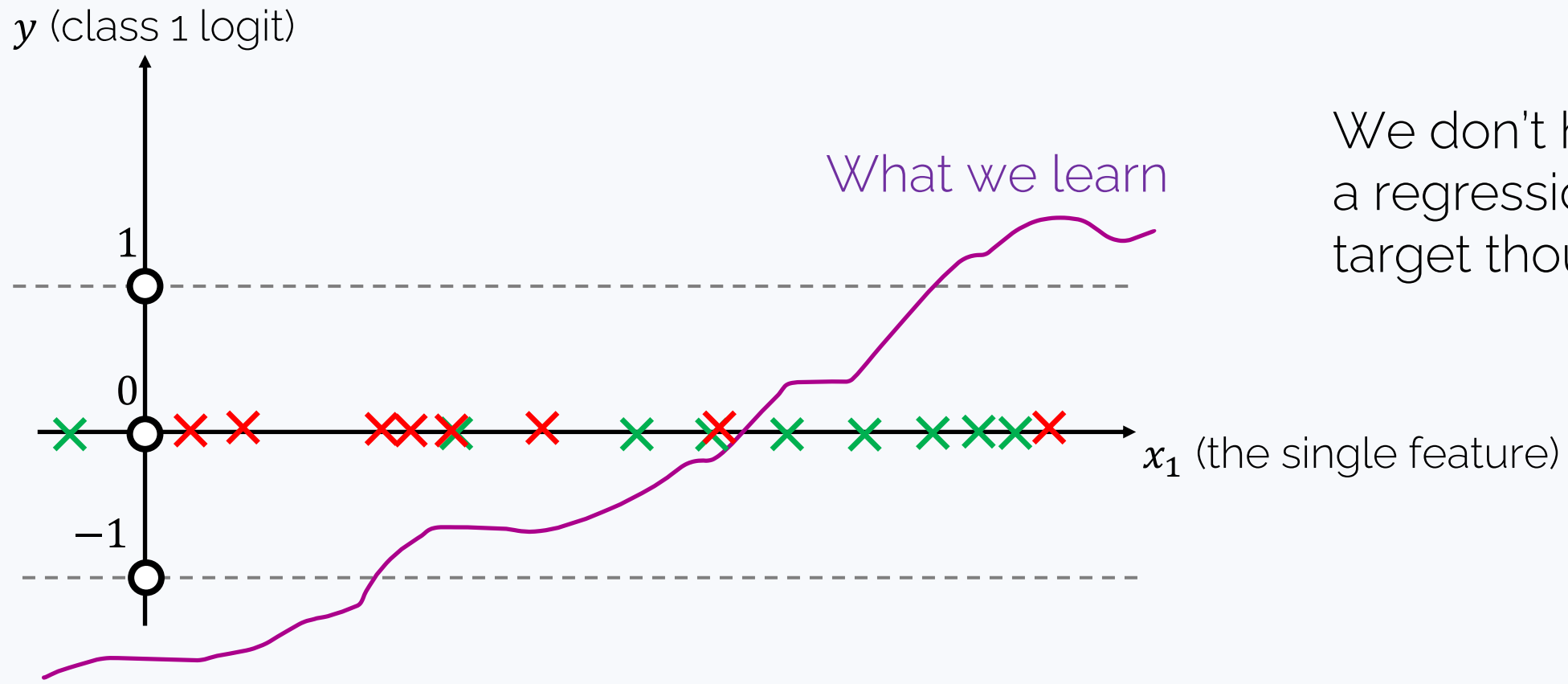
$$\mathcal{L}(y, \hat{y}) = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

MAE (Mean Absolute Error)

$$\mathcal{L}(y, \hat{y}) = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|^2$$

Linear classification is... regression!

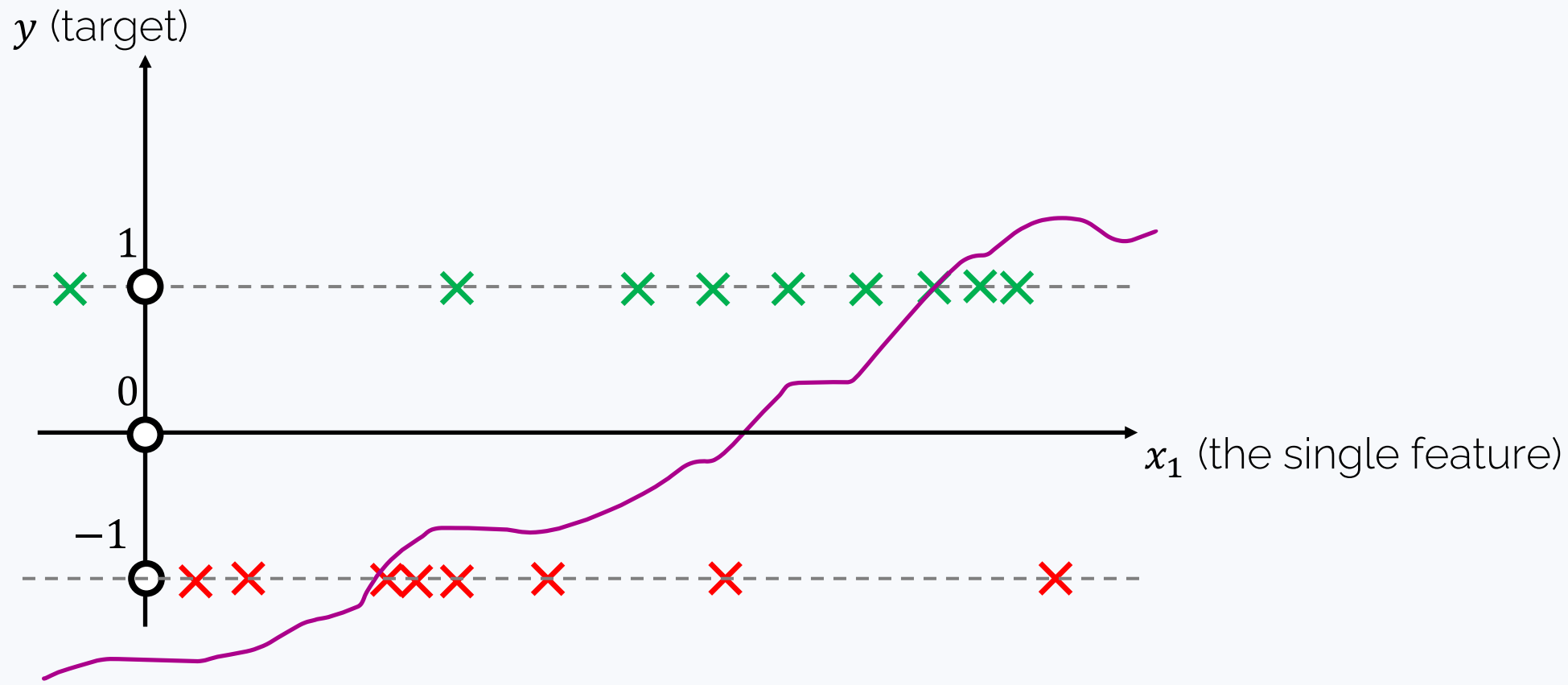
A linear model outputs not a class index, but a real number, a “class 1 score”. We may suppose that > 0 means “class 1”.



We don't have a regression target though...

Attempt no. 1

Let class labels -1 and 1 be our target



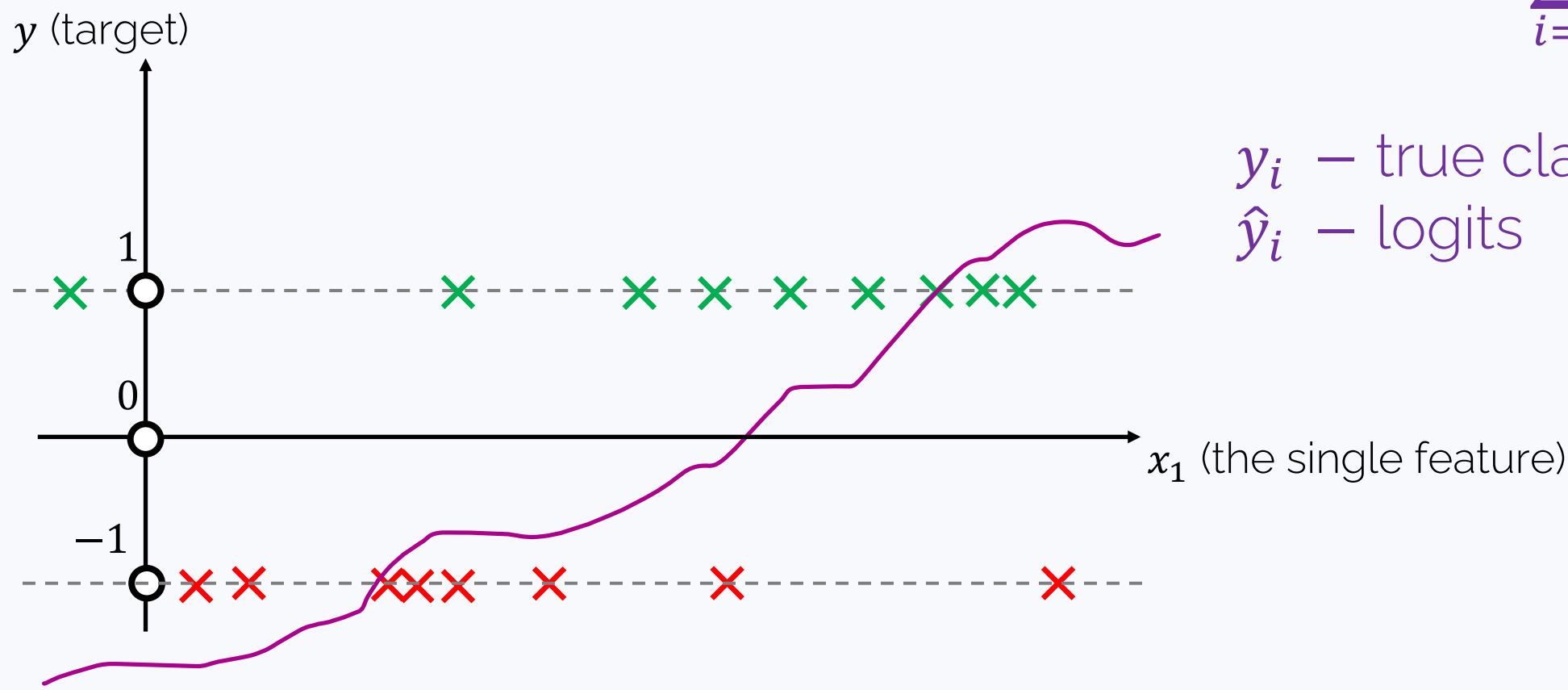
Attempt no. 1

Let class labels -1 and 1 be our target

Loss

$$\mathcal{L}(y, \hat{y}) = \sum_{i=1}^N \mathcal{L}(y_i, \hat{y}_i)$$

y_i – true class labels
 $\hat{y}_i$ – logits



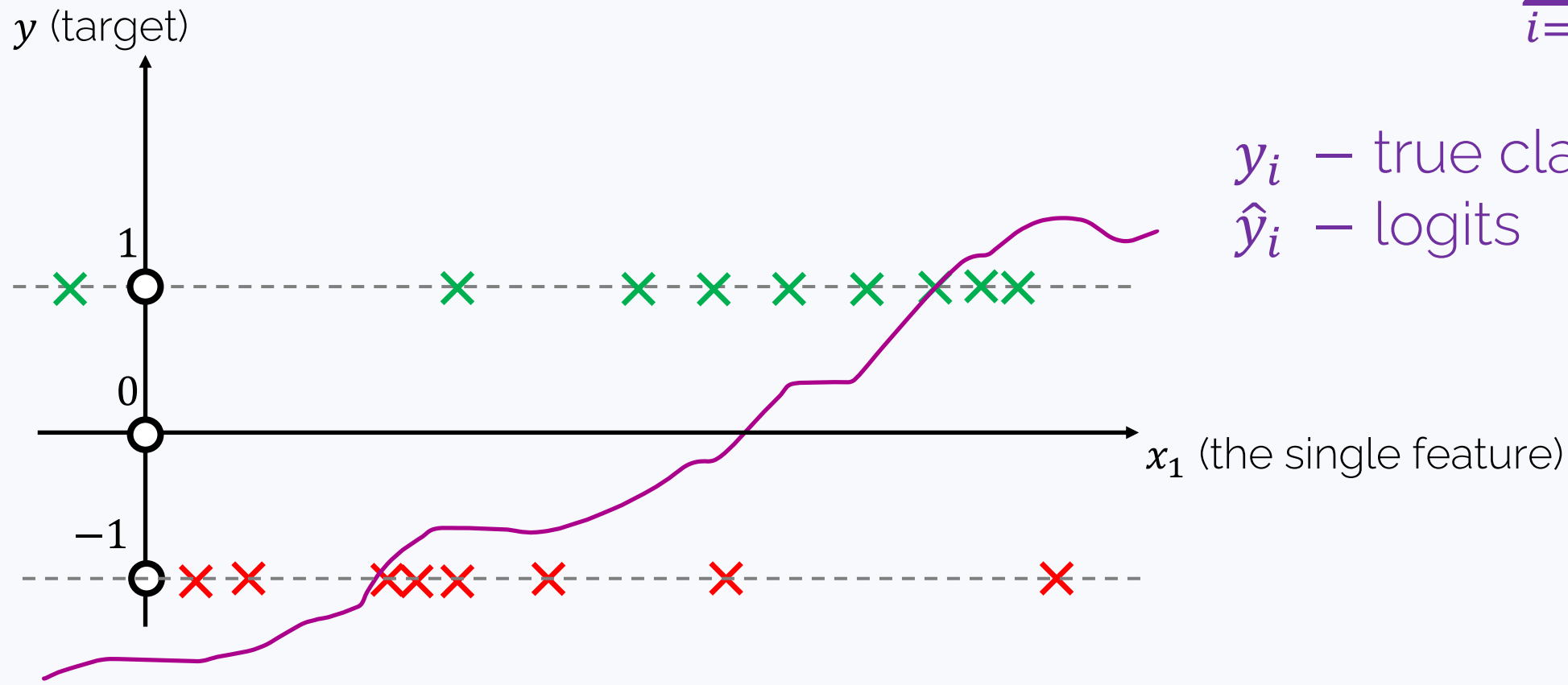
Attempt no. 1. MSE loss

Let class labels -1 and 1 be our target

MSE Loss

$$\mathcal{L}(y, \hat{y}) = \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

y_i – true class labels
 $\hat{y}_i$ – logits



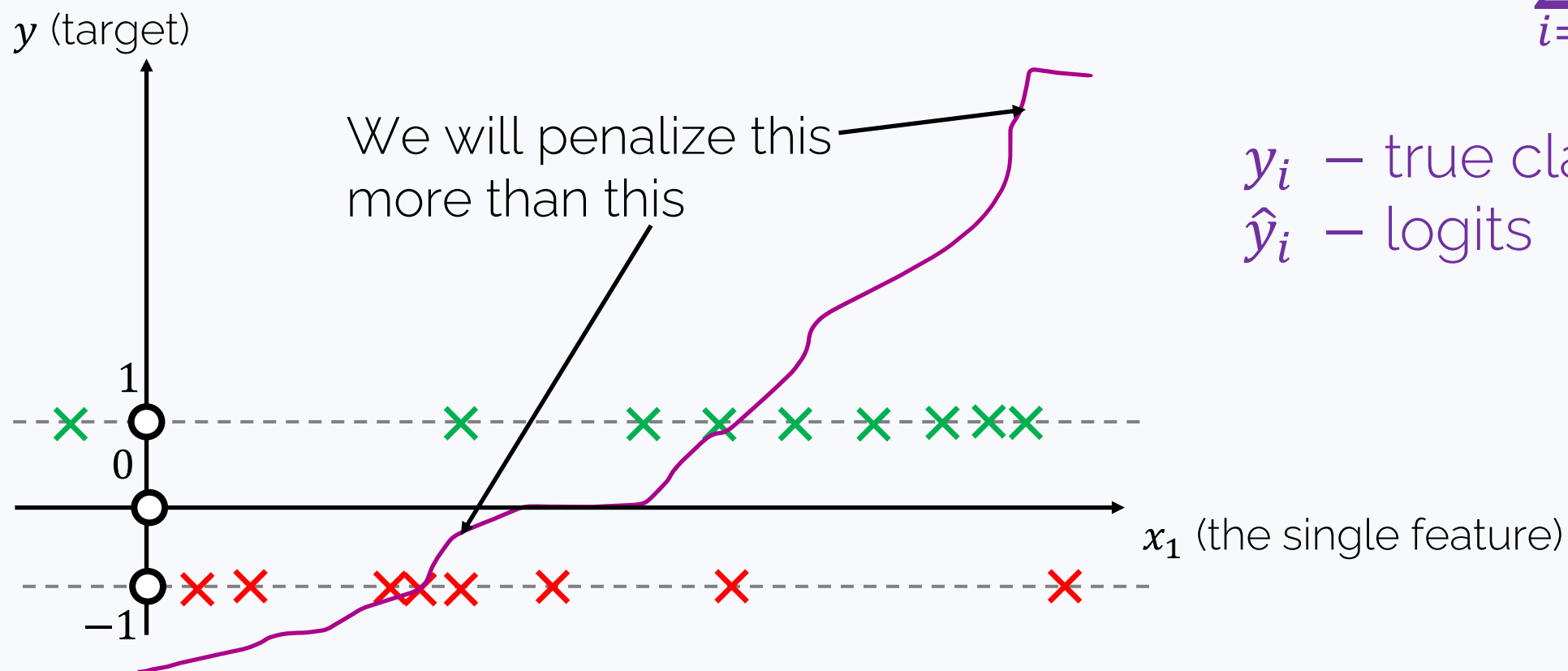
Attempt no. 1

Let class labels -1 and 1 be our target

MSE Loss

$$\mathcal{L}(y, \hat{y}) = \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

y_i – true class labels
 $\hat{y}_i$ – logits



Attempt no. 2: let's use probabilities!

Predicted class 1 probability $\hat{p} = \sigma(xw^T)$ are between 0 and 1.



Attempt no. 2: maximize the right probability

We maximize:

$$l(y, \hat{p}) = \begin{cases} \hat{p}_i, & \text{if } y_i = 1 \\ 1 - \hat{p}_i, & \text{if } y_i = 0 \end{cases}$$

y_i — true class labels

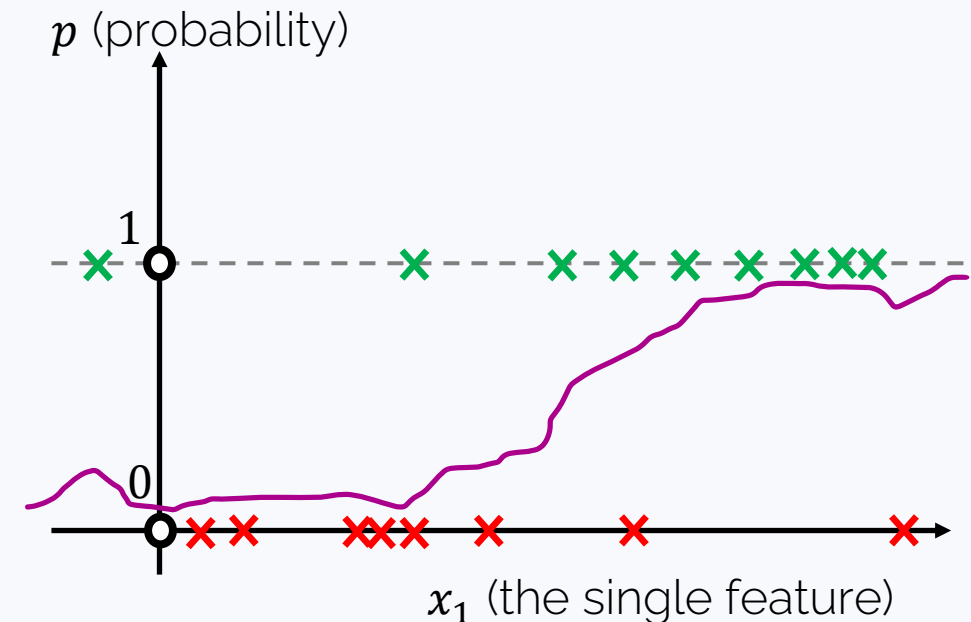
$\hat{p}_i$ — predicted probabilities

If the true class is 1, we want

$P(\text{class 1}) = \hat{p}_i$ as large as possible.

If the true class is 0, we want

$P(\text{class 0}) = 1 - \hat{p}_i$ as large as possible.



Attempt no. 2: logarithm is better

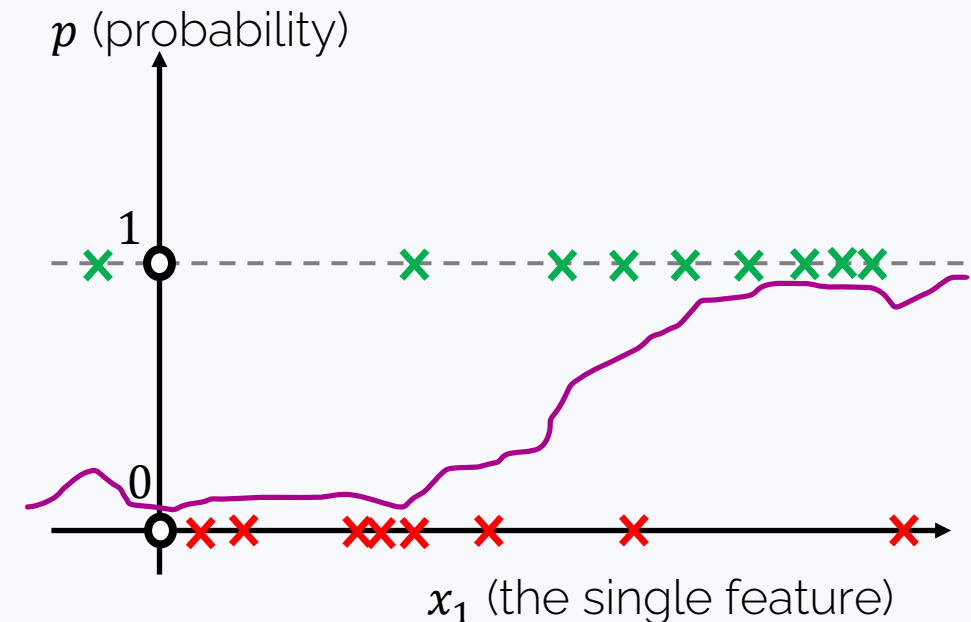
We maximize:

$$l(y_i, \hat{p}_i) = \begin{cases} \log \hat{p}_i, & \text{if } y_i = 1 \\ \log(1 - \hat{p}_i), & \text{if } y_i = 0 \end{cases}$$

y_i — true class labels
 $\hat{p}_i$ — predicted probabilities

If the true class is 1, we want
 $P(\text{class 1}) = \hat{p}_i$ as large as possible.

If the true class is 0, we want
 $P(\text{class 0}) = 1 - \hat{p}_i$ as large as possible.



Attempt no. 2: logarithm is better

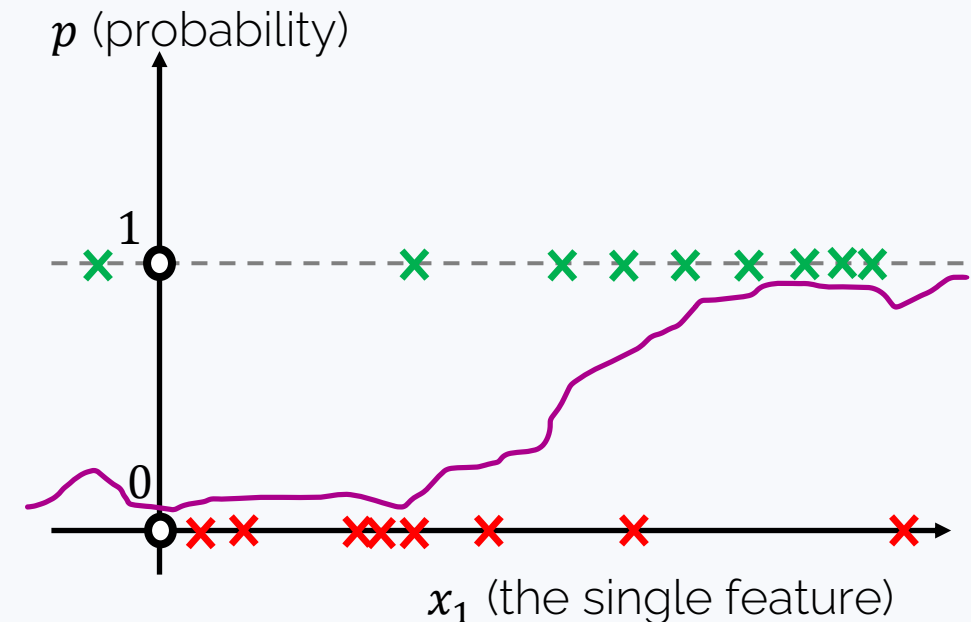
We **minimize**:

$$l(y_i, \hat{p}_i) = \begin{cases} -\log \hat{p}_i, & \text{if } y_i = 1 \\ -\log(1 - \hat{p}_i), & \text{if } y_i = 0 \end{cases}$$

y_i — true class labels
 $\hat{p}_i$ — predicted probabilities

If the true class is 1, we want
 $P(\text{class 1}) = \hat{p}_i$ as large as possible.

If the true class is 0, we want
 $P(\text{class 0}) = 1 - \hat{p}_i$ as large as possible.

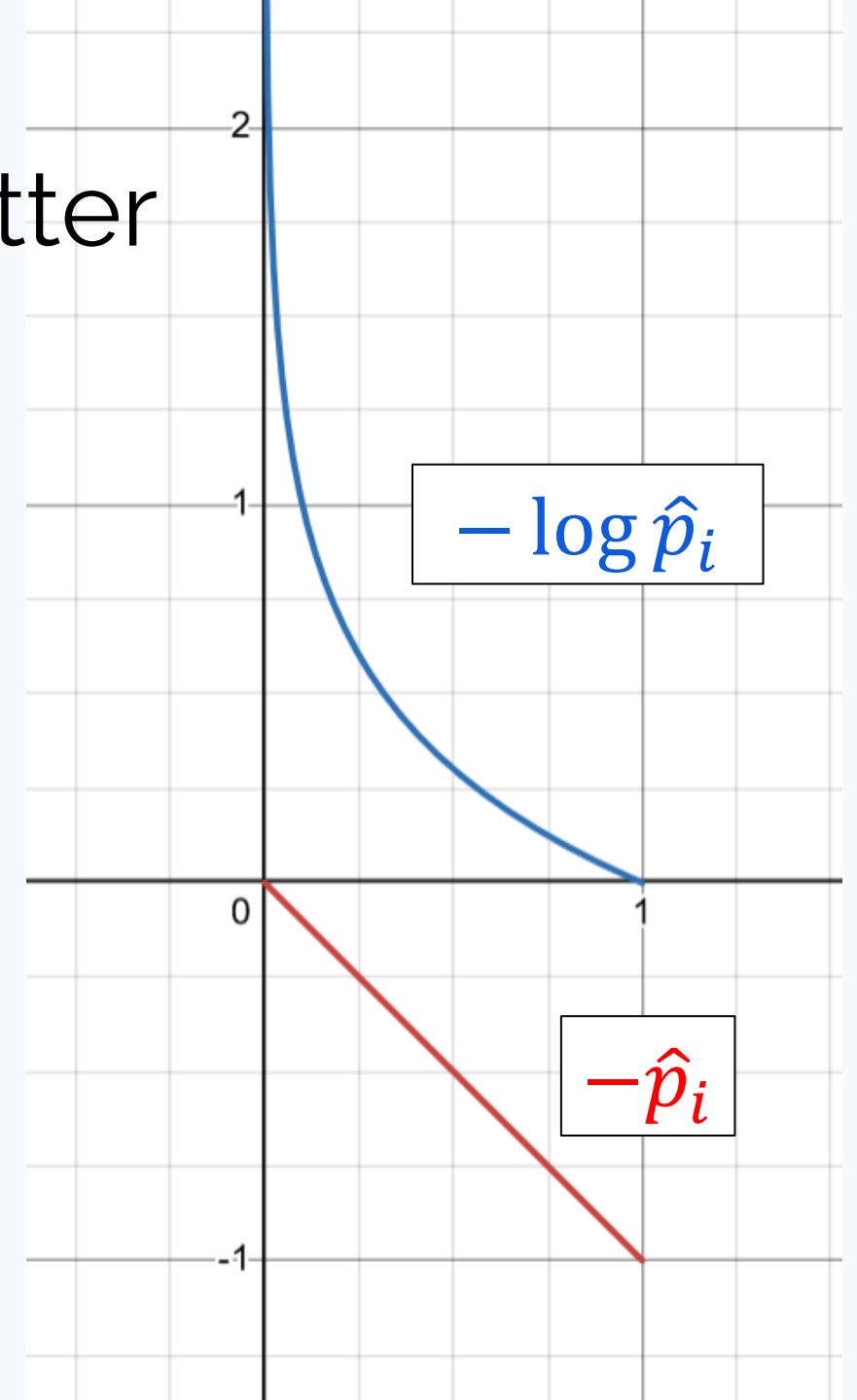


Attempt no. 2: logarithm is better

We maximize:

$$l(y_i, \hat{p}_i) = \begin{cases} \log \hat{p}_i, & \text{if } y_i = 1 \\ \log(1 - \hat{p}_i), & \text{if } y_i = 0 \end{cases}$$

y_i — true class labels
 $\hat{p}_i$ — predicted probabilities



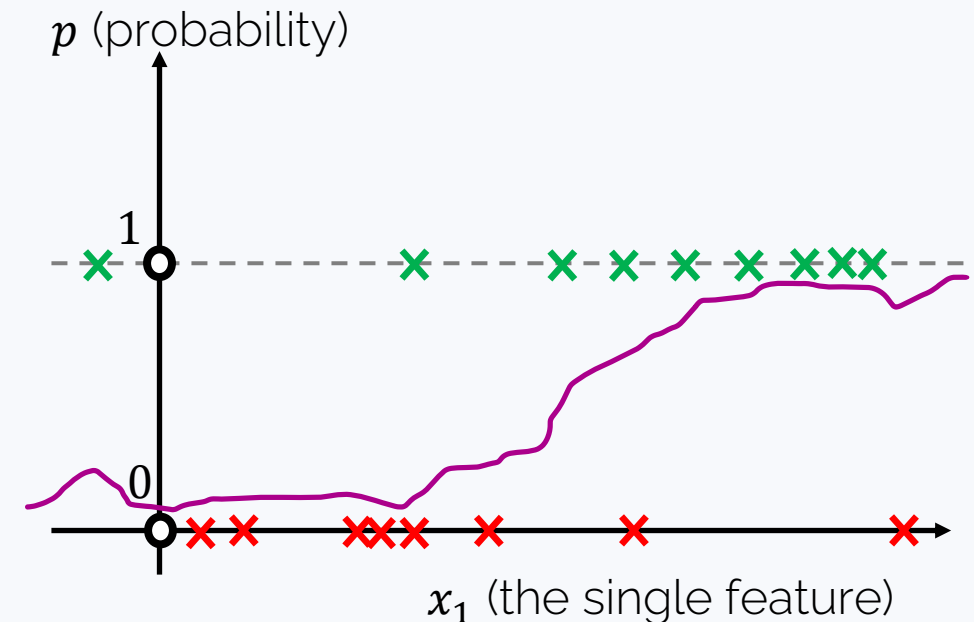
Attempt no. 2: Cross-entropy loss

We **minimize**:

$$\mathcal{L}(y_i, \hat{p}_i) = -y_i \log \hat{p}_i - (1 - y_i) \log(1 - \hat{p}_i)$$

$$l(y_i, \hat{p}_i) = \begin{cases} -\log \hat{p}_i, & \text{if } y_i = 1 \\ -\log(1 - \hat{p}_i), & \text{if } y_i = 0 \end{cases}$$

y_i — true class labels
 $\hat{p}_i$ — predicted probabilities



Regularization

What is regularization?

- Countering overfitting
- Stabilizing optimization
- Imposing conditions

Multicollinearity

The worst regression task:

$$X = \begin{pmatrix} 1 & 0 \\ 1 & 0 \\ \vdots & \vdots \\ 1 & 0 \end{pmatrix}, \quad y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix}$$

$$\mathcal{L}(y, X, \mathbf{w}) = \frac{1}{N} \sum_{i=1}^N (y_i - w_1)^2$$

Solution:

$$w_1 = \text{mean}(y_i), \quad w_2 = \text{any!}$$

Multicollinearity

The worst regression task:

$$X = \begin{pmatrix} 1 & 0 \\ 1 & 0 \\ \vdots & \vdots \\ 1 & 0 \end{pmatrix}, \quad y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix}$$

$$\mathcal{L}(y, X, \mathbf{w}) = \frac{1}{N} \sum_{i=1}^N (y_i - w_1)^2$$

Solution:

$$w_1 = \text{mean}(y_i), \quad w_2 = \text{any!}$$

$$X^T X = \begin{pmatrix} N & 0 \\ 0 & 0 \end{pmatrix}$$

Not invertible!

$$\hat{\mathbf{w}}^T = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

Multicollinearity

A quite bad regression task:

$$X = \begin{pmatrix} 1 & \varepsilon \\ 1 & 0 \\ \vdots & \vdots \\ 1 & 0 \end{pmatrix}, \quad y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix}$$

ε is a very small number.

$$\hat{w}^T = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T y$$

$$X^T X = \begin{pmatrix} N & \varepsilon \\ \varepsilon & \varepsilon^2 \end{pmatrix}$$

is close to degenerate.

Multicollinearity

A quite bad regression task:

$$X = \begin{pmatrix} 1 & \varepsilon \\ 1 & 0 \\ \vdots & \vdots \\ 1 & 0 \end{pmatrix}, \quad y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix}$$

ε is a very small number.

$$\hat{w}^T = (\mathbf{X^T X})^{-1} X^T y$$

$$X^T X = \begin{pmatrix} N & \varepsilon \\ \varepsilon & \varepsilon^2 \end{pmatrix}$$

is close to degenerate.

$$(X^T X)^{-1} = \frac{1}{N-1} \begin{pmatrix} 1 & -\frac{1}{\varepsilon} \\ -\frac{1}{\varepsilon} & \frac{N}{\varepsilon^2} \end{pmatrix}$$

$$(X^T X)^{-1} X^T = \frac{1}{N-1} \begin{pmatrix} 0 & 1 & \dots \\ \frac{N-1}{\varepsilon} & -\frac{1}{\varepsilon} & \dots \end{pmatrix}$$

$$\hat{w}^T \rightarrow (X^T X)^{-1} X^T (y + \delta)$$

A small change in y may lead to a catastrophic change in $\hat{w}$.

Multicollinearity

A quite bad regression task:

$$X = \begin{pmatrix} 1 & \varepsilon \\ 1 & 0 \\ \vdots & \vdots \\ 1 & 0 \end{pmatrix}, \quad y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix}$$

$$\varepsilon = 10^{-4},$$

$$N = 100$$

$$X^T X = \begin{pmatrix} 100 & 10^{-4} \\ 10^{-4} & 10^{-8} \end{pmatrix}$$

is close to degenerate.

$$(X^T X)^{-1} = \frac{1}{99} \begin{pmatrix} 1 & -10^4 \\ -10^4 & 10^{10} \end{pmatrix} \approx \begin{pmatrix} 10^{-2} & -10^2 \\ -10^2 & 10^8 \end{pmatrix}$$

$$(X^T X)^{-1} X^T \approx \begin{pmatrix} 0 & 10^{-2} & \cdots \\ 10^4 & 10^2 & \cdots \end{pmatrix}$$

$$\hat{w}^T \rightarrow \begin{pmatrix} 0 & 10^{-2} & \cdots \\ 10^4 & 10^2 & \cdots \end{pmatrix} (y + 10^{-2})$$

A small change in y may lead to a catastrophic change in $\hat{w}$.

Multicollinearity

A quite bad regression task:

$$X = \begin{pmatrix} 1 & \varepsilon \\ 1 & 0 \\ \vdots & \vdots \\ 1 & 0 \end{pmatrix}, \quad y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix}$$

$$\varepsilon = 10^{-4}, \\ N = 100$$

$$\hat{w}^T = (\mathbf{X}^T \mathbf{X})^{-1} X^T y$$

$$X^T X + 10^{-4} I = \begin{pmatrix} 100 + 10^{-4} & 10^{-4} \\ 10^{-4} & 10^{-8} + 10^{-4} \end{pmatrix}$$

$$\begin{aligned} & (X^T X + 10^{-4} I)^{-1} X^T \\ & \approx \begin{pmatrix} 10^{-2} & 10^{-2} & \dots \\ 1 & -10^2 & \dots \end{pmatrix} \end{aligned}$$

That's much better!

L2-regularization

How to fix an almost-degenerate matrix:

$$\hat{w}^T = (X^T X + \lambda I)^{-1} X^T y$$

L2-regularization

How to fix an almost-degenerate matrix:

$$\hat{w}^T = (X^T X + \lambda I)^{-1} X^T y$$

We can prove that this corresponds to the following task:

$$\mathcal{L}(y, X, w) = \left[\sum_{i=1}^N (y_i - x_i w^T)^2 \right] + \lambda \|w\|_2^2$$

where $\|w\|_2^2 = w_1^2 + \dots + w_D^2$

L2-regularization in general case

$$\mathcal{L}_{reg}(y, X, \mathbf{w}) = \mathcal{L}(y, X, \mathbf{w}) + \lambda \|\mathbf{w}\|_2^2$$

where $\|\mathbf{w}\|_2^2 = w_1^2 + \dots + w_D^2$

Theory says that optimizing this new loss is equivalent to optimizing $\mathcal{L}(y, X, \mathbf{w})$ inside some $\{\|\mathbf{w}\|_2 \leq R\}$

L2-regularization in general case

$$\mathcal{L}_{reg}(y, X, \mathbf{w}) = \mathcal{L}(y, X, \mathbf{w}) + \lambda \|\mathbf{w}\|_2^2$$

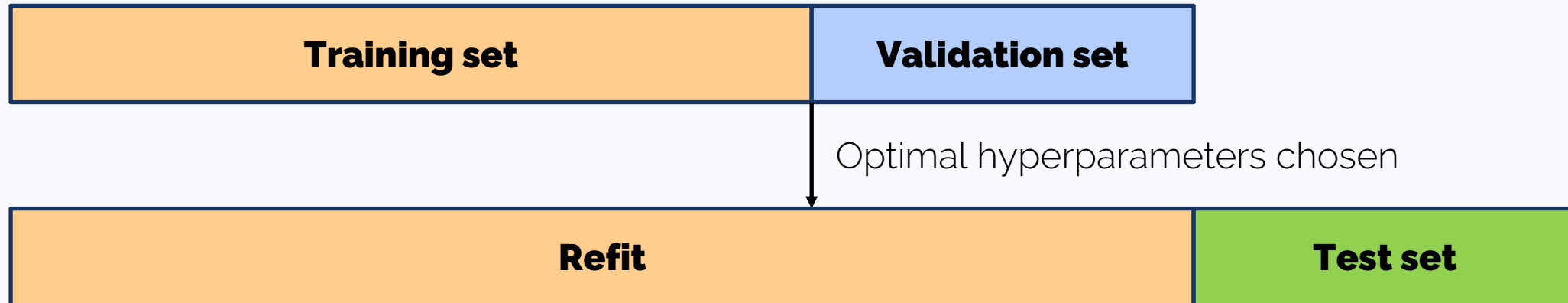
where $\|\mathbf{w}\|_2^2 = w_1^2 + \dots + w_D^2$

Helps not to overfit on noisy features.

λ is a **hyperparameter**. It is tuned on a logarithmic scale.

Hyperparameter search

1. The **training set** is used to train the model.
2. The **validation set** is used to tune hyperparameters and select the best model.
3. The **test set** is used for a final evaluation of the model's performance

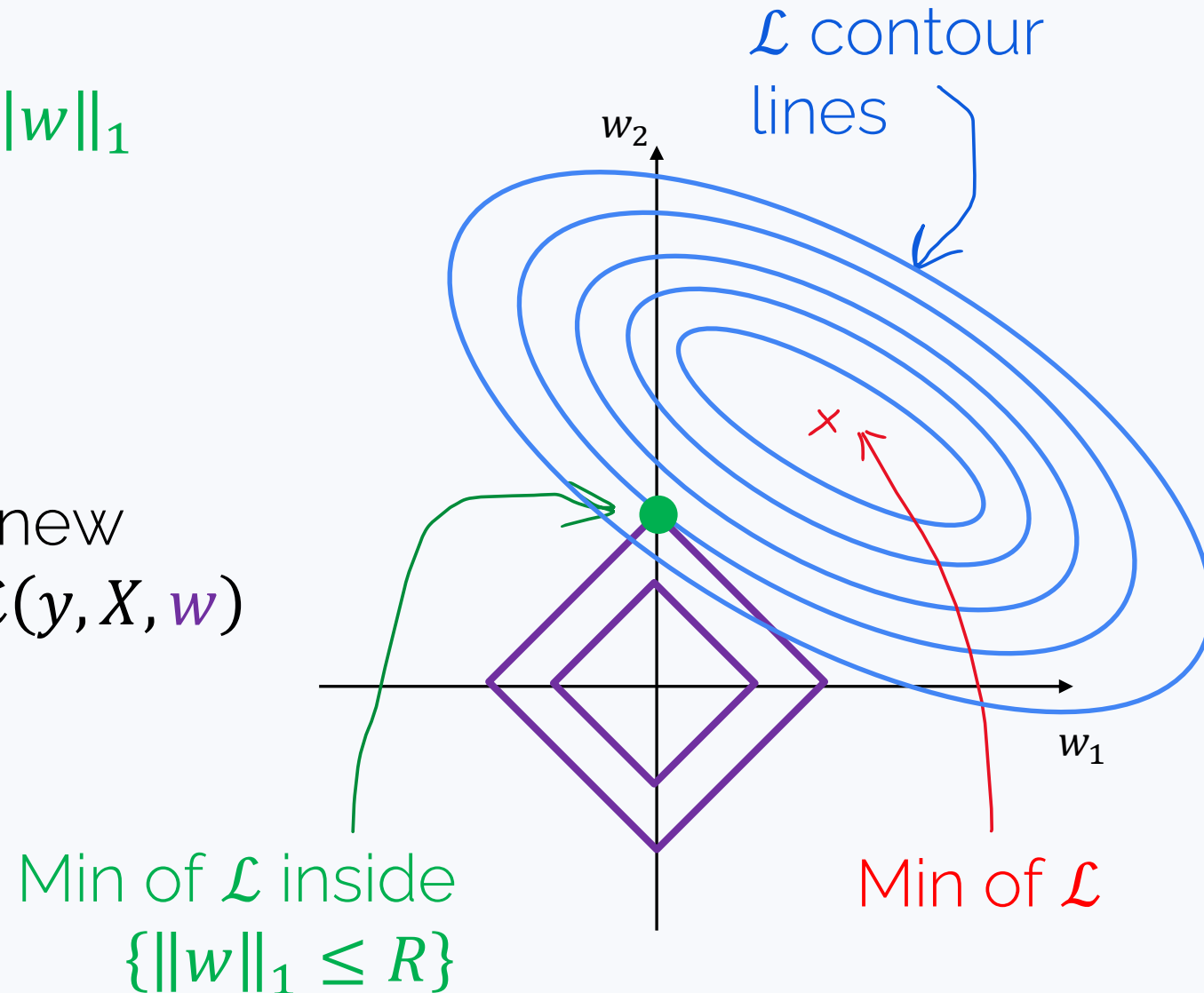


L1-regularization in general case

$$\mathcal{L}_{reg}(y, X, \mathbf{w}) = \mathcal{L}(y, X, \mathbf{w}) + \lambda \|\mathbf{w}\|_1$$

where $\|\mathbf{w}\|_1 = |w_1| + \dots + |w_D|$

Theory says that optimizing this new loss is equivalent to optimizing $\mathcal{L}(y, X, \mathbf{w})$ inside some $\{\|\mathbf{w}\|_1 \leq R\}$



L1-regularization leads to sparsity

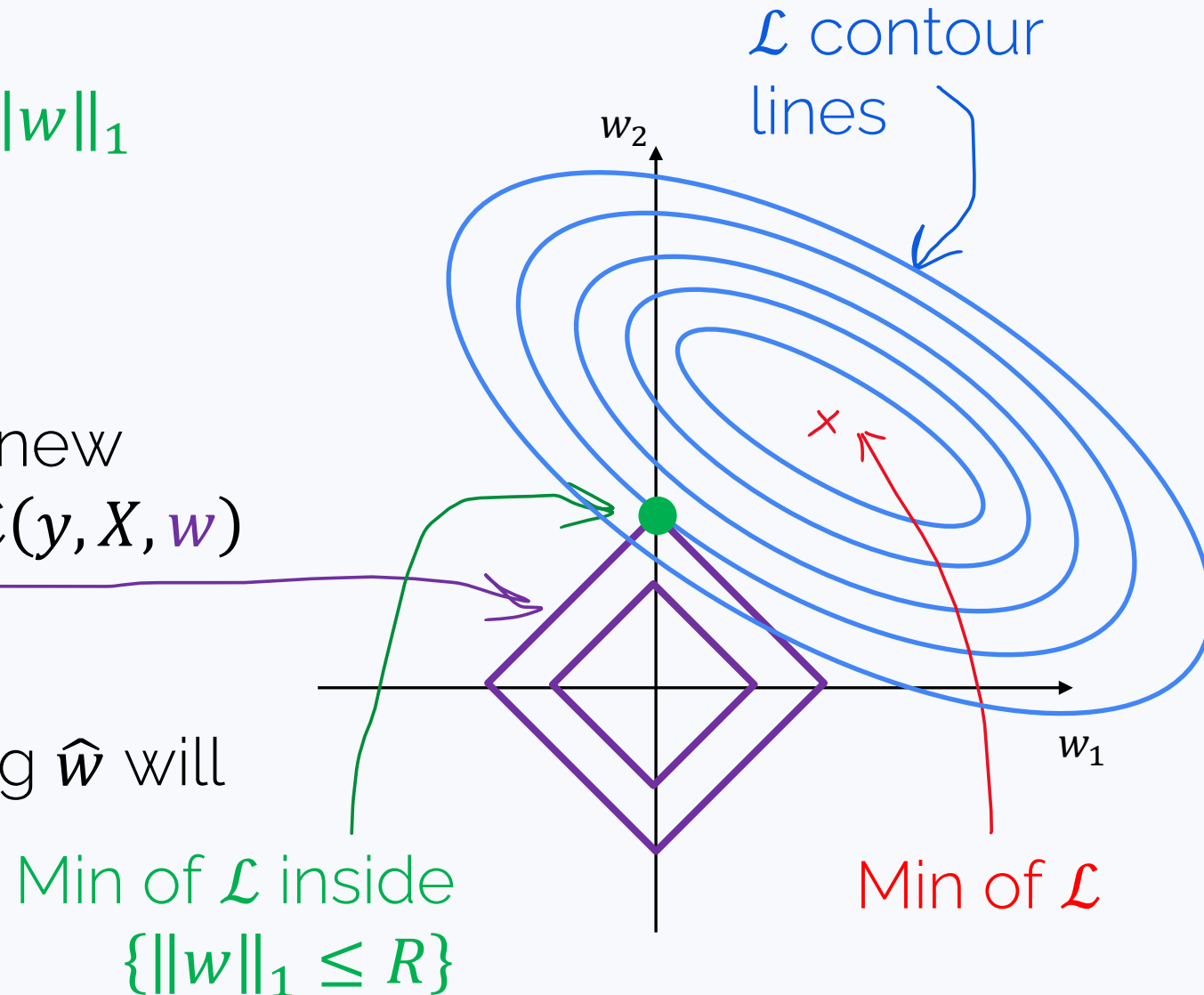
$$\mathcal{L}_{reg}(y, X, \mathbf{w}) = \mathcal{L}(y, X, \mathbf{w}) + \lambda \|\mathbf{w}\|_1$$

where $\|\mathbf{w}\|_1 = |w_1| + \dots + |w_D|$

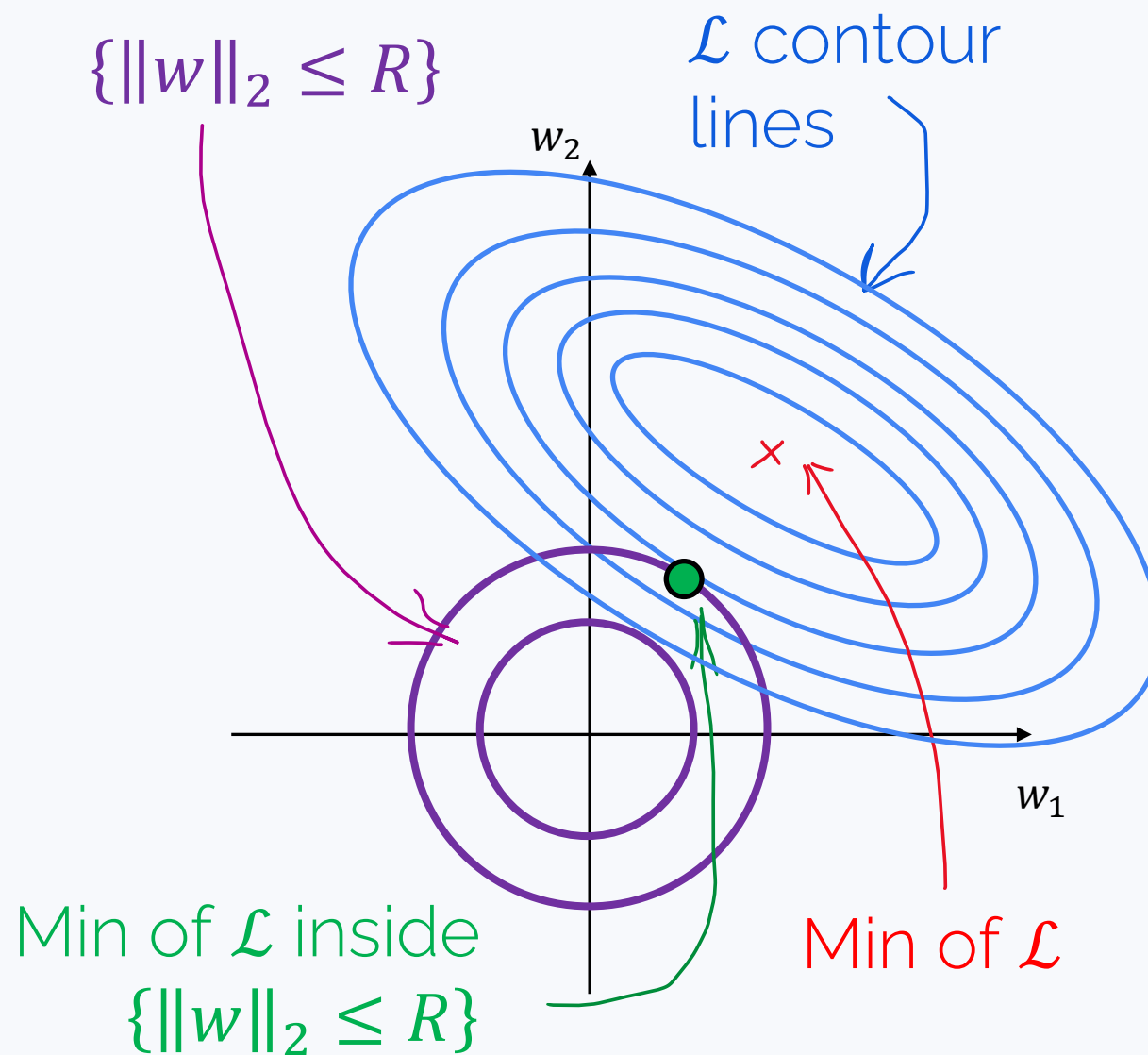
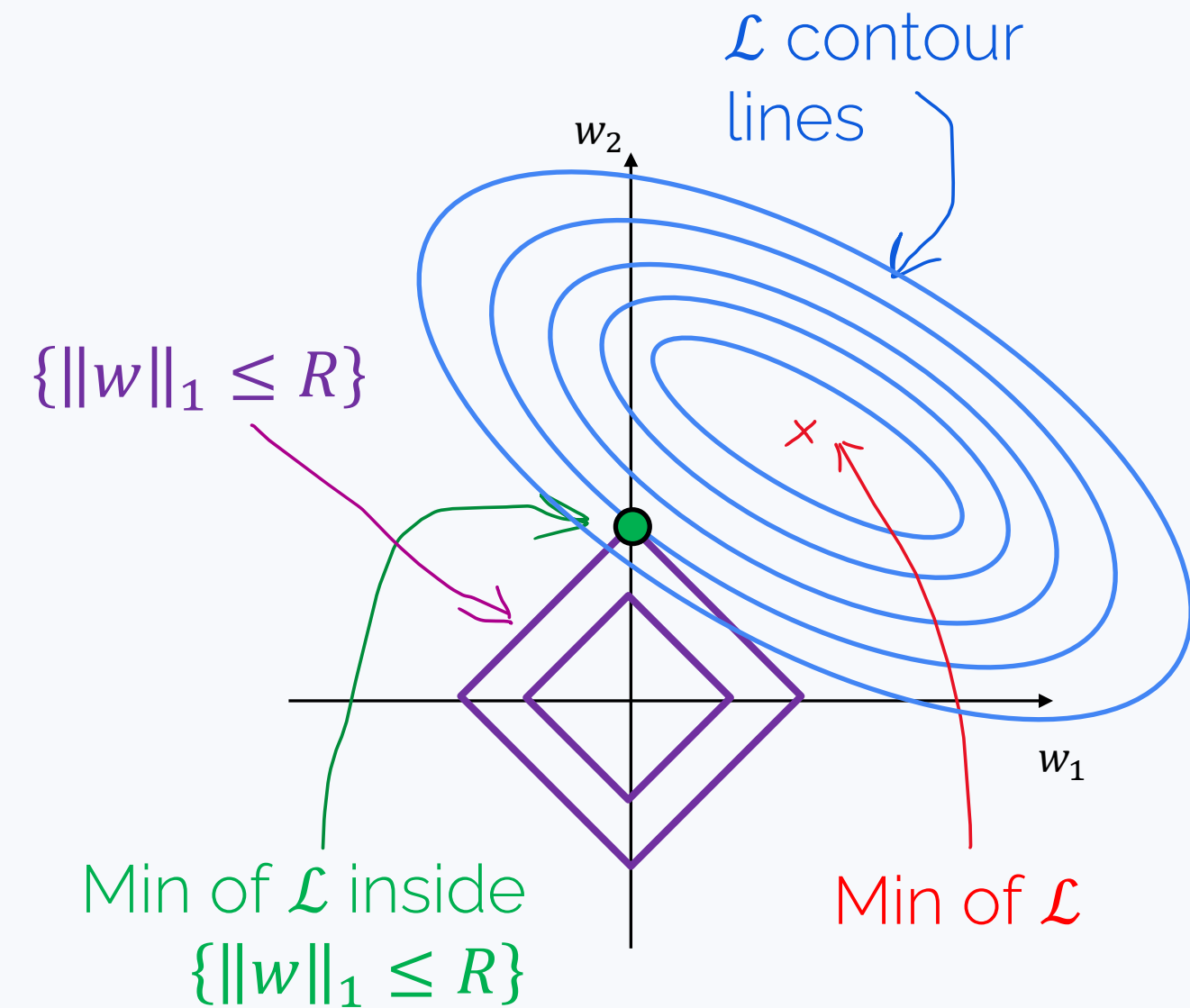
Theory says that optimizing this new loss is equivalent to optimizing $\mathcal{L}(y, X, \mathbf{w})$ inside some $\{\|\mathbf{w}\|_1 \leq R\}$

Many coordinates of the resulting $\hat{\mathbf{w}}$ will likely be zero.

The larger λ , the more.



Compare: L1 vs L2



L1-regularization: why sparsity?

$$\mathcal{L}_{reg}(y, X, \mathbf{w}) = \mathcal{L}(y, X, \mathbf{w}) + \lambda \|\mathbf{w}\|_1$$

where $\|\mathbf{w}\|_1 = |w_1| + \dots + |w_D|$

Many coordinates of the resulting $\hat{\mathbf{w}}$ will likely be zero.

Why bother? There's hope that we'll get rid of noise and irrelevant features. Counters overfitting.

